[Наименование министерства / ведомства]
[Полное наименование образовательной организации]

[Факультет]
[Кафедра]

«Допустить к защите»
Заведующий кафедрой
[звание, степень]
_____ [И.О. Фамилия]
«___» _____ 20__ г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Пояснительная записка
ВКР.000000.000ПЗ

на тему:

«Повышение оперативности анализа данных об аварийных отключениях электрической сети на основе её топологической модели в ГИС QGIS (на примере электросети Норвегии)»

[Направление подготовки / специальность: код – наименование]

Обучающийся: [группа] _____ [И.О. Фамилия]

Руководитель: [должность, звание, степень] _____ [И.О. Фамилия]

Консультант: [при наличии] _____ [И.О. Фамилия]

Нормоконтролёр: _____ [И.О. Фамилия]

[Город] – 20__

Содержание

Введение.....4
1 Анализ задачи оперативного анализа данных об аварийных отключениях электрической сети.....8
1.1 Анализ роли и места анализа аварийных отключений в системе оперативно-диспетчерского управления.....8
1.2 Теоретические основы геометрической топологии в геоинформационных системах.....12
1.3 Сравнительный анализ геоинформационных платформ для построения и анализа инженерных сетей.....16
1.4 Проблема вендорной зависимости и выбор формата данных.....17
1.5 Анализ исходных данных электросети Норвегии.....19
1.6 Постановка задачи исследования.....21
Выводы по 1 разделу.....23
2 Разработка моделей и методики оперативного анализа аварийных отключений.....25
2.1 Математическое представление электрической сети в виде графа.....25
2.2 Модель геометрического допуска и правила построения топологической модели.....28
2.3 Модель определения обесточенных участков сети.....36
2.4 Инкрементальный алгоритм определения обесточенных участков для сетей большой размерности.....39
2.5 Модель показателей оперативности анализа.....41
2.6 Методика оперативного анализа аварийных отключений.....43
Выводы по 2 разделу.....44
3 Разработка программного комплекса и оценивание оперативности анализа.....46
3.1 Общее описание программы.....46

3.2 Схемы алгоритмов программы и обеспечение оперативности.....49
3.3 Руководство оператора.....51
3.4 Результаты решения контрольной задачи.....53
3.5 Применение программного комплекса для решения прикладных задач.....59
3.6 Оценивание оперативного и технико-экономического эффекта.....60
Выводы по 3 разделу.....64
Заключение.....66
Перечень сокращений и условных обозначений.....69
Приложение А. Исходный текст программы outage_qgis.py.....70
Приложение Б. Руководство оператора.....91
Приложение В. Протокол построения топологической модели электросети Норвегии.....93
Список использованной литературы.....94

Введение

Современный этап развития электроэнергетики характеризуется переходом сетевых организаций к цифровому управлению сетями: пространственные данные о линиях электропередачи, подстанциях и опорах ведутся в геоинформационных системах, а требования к надёжности электроснабжения закреплены Федеральным законом «Об электроэнергетике» [1] и Правилами оперативно-диспетчерского управления [2]. Показатели надёжности – средняя продолжительность ($П_{SAIDI}$) и средняя частота ($П_{SAIFI}$) прекращений передачи электроэнергии – являются регулируемыми и учитываются при установлении тарифов сетевых организаций [3]. Каждое аварийное отключение линии или подстанции требует от диспетчера в кратчайшее время определить, какие участки сети и какие потребители остались без напряжения, какие схемы резервирования можно задействовать и куда направить ремонтные бригады.

Первостепенную роль в получении такой информации играют геоинформационные технологии. Открытые наборы данных, данные государственных информационных систем и эксплуатационные схемы предоставляют возможности для моделирования инженерных сетей любого масштаба: так, Норвежское управление водных ресурсов и энергетики публикует единый набор данных обо всех электросетевых объектах страны – около 155 тыс. линий и 734 тыс. опор [4, 5]. Однако такие данные представляют собой «картографическую» модель: линии нарисованы, но не связаны топологически, содержат разрывы, дубликаты и ответвления без узлов, из-за чего автоматический расчёт связности по ним невозможен, а ручная проверка и исправление требуют огромных временных затрат [6, 7].

Таким образом, имеет место проблемная ситуация, заключающаяся, **с одной стороны**, в стремительном росте объёма и доступности пространственных данных об электрических сетях и в требовании сокращать продолжительность перерывов электроснабжения, а **с другой стороны** – в том, что анализ последствий аварийных отключений в большинстве сетевых организаций по-прежнему выполняется оператором вручную по схемам, занимает от минут до часов и не может быть

выполнен для всех возможных отказов в сети большой размерности. Существующие коммерческие решения (ArcGIS Network Analyst, ArcGIS Utility Network, ГИС Zulu) либо дороги, либо привязаны к проприетарным форматам данных, что порождает проблему вендорной зависимости [8, 9].

Разрешение данной проблемной ситуации определило тему выпускной квалификационной работы: «Повышение оперативности анализа данных об аварийных отключениях электрической сети на основе её топологической модели в ГИС QGIS (на примере электросети Норвегии)».

Объектом исследования является процесс анализа данных об аварийных отключениях элементов электрической сети.

Предметом исследования являются методы построения топологической модели электрической сети по пространственным данным и алгоритмы оперативного определения обесточенных участков сети большой размерности.

Целью выпускной квалификационной работы является повышение оперативности анализа данных об аварийных отключениях электрической сети за счёт автоматизированного построения топологической модели сети в открытой ГИС QGIS и программного определения обесточенных участков.

Для достижения поставленной цели в выпускной квалификационной работе должны быть решены следующие **задачи**:

1) проанализировать роль и место анализа аварийных отключений в системе оперативно-диспетчерского управления, классифицировать решаемые с его помощью инженерные задачи и обосновать показатели оперативности анализа;

2) исследовать теоретические основы геометрической топологии в ГИС, классифицировать топологические ошибки и оценить топологическое качество исходных данных электрической сети;

3) провести сравнительный анализ ГИС-платформ (ArcGIS, MapInfo, GRASS GIS, QGIS), рассмотреть проблему вендорной зависимости и обосновать выбор открытых программных средств и форматов хранения данных;

4) разработать математическое представление электрической сети в виде графа и формализованные правила построения топологически корректной модели

сети, включая модель геометрического допуска и присвоение уникальных идентификаторов элементам;

5) формализовать понятие обесточенного участка и разработать алгоритм его определения, обеспечивающий оперативность анализа для сетей общенационального масштаба;

6) разработать методику оперативного анализа аварийных отключений и программный комплекс на языке Python для консоли ГИС QGIS, обеспечивающий выбор отключаемых узлов и линий, выделение отключённых и обесточенных элементов цветом и формирование отчёта, а также описание программы и руководство оператора;

7) разработать модель показателей оперативности, решить контрольную задачу на данных электросети Норвегии и оценить повышение оперативности анализа по сравнению с ручным анализом оператором, а также технико-экономический эффект.

Взаимосвязь цели, задач и разделов работы показана на рисунке 1.3 (подраздел 1.6).

Методы исследования: системный анализ, теория графов, алгоритмы обхода графов, методы вычислительной геометрии, геоинформационное моделирование, вычислительный эксперимент.

Научная новизна работы заключается в формализованном наборе правил приведения картографических данных электрической сети к топологической модели с учётом класса напряжения, параллельности линий и привязки к подстанциям; в модели классификации элементов по четырём состояниям, отделяющей последствия отключения от разрывов исходных данных; в инкрементальном алгоритме анализа, время работы которого определяется размером затронутого фрагмента, а не всей сети.

Практическая значимость состоит в создании работоспособного программного комплекса, проверенного в QGIS 3.40.3 на данных всей электросети Норвегии, который сокращает время анализа одного отключения с минут до

десятков миллисекунд и позволяет выполнить анализ N-1 для десятков тысяч элементов сети.

Решение поставленных задач начинается с анализа роли и места анализа аварийных отключений в системе оперативно-диспетчерского управления, теоретических основ топологии в ГИС, существующих программных средств и исходных данных, которому посвящён первый раздел выпускной квалификационной работы.

1 АНАЛИЗ ЗАДАЧИ ОПЕРАТИВНОГО АНАЛИЗА ДАННЫХ ОБ АВАРИЙНЫХ ОТКЛЮЧЕНИЯХ ЭЛЕКТРИЧЕСКОЙ СЕТИ

1.1 Анализ роли и места анализа аварийных отключений в системе оперативно-диспетчерского управления

Электрическая сеть представляет собой совокупность подстанций, распределительных устройств и соединяющих их линий электропередачи, предназначенную для передачи и распределения электрической энергии [10]. По назначению сети подразделяют на системообразующие и питающие сети напряжением 110–750 кВ, обеспечивающие связь электростанций и крупных узлов нагрузки, и распределительные сети напряжением 6–35 кВ и 0,4 кВ, по которым электроэнергия доставляется непосредственно потребителям [11]. Распределительные сети 6–20 кВ, как правило, выполняются по разомкнутым (радиальным или магистральным) схемам с возможностью резервирования от соседних фидеров посредством коммутационных аппаратов, что определяет характер последствий их отключений.

Аварийным отключением называют автоматическое или вынужденное оперативное отключение элемента сети, вызванное его повреждением или недопустимым режимом: коротким замыканием, обрывом провода, падением опоры, отказом оборудования подстанции. Последствием отключения является прекращение электроснабжения всех участков сети, которые после отключения потеряли электрическую связь с источниками питания. В радиальной сети отключение головного участка фидера обесточивает весь фидер; в сети с резервированием последствия зависят от наличия альтернативных путей питания [12, 13]. Отключения могут сопровождаться также ухудшением качества электроэнергии у остающихся в работе потребителей [14], однако в работе рассматривается прекращение электроснабжения как наиболее тяжёлое последствие.

Анализ последствий аварийного отключения является звеном технологического цикла оперативно-диспетчерского управления (ОДУ),

регламентированного Правилами оперативно-диспетчерского управления в электроэнергетике [2]. Место этого анализа в процессе ликвидации аварии показано на рисунке 1.1.

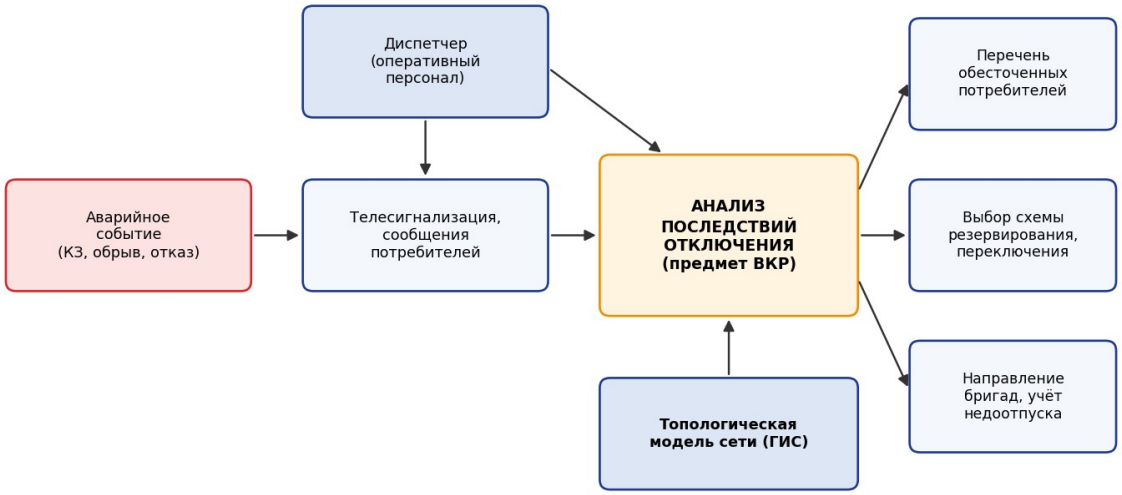


Рисунок 1.1 – Место анализа последствий отключения в процессе оперативно-диспетчерского управления

По получении сигнала об отключении (от устройств телемеханики или от потребителей) диспетчер должен:

– определить отключившийся элемент и зону обесточения;

– установить перечень обесточенных подстанций, трансформаторных пунктов и потребителей, в том числе потребителей первой категории надёжности, для которых перерыв электроснабжения допускается лишь на время автоматического восстановления питания [11, 15];

– выбрать и выполнить переключения для восстановления питания от резервных источников;

– направить ремонтный персонал к месту повреждения;

– зафиксировать продолжительность и объём прекращения передачи электроэнергии для учёта показателей надёжности.

Инженерные задачи, решаемые с помощью анализа аварийных отключений, целесообразно классифицировать по трём основаниям (таблица 1.1).

Таблица 1.1 – Классификация инженерных задач, решаемых с помощью анализа аварийных отключений

Основание	Класс задач	Требования к оперативности
По цели анализа	Оперативно-диспетчерские задачи: состав обесточенных потребителей, масштаб аварийной зоны, выбор переключений	Максимальные: решение принимается за секунды – минуты
	Задачи проектирования и развития сети: оценка надёжности схемы, поиск «узких мест», обоснование резервирования (анализ N-1)	Требуется многократный перебор тысяч вариантов отключений
	Задачи послеаварийного анализа: оценка недоотпуска электроэнергии, разработка противоаварийных мероприятий	Ретроспективный анализ больших журналов аварий
	Обучение и тренировка диспетчерского персонала	Мгновенная проверка ответа обучаемого
По типу моделируемого события	Отключение линейного элемента (ВЛ, КЛ)	Наиболее массовый вид отказа
	Отключение узлового элемента (подстанции, распределительного пункта)	Одновременно разрываются все присоединённые линии
	Каскадное отключение	Требует режимного расчёта (за рамками топологического анализа)
По категории потребителей	I категория – перерыв электроснабжения недопустим	Информация о потере питания нужна немедленно
	II категория – массовый недоотпуск продукции	Высокие
	III категория – перерыв допустим на время ремонта	Обычные

Таким образом, **целью** анализа аварийных отключений является получение полной и достоверной картины последствий отключения в минимальное время. **Задачами** анализа являются:

1) определение множества элементов сети (линий, узлов, опор, подстанций), потерявших связь с источниками питания;

2) количественная оценка масштаба обесточения – протяжённость, число участков и ответвлений, классы напряжения;

3) определение затронутых линий и потребителей для оповещения и приоритизации восстановления;

4) оценка вариантов восстановления питания путём моделирования дополнительных переключений;

5) заблаговременный анализ «что будет, если» (анализ N-1) при планировании ремонтных схем.

От продолжительности анализа непосредственно зависит продолжительность перерыва электроснабжения. Международная практика для оценки надёжности распределительных сетей использует индексы SAIDI и SAIFI стандарта IEEE 1366 [16]; в Российской Федерации аналогичные показатели $П_{SAIDI}$ и $П_{SAIFI}$ введены методическими указаниями Минэнерго России [3]. Связь показателей надёжности с анализом отключений показана в таблице 1.2.

Таблица 1.2 – Показатели надёжности электроснабжения и их связь с анализом отключений

Показатель	**Определение**	**Связь с анализом отключений**
$П_{SAIDI}$ (SAIDI)	Средняя продолжительность прекращений передачи электроэнергии на одну точку поставки за год	Уменьшается при сокращении времени локализации и восстановления
$П_{SAIFI}$ (SAIFI)	Средняя частота прекращений передачи электроэнергии на одну точку поставки за год	Зависит от числа потребителей в зоне обесточения
Недоотпуск электроэнергии ΔW	Объём электроэнергии, не переданной потребителям за время перерыва	Пропорционален мощности обесточенной нагрузки и продолжительности перерыва
Время восстановления	Интервал от отключения до восстановления питания	Включает время анализа, принятия решения и переключений

Время собственно анализа последствий входит в продолжительность перерыва электроснабжения аддитивно. Следовательно, ключевым свойством процесса анализа является его **оперативность** – способность получать полный и достоверный результат анализа за время, не превышающее допустимого для принятия решения. Для количественной оценки оперативности в работе используются следующие показатели:

– время анализа одного отключения $T_{ан}$ – интервал от задания отключённых элементов до получения полного перечня обесточенных элементов (среднее и максимальное значения);

– коэффициент повышения оперативности $K_{оп} = T_{руч} / T_{авт}$ – отношение времени ручного и автоматизированного анализа;

– вероятность своевременного анализа $P_{св} = P(T_{ан} \leq T_{доп})$ – доля аварий, анализ которых укладывается в допустимое время $T_{доп}$;

– трудоёмкость анализа N-1 – суммарное время анализа отключений всех элементов сети.

Оперативность не может оцениваться в отрыве от **достоверности**: быстрый, но неполный результат недопустим. Поэтому дополнительным ограничением является полнота выявления обесточенных участков (отношение числа найденных участков к фактическому) и отсутствие ложно обесточенных участков. Математическая модель показателей оперативности разработана в подразделе 2.5.

1.2 Теоретические основы геометрической топологии в геоинформационных системах

Геоинформационная система, согласно ГОСТ Р 52438–2005, представляет собой информационную систему, оперирующую пространственными данными [17]. В ГИС объекты электрической сети представлены векторными слоями: линии – полилиниями, подстанции и опоры – точками [18, 6].

Топология в геоинформатике – совокупность правил и отношений, описывающих взаимное пространственное расположение соседних векторных объектов: точек, линий и полигонов [19, 20]. В более строгом определении

топология представляет собой набор правил, которые вместе с инструментами редактирования позволяют моделировать геометрические отношения в базе геоданных. Топология хранит одно или несколько отношений, определяющих, как пространственные объекты одного или нескольких классов совместно используют геометрию [21, 22].

В модели топологических данных пространственные объекты представляются в виде топологических примитивов – узлов и рёбер. Для линейно-узловых сетей, каковой является электрическая сеть:

– **узел** – точка соединения двух или более рёбер либо конечная точка ребра (трансформаторная подстанция, точка ответвления, конец линии);

– **ребро** – линейный объект, соединяющий два узла (участок воздушной или кабельной линии, шина подстанции).

Топологическая корректность – свойство векторных данных, при котором их топологические отношения удовлетворяют заданным требованиям (правилам). **Топологические правила** – формальные ограничения, накладываемые на взаимное расположение объектов. Многие ГИС, включая QGIS (модуль Topology Checker), позволяют выбирать правила и проверять их выполнение [23]. Типовые правила для сетевых данных приведены в таблице 1.3.

Таблица 1.3 – Типовые топологические правила для линейно-узловых сетей

Слой	Правило (Topology Checker)	Содержание
Точечный (узлы)	Must be covered by	Точки должны лежать на объектах другого слоя (узлы – на линиях)
	Must be covered by endpoints of	Точки должны совпадать с конечными точками линий
	Must not have duplicates	Запрещены дублирующиеся точки
	Must not have invalid geometries	Геометрия должна быть валидной
Линейный (рёбра)	End points must be covered by	Концы линий должны покрываться точками слоя узлов
	Must not have dangles	Запрещены «висячие» концы
	Must not have duplicates	Запрещены дублирующиеся линии
	Must not have pseudos	Запрещены псевдоузлы (конец линии соединён только с одной другой линией)

В процессе оцифровки и обработки пространственных данных неизбежно возникают топологические ошибки. Наиболее распространённые ошибки линейных объектов приведены в таблице 1.4 и на рисунке 1.2.

Таблица 1.4 – Топологические ошибки линейных объектов в ГИС

Тип ошибки	**Описание**	**Последствие для анализа отключений**
Недолёт (undershoot)	Линия не достигает точки соединения, между линиями существует разрыв	Ложный разрыв: сеть распадается на изолированные части
Перелёт (overshoot)	Линия проходит за точку соединения, пересекая другую линию	Ложное пересечение, лишний висячий конец
Висячий узел (dangling node)	Конечная точка линии не соединена ни с одним другим объектом	Участок не связан с источником питания
Пересечение без узла	Линии пересекаются или отпайка примыкает к середине магистрали, но узла нет	Ответвление не связано с магистралью
Дублирование геометрии	Один и тот же объект присутствует в двух экземплярах	Искажение протяжённости и отчётов
Многочастный объект	Одна запись содержит несколько несвязанных частей	Неверная длина и связность
Отсутствие привязки к подстанциям	Линия заканчивается рядом с подстанцией, но не в её точке	Нет связи с источником питания



Рисунок 1.2 – Основные виды топологических ошибок линейных объектов

Топологические ошибки нарушают взаимоотношения между объектами и должны быть устранены, чтобы векторные данные можно было использовать в сетевом анализе: линии, визуально пересекающиеся или соприкасающиеся на карте, но не имеющие общей вершины, не считаются соединёнными, что делает невозможным построение связного графа и, следовательно, анализ аварийных отключений [7].

Для расчёта связности необходима **топологическая модель** – представление сети в виде графа, в котором явно заданы узлы и рёбра, а каждое ребро ссылается на свой начальный и конечный узел (атрибуты from_node и to_node) [24]. Только при наличии такой модели задача «какие участки потеряли связь с источником» сводится к стандартной задаче обхода графа [25, 26]. Топологическая модель является «мостом» между пространственными данными, которые ведёт сетевая организация, и алгоритмами расчёта связности; её качество определяет достоверность результата: ложный разрыв приведёт к ложному обесточению, а ложное соединение (например, двух параллельных цепей) – к пропуску обесточенных участков.

Отношения между линиями и узлами устанавливаются несколькими видами поиска (таблица 1.5); их выбор зависит от этапа обработки данных.

Таблица 1.5 – Виды поиска взаимосвязей между линиями и узлами

Вид поиска	**Условие**	**Этап применения**
По совпадению координат	$d(p, q) \leq \varepsilon$, p – конец линии, q – узел	Сведение концов линий (snapping)
По пространственной близости (буферный)	Конец линии в буфере радиусом ε вокруг объекта	Выявление недолётов, привязка к подстанциям и опорам
По отношению «пересечение»	Точка пересечения или примыкания без узла	Создание узлов-ответвлений
По отношению «смежность»	Списки смежности вершин графа	Анализ связности (обход графа)
По атрибутивным связям	Поля from_node, to_node ссылаются на идентификаторы узлов	Построение графа без пространственных запросов

На этапе подготовки данных доминируют пространственные методы, требующие пространственных индексов для обработки больших наборов данных; на этапе анализа – топологические и атрибутивные методы, скорость которых определяет оперативность анализа.

1.3 Сравнительный анализ геоинформационных платформ для построения и анализа инженерных сетей

Для построения топологически корректной модели инженерной сети и её анализа могут быть использованы различные ГИС-платформы.

ArcGIS (Esri) – ведущая коммерческая система, содержащая модуль Network Analyst (кратчайшие пути, зоны обслуживания, связность) и модель Utility Network для инженерных сетей с функциями трассировки [8, 27]. Топология реализуется набором правил в базе геоданных; автоматизация – на Python (ArcPy). Недостатки – высокая стоимость лицензий (топология доступна в лицензиях Standard и Advanced) и проприетарный формат хранения (база геоданных ESRI).

MapInfo Professional – коммерческая ГИС с инструментами проверки линий на разрывы и перекрытия и разбиения в точках пересечения [28]. Сетевой анализ ограничен и требует дополнительного модуля RouteFinder; формат данных TAB, язык программирования MapBasic узкоспециализирован; система работает только под Windows.

GRASS GIS – открытая ГИС с модулями v.net.* для сетевого анализа (компоненты связности, центральность, кратчайшие пути) и собственной топологической векторной моделью [29]. Система бесплатна и программируется на Python, однако сложна в освоении и ориентирована на командную строку, что затрудняет интерактивную работу диспетчера.

QGIS – свободно распространяемая (GNU GPL) кроссплатформенная ГИС с открытым исходным кодом [23]. Она содержит инструменты прилипания (snapping), проверки топологии (Topology Checker), поддерживает все распространённые форматы (Shapefile, GeoPackage, FileGDB через GDAL [30], PostGIS), встроенный интерпретатор Python с интерфейсом PyQGIS и библиотекой Qt для создания пользовательского интерфейса [31].

Помимо ГИС общего назначения, задачи связности решаются **программными комплексами расчёта режимов** (RastrWin3 [32]), расчётная схема которых не связана с пространственным положением объектов, и **специализированными ГИС инженерных сетей** (ГИС Zulu [9]), которые требуют

лицензий и перевода данных в собственную модель. В районных электрических сетях по-прежнему распространён **ручной анализ** по карте или однолинейной схеме. Сравнение платформ по ключевым критериям приведено в таблице 1.6.

Таблица 1.6 – Сравнительная характеристика ГИС-платформ и средств анализа отключений

Критерий	**ArcGIS**	**MapInfo**	**GRASS GIS**	**QGIS (базовая)**	**Ручной анализ**	**Разрабатываемый комплекс (QGIS + Python)**
Лицензия, стоимость	коммерческая, высокая	коммерческая	GNU GPL, бесплатно	GNU GPL, бесплатно	–	бесплатно
Топологическая модель	да (правила в базе геоданных)	частично	да	проверка (Topology Checker)	нет	строится автоматически
Анализ связности с несколькими источниками	да (Utility Network)	нет	частично (v.net)	нет	да	да
Отключение узлов и линий	да	нет	частично	нет	да	да
Формат данных	проприетарный (база геоданных)	проприетарный (TAB)	собственный	открытые (SHP, GPKG)	бумажная схема	открытые (GPKG, SHP, чтение FileGDB)
Программирование	Python (ArcPy)	MapBasic	Python, C	Python (PyQGIS)	–	Python
Время анализа одного отключения	секунды	–	секунды	–	минуты – часы	миллисекунды

Из анализа следует, что QGIS является оптимальной платформой для решения поставленной задачи: она сочетает бесплатность, поддержку открытых форматов, достаточный набор инструментов и встроенную среду программирования. Однако базовая QGIS не содержит средств автоматического построения топологической модели электрической сети и анализа отключений с учётом множества источников – их разработка и составляет содержание настоящей работы.

1.4 Проблема вендорной зависимости и выбор формата данных

Вендорная зависимость возникает, когда данные создаются и хранятся в форматах, полноценно поддерживаемых только одним производителем.

Организация, начавшая использовать определённое программное обеспечение, оказывается зависимой от его лицензионной политики, стоимости сопровождения и дальнейшей поддержки, а перенос данных в другую среду требует конвертации с риском потери топологии и атрибутов. Для сетевых организаций риск усиливается длительным (десятилетия) сроком жизни пространственных данных о сетях. Характеристика форматов пространственных данных приведена в таблице 1.7.

Таблица 1.7 – Характеристика форматов хранения пространственных данных

Формат	**Спецификация**	**Достоинства**	**Ограничения**
ESRI File Geodatabase	закрытая	Большие объёмы, топология ArcGIS	Запись только средствами Esri; чтение – драйвер OpenFileGDB GDAL
MapInfo TAB	частично открытая	Распространён в России	Привязан к MapInfo, ограниченная поддержка топологии
ESRI Shapefile	открытая [33]	Поддерживается всеми ГИС	Один слой в наборе файлов, имена полей до 10 символов, размер до 2 ГБ, нет индекса по атрибутам
OGC GeoPackage	открытый стандарт OGC [34]	Один файл SQLite, несколько слоёв, пространственный индекс R-tree, SQL-запросы	Требует поддержки SQLite (имеется во всех ГИС и в Python)

Для исключения вендорной зависимости в работе приняты следующие решения:

1) исходные данные читаются непосредственно из файловой геобазы NVE открытым драйвером OpenFileGDB [30] или из набора Shapefile, без использования ПО Esri;

2) топологическая модель сохраняется в открытом формате **GeoPackage**, который, в отличие от Shapefile, хранит в одном файле слои рёбер, узлов и опор, поддерживает длинные имена полей, пространственный индекс R-tree и индексы по атрибутам, что необходимо для оперативного выбора 734 тыс. опор по номеру линии; при необходимости слои экспортируются в Shapefile стандартными средствами QGIS;

3) вычислительная логика реализуется на языке **Python** [35] без сторонних библиотек (NetworkX [36], GeoPandas и т. п.), не входящих в стандартную поставку QGIS: граф, пространственные индексы и алгоритмы обхода реализованы в самой программе, что обеспечивает запуск на любом компьютере с QGIS без установки дополнительных модулей.

Вся цепочка инструментов – QGIS, GeoPackage, Python – является открытой, что полностью исключает вендорную зависимость.

1.5 Анализ исходных данных электросети Норвегии

Исследуемый набор – открытые данные Норвежского управления водных ресурсов и энергетики NVE «Nettanlegg» [4, 5] – охватывает электрические сети всей Норвегии и распространяется в формате FileGDB в системе координат EPSG:25833 (ETRS89 / UTM 33N). Выбор общенационального набора обусловлен тем, что он является одним из немногих открытых наборов данных, содержащих полную сеть 0,4–420 кВ с атрибутами напряжения, и позволяет проверить оперативность анализа на сети наибольшей размерности. Состав набора приведён в таблице 1.8.

Таблица 1.8 – Состав общенационального набора данных NVE «Nettanlegg»

Слой	Тип	Объектов	Содержание
el_luftlinje	линия	145 416	Воздушные линии 0,4–420 кВ
el_hengekabel	линия	645	Подвесные кабели
el_sjokabel	линия	8 445	Подводные (морские) кабели, в том числе межгосударственные вставки постоянного тока
el_transformatorstasjon	точка	2 090	Трансформаторные и распределительные станции
el_mast	точка	734 293	Опоры линий

Требования к исходным данным, сформулированные на основе анализа топологических ошибок и возможностей формата GeoPackage, и оценка их выполнения приведены в таблице 1.9.

Таблица 1.9 – Требования к исходным пространственным данным и их выполнение для набора NVE

Группа	Требование	Выполнение для набора NVE
Система координат	Единая прямоугольная проекция в метрах	Выполняется (EPSG:25833); допуски задаются в метрах
Геометрия линий	LineString / MultiLineString, ненулевая длина	Частично: 475 многочастных объектов, 86 вырожденных частей
Атрибуты линий	Напряжение, наименование, уровень сети, владелец	Выполняется, кроме кабелей постоянного тока (нет напряжения)
Топологическая согласованность	Концы соединяемых линий совпадают; нет дубликатов	Не выполняется: 72 245 висячих концов, 554 дубликата
Полнота	Наличие всех элементов путей питания	Не выполняется: нет подземных кабелей и электростанций
Идентификация	Уникальные идентификаторы, поля from_node, to_node	Поля начального и конечного узла отсутствуют

Проверка топологической согласованности показала, что 154 506 линейных объектов содержат 155 839 частей и, соответственно, 311 678 концов, которые образуют 178 982 различные точки; из них 72 245 точек являются «висячими» – не совпадают ни с одним другим концом. Характеристика линий по классам напряжения (после построения топологии) приведена в таблице 1.10.

Таблица 1.10 – Характеристика линий электросети Норвегии по классам напряжения

Класс напряжения, кВ	Число участков (рёбер)	Протяжённость, км
6–49	140 940	59 255,5
50–109	2 313	11 381,5
110–299	1 917	14 304,0
300–420	389	10 123,0
до 6 и не указано	4 286	2 460,7
Итого	149 845	97 524,6

Распределительная сеть 6–49 кВ составляет 94 % участков и 61 % протяжённости при средней длине участка около 420 м, тогда как у линий 300–420 кВ – около 26 км. Следовательно, основная доля трудоёмкости анализа приходится на разветвлённые распределительные сети. Размерность задачи – около 155 тыс.

линейных объектов и 734 тыс. опор – исключает ручное исправление топологии и ручной анализ связности и предъявляет высокие требования к вычислительной эффективности как построения топологии, так и расчёта последствий каждого отключения. Для отладки методики использовался также фрагмент набора (район Гримстад–Арендал: 1804 линии, 30 подстанций, 6394 опоры).

1.6 Постановка задачи исследования

На основании проведённого анализа сформулирована научно-практическая задача исследования.

Дано:

– пространственные данные электрической сети общенационального масштаба (около 155 тыс. линий, 2 тыс. подстанций, 734 тыс. опор) в формате FileGDB: векторные слои линий с атрибутами напряжения и наименования, точечные слои подстанций и опор в единой прямоугольной системе координат;

– ГИС QGIS 3.x со встроенным интерпретатором Python;

– множество отключаемых линий E_{off} и узлов V_{off}, задаваемое оператором интерактивно.

Требуется:

1) разработать правила и алгоритм автоматического приведения исходных данных к топологической модели «узлы – рёбра» с выявлением источников питания;

2) разработать модель и алгоритм определения обесточенных элементов сети при отключении произвольного множества линий и узлов;

3) разработать методику оперативного анализа и программный комплекс, обеспечивающий выбор отключаемых элементов на карте, выделение отключённых и обесточенных элементов цветом и формирование отчёта;

4) оценить повышение оперативности анализа по сравнению с ручным методом.

Ограничения и допущения:

– анализ является топологическим: определяется наличие связи с источником, без расчёта токов, напряжений и допустимой загрузки резервных линий;

– коммутационные аппараты в исходных данных отсутствуют, отключение моделируется выводом из работы ребра или узла целиком;

– источниками питания считаются трансформаторные станции и точки выхода линий напряжением 110 кВ и выше за границу рассматриваемого района.

Требования к оперативности и достоверности: время отклика на изменение множества отключений $T_{авт} \leq 1$ с для сети общенационального масштаба; вероятность своевременного анализа $P_{св} = 1$ при $T_{доп} = 5$ мин; полнота выявления обесточенных участков 100 % при отсутствии ложно обесточенных участков.

Взаимосвязь цели, частных задач и разделов работы показана на рисунке 1.3, а соответствие задач, подразделов и ожидаемых результатов – в таблице 1.11.



Рисунок 1.3 – Взаимосвязь цели, частных задач и разделов выпускной квалификационной работы

Таблица 1.11 – Соответствие частных задач, подразделов и результатов работы

№	Частная задача	Подраздел	Результат
1	Роль и место анализа отключений в ОДУ, классы задач, показатели оперативности	1.1	Классификация задач, система показателей оперативности
2	Теоретические основы топологии, топологические ошибки, качество исходных данных	1.2, 1.5	Классификация ошибок, оценка набора NVE
3	Сравнение ГИС-платформ, вендорная зависимость, выбор средств и форматов	1.3, 1.4	Обоснование QGIS + Python + GeoPackage
4	Графовое представление сети, правила топологии, модель допуска, идентификаторы	2.1, 2.2	Графовая модель, 10 правил, топологическая модель сети Норвегии
5	Формализация обесточенного участка, алгоритм его определения	2.3, 2.4	Модель четырёх состояний, инкрементальный алгоритм
6	Методика и программный комплекс для консоли QGIS, описание и руководство оператора	2.6, 3.1–3.3, прил. А, Б	Методика из шести этапов, программа outage_qgis.py
7	Модель показателей оперативности, контрольная задача, оценка эффекта	2.5, 3.4–3.6	Результаты сценариев, N-1, $K_{оп}$, $P_{св}$, экономический эффект

Выводы по 1 разделу

Анализ последствий аварийных отключений является обязательным звеном оперативно-диспетчерского управления; его продолжительность непосредственно входит в продолжительность перерыва электроснабжения и влияет на регулируемые показатели надёжности. Решаемые с его помощью задачи классифицированы по цели, типу события и категории потребителей; для оценки оперативности обоснованы показатели: время анализа, коэффициент повышения оперативности, вероятность своевременного анализа и трудоёмкость анализа N-1 (задача 1).

Рассмотрены теоретические основы геометрической топологии в ГИС, топологические правила и ошибки линейных объектов. Показано, что общенациональный набор данных NVE (154 506 линий, 72 245 висячих концов, 554 дубликата, отсутствие полей from_node и to_node) не является топологически

корректным, поэтому анализ отключений по нему невозможен без автоматизированного построения топологии (задача 2).

Сравнительный анализ ГИС-платформ показал, что коммерческие системы ArcGIS и MapInfo порождают вендорную зависимость, GRASS GIS неудобна для интерактивной работы, а базовая QGIS не содержит средств анализа отключений. Обоснован выбор открытой цепочки QGIS – Python – GeoPackage с непосредственным чтением FileGDB без ПО Esri (задача 3).

Поставлена задача разработки моделей, методики и программного комплекса оперативного анализа аварийных отключений с требованием времени отклика не более 1 с для сети общенационального масштаба.

2 РАЗРАБОТКА МОДЕЛЕЙ И МЕТОДИКИ ОПЕРАТИВНОГО АНАЛИЗА АВАРИЙНЫХ ОТКЛЮЧЕНИЙ

2.1 Математическое представление электрической сети в виде графа

Топологическая модель электрической сети задаётся неориентированным мультиграфом [25, 37]

$$G = (V, E), \tag{2.1}$$

где V – конечное множество узлов (подстанции, точки соединения и ответвления линий, концы линий); E – множество рёбер (участки линий, шины подстанций, условные вводы). Направление перетока мощности при анализе связности не учитывается, поэтому рёбра неориентированы; кратные рёбра допускаются (параллельные цепи между одними и теми же узлами).

Каждое ребро $e \in E$ задаётся кортежем

$$e = (id, u, v, kind, U, l, P), \tag{2.2}$$

где id – уникальный идентификатор; $u, v \in V$ – начальный и конечный узлы (атрибуты from_node, to_node); $kind \in \{LINE, BUS, VIRT\}$ – вид ребра (линия, шина подстанции, условный кабельный ввод); U – номинальное напряжение, кВ; l – длина, м; $P = (p_1, \ldots, p_k)$ – геометрия ребра, причём p_1 совпадает с координатами узла u, а p_k – узла v.

Множество источников питания $S \subseteq V$ включает узлы-подстанции (тип STATION) и узлы внешнего питания (тип EXTERNAL). Тип остальных узлов определяется степенью deg(v): END при deg(v) = 1, PASS при deg(v) = 2, BRANCH при deg(v) ≥ 3.

Структура графа формально описывается **матрицей смежности** A размером n × n, где n = |V| [38, 39]:

$$a_{ij} = 1, \text{ если существует ребро } (v_i, v_j) \in E; \quad a_{ij} = 0 - \text{иначе}. \tag{2.3}$$

Для неориентированного графа матрица симметрична ($a_{ij} = a_{ji}$), на главной диагонали – нули, так как петли отсутствуют. Соседями вершины v_i являются все вершины v_j, для которых $a_{ij} = 1$.

Пример графа сети с одним источником И1 приведён на рисунке 2.1, его матрица смежности – в таблице 2.1.

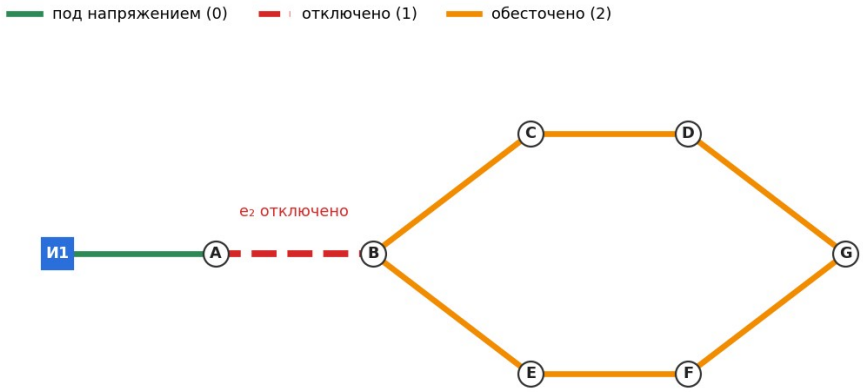


Рисунок 2.1 – Пример графа сети и определения состояний рёбер при отключении ребра e_2 (A–B)

Таблица 2.1 – Матрица смежности графа, приведённого на рисунке 2.1

	И1	**A**	**B**	**C**	**D**	**E**	**F**	**G**
И1	0	1	0	0	0	0	0	0
A	1	0	1	0	0	0	0	0
B	0	1	0	1	0	1	0	0
C	0	0	1	0	1	0	0	0
D	0	0	0	1	0	0	0	1
E	0	0	1	0	0	0	1	0
F	0	0	0	0	0	1	0	1
G	0	0	0	0	1	0	1	0

При моделировании отключения множества рёбер E_{off} формируется **активная матрица смежности** A', в которой элементы, соответствующие отключённым рёбрам, обнуляются:

$$a'_{ij} = 0, \text{ если } (v_i, v_j) \in E_{off};\quad a'_{ij} = a_{ij} - \text{иначе.} \qquad (2.4)$$

Отключение узла $v_k \in V_{off}$ соответствует обнулению k-й строки и k-го столбца матрицы.

Матрица смежности удобна для аналитического описания, но неприменима для сетей большой размерности: для сети Норвегии n = 159 543, и матрица содержит $n^2 \approx 2{,}55 \cdot 10^{10}$ элементов – 25,5 ГБ при хранении одного байта на элемент и 3,2 ГБ при битовом хранении, при этом доля ненулевых элементов составляет около 10^{-5}. Поэтому в программной реализации граф хранится в виде **списков смежности**: каждому узлу сопоставлен список пар «ребро – соседний узел». Объём памяти составляет O(|V| + |E|), а перебор соседей узла выполняется за время, пропорциональное его степени.

Для определения связности используются два классических алгоритма обхода графа [26, 40]:

– **поиск в глубину** (DFS, Depth-First Search) – обход продолжается от последней достигнутой вершины, пока это возможно, после чего выполняется возврат; реализуется стеком;

– **поиск в ширину** (BFS, Breadth-First Search) – вершины посещаются «послойно»: сначала все соседи стартовой вершины, затем их соседи и т. д.; реализуется очередью; в невзвешенном графе BFS находит кратчайшие по числу рёбер пути.

Оба алгоритма имеют сложность O(|V| + |E|) и позволяют выделить компоненты связности активного графа. Для выбора алгоритма анализа отключений выполнен вычислительный эксперимент на графе сети Норвегии (таблица 2.2; 10 сценариев отключения двух случайных линий, Python 3.11).

Таблица 2.2 – Сравнение алгоритмов определения обесточенных узлов на сети Норвегии (159 543 узла, 154 722 ребра)

Алгоритм	Время, мс	Особенности
Выделение всех компонент связности поиском в глубину с проверкой наличия источника	121,5	Обходит всю сеть; результат не зависит от места отключения
Выделение всех компонент связности поиском в ширину	135,5	То же
Многоисточниковый поиск в ширину от всех источников (подраздел 2.3)	120,9	Обходит только запитанную часть сети; является эталоном
Инкрементальный поиск в ширину (подраздел 2.4)	3,1	Обходит только фрагменты, примыкающие к месту отключения

Результаты всех алгоритмов совпадают, однако алгоритмы полного обхода требуют 120–135 мс на каждое отключение, что приемлемо для единичного расчёта, но недопустимо для анализа N-1 десятков тысяч элементов. Наибольшую оперативность обеспечивает инкрементальный поиск в ширину, основанный на свойстве BFS обходить граф послойно от места отключения и останавливаться при достижении источника; этот алгоритм принят в качестве основного.

2.2 Модель геометрического допуска и правила построения топологической модели

Подготовка исходных данных к анализу включает четыре этапа: загрузку слоёв линий, подстанций и опор; проверку и при необходимости перепроецирование в единую прямоугольную систему координат; построение топологически корректной модели; присвоение идентификаторов и формирование атрибутов связности from_node, to_node.

Перепроецирование необходимо, поскольку допуски задаются в метрах, а открытые данные часто распространяются в географических координатах (φ, λ). Переход к плоским прямоугольным координатам выполняется по формулам поперечно-цилиндрической проекции (UTM, Гаусса–Крюгера):

$$(X,\ Y) = f(\varphi,\ \lambda,\ \text{параметры эллипсоида, номер зоны}), \qquad (2.5)$$

где f – функция проекции, реализуемая в ГИС библиотекой PROJ [30]. Набор NVE распространяется в прямоугольной проекции EPSG:25833 (ETRS89 / UTM 33N), поэтому для него перепроецирование не требуется; программа проверяет, что система координат слоёв метрическая.

В основе построения топологии лежит **модель геометрического допуска**. Пусть $p = (x_p, y_p)$ – конечная точка линии, $Q = \{q_1, \ldots, q_m\}$ – множество точечных объектов (узлов). Расстояние определяется евклидовой метрикой

$$d(p, q) = \sqrt{((x_p - x_q)^2 + (y_p - y_q)^2)}. \quad (2.6)$$

Точка p считается привязанной к узлу q, если $d(p, q) \le \varepsilon$, где ε – допуск; при нескольких кандидатах выбирается ближайший. Если расстояние до ближайшего узла превышает ε, привязка невозможна, что свидетельствует о топологической ошибке либо о действительном конце линии.

Ручное исправление ошибок средствами прилипания и Topology Checker для набора из 155 тыс. линий практически невыполнимо, поэтому построение графа автоматизировано и выполняется последовательным применением десяти правил, схема которых приведена на рисунке 2.2. Для ускорения пространственных запросов используются регулярные сеточные индексы точек и отрезков.

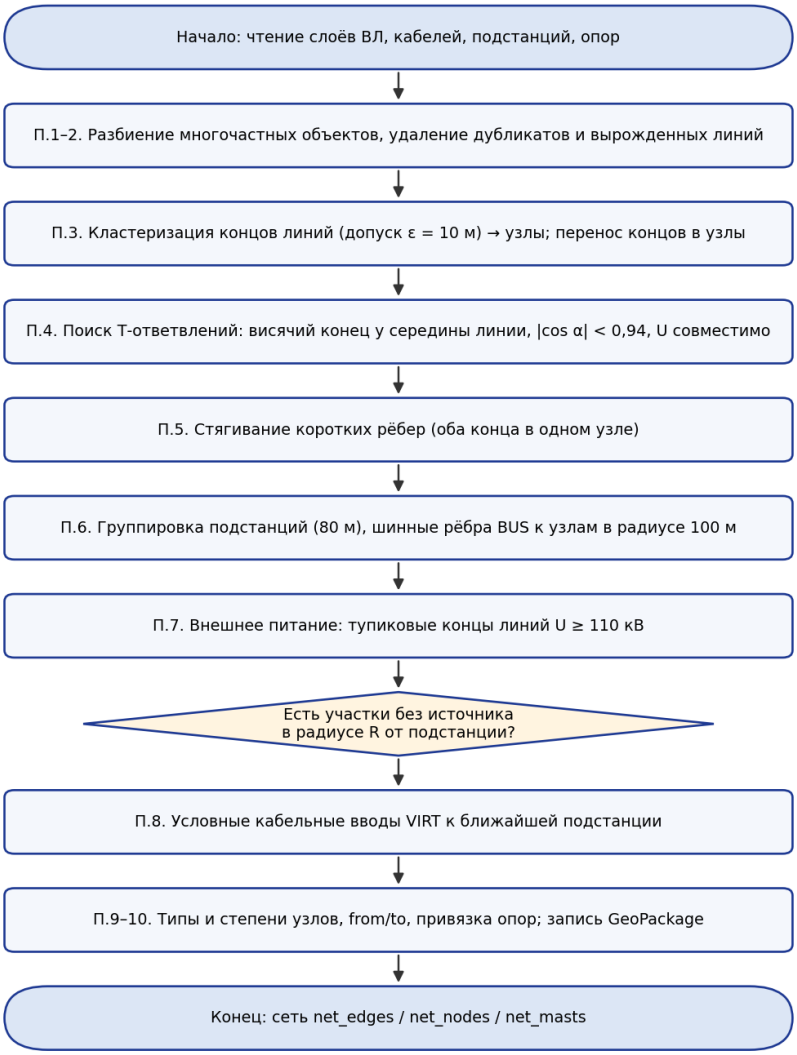


Рисунок 2.2 – Алгоритм построения топологической модели сети

Правило 1 (одночастность). Многочастный объект $M = \{P_1, \ldots, P_m\}$ заменяется m независимыми полилиниями; повторяющиеся подряд вершины ($|p_i - p_{i+1}| < 10^{-6}$ м) удаляются.

Правило 2 (очистка). Удаляются полилинии длиной менее $l_{min} = 0{,}01$ м и дубликаты: полилиния P удаляется, если ранее встречена полилиния с той же последовательностью вершин (с округлением до 1 см) в прямом или обратном порядке.

Правило 3 (сведение концов). Пусть $Q = \{q_1, \ldots, q_{2n}\}$ – множество концов всех полилиний. Концы q_i и q_j принадлежат одному узлу, если существует цепочка концов, соседние элементы которой удалены не более чем на допуск ε:

$$q_i \sim q_j \;\Leftrightarrow\; \exists\, q_i = r_0, r_1, \ldots, r_s = q_j : d(r_t, r_{t+1}) \le \varepsilon,\; t = 0, \ldots, s-1. \quad (2.7)$$

Отношение (2.7) является отношением эквивалентности; его классы находятся с помощью структуры «система непересекающихся множеств» (union–find) [26] с пространственной сеткой с ячейкой ε, что обеспечивает близкую к линейной сложность. Координаты узла принимаются равными центру тяжести класса, концы полилиний переносятся в узел. Правило устраняет недолёты, перелёты и висячие узлы в пределах допуска.

Правило 4 (Т-ответвления). Для каждого узла v степени 1 с направлением примыкающего отрезка d_v ищется ближайший отрезок [a, b] другой полилинии. Проекция узла на отрезок

$$t = max(0, min(1, ((v-a)\cdot(b-a)) / |b-a|\hat{}2\hat{})),\quad q = a + t\cdot(b-a). \quad (2.8)$$

Т-ответвление создаётся (полилиния разрезается в точке q, узел v переносится в q) при одновременном выполнении условий:

$$d(v, q) \le \varepsilon_T,\;\; d(q, p_1) > \varepsilon,\;\; d(q, p_k) > \varepsilon,\;\; |\cos\angle(d_v, b-a)| \le c_{пар},\;\; U_1 \approx U_2, \quad (2.9)$$

где ε_T = 10 м – допуск Т-ответвления; $c_{пар}$ = 0,94 (угол не менее 20°) – порог параллельности, исключающий ложное соединение параллельных цепей двухцепных линий; $U_1 \approx U_2$ – совместимость классов напряжения (равенство, пары 22/24 кВ и 11/12 кВ, неизвестное напряжение). Правило устраняет ошибку «пересечение без узла».

Условие параллельности введено по результатам проверки на реальных данных: без него конец одной цепи двухцепной линии 22 кВ присоединялся к соседней цепи, находящейся в 5 м, что создавало ложную кольцевую связь.

Правило 5 (стягивание коротких рёбер). Ребро, оба конца которого после правила 3 попали в один узел и длина которого менее 2ε, удаляется (стягивается в узел).

Правило 6 (подстанции). Точки подстанций, удалённые не более чем на r_g = 80 м, объединяются в одну станцию (так, «FJÆRE TS» и «FJÆRE 132» образуют одну станцию). Для станции создаётся узел типа STATION; каждый узел линии, удалённый от точки станции не более чем на r_s = 100 м, соединяется с ней ребром вида BUS (шина).

Правило 7 (внешнее питание). Висячий узел линии напряжением $U \geq U_{ext}$ = 110 кВ помечается как EXTERNAL и включается в множество источников S: линия высокого напряжения не может физически заканчиваться тупиком, следовательно, она подходит к электростанции (в наборе NVE электростанции не представлены) или уходит за границу рассматриваемой территории.

Правило 8 (условные вводы). Для каждой компоненты связности без источника находится пара «узел компоненты – станция» с минимальным расстоянием d_{min}. Если $d_{min} \leq R_{feed}$ (по умолчанию 1500 м), добавляется ребро вида VIRT. Правило компенсирует отсутствие в данных подземных кабельных вводов и может быть отключено ($R_{feed} = 0$).

Правило 9 (идентификация и типизация). Узлам и рёбрам присваиваются уникальные идентификаторы node_id, edge_id; рёбрам – идентификаторы начального и конечного узлов (from_node, to_node); узлам – степень и тип; поиском в ширину определяются компоненты связности и признак наличия в них источника.

Правило 10 (опоры). Опора с координатой m привязывается к ребру вида LINE с минимальным расстоянием до его отрезков, если оно не превышает r_m = 25 м.

Присвоение идентификаторов. Идентификатор элемента модели должен быть уникальным, не зависеть от координат и позволять однозначно ссылаться на элемент при анализе и хранении результатов. Рассмотрены два варианта: глобально уникальные идентификаторы UUID версии 4 (128-битные случайные значения, 36 символов в текстовой записи [41]) и сквозные целочисленные номера. Для модели

из 159 543 узлов и 154 722 рёбер выбраны целочисленные идентификаторы: они уникальны в пределах модели, занимают 4–8 байт вместо 36, ускоряют соединение таблиц и построение списков смежности и используются как первичные ключи GeoPackage с индексом. Связь с исходными объектами (глобальная уникальность) обеспечивается составным ключом (src_layer, src_id) – имя исходного слоя и идентификатор исходного объекта, что позволяет проследить каждое ребро до исходной записи набора NVE. При необходимости обмена с системами, требующими UUID, он может быть сформирован детерминированно из этого ключа.

Соответствие логических правил топологической корректности и способов их обеспечения приведено в таблице 2.3.

Таблица 2.3 – Логические правила топологической корректности и их обеспечение

Логическое правило	**Обеспечение в модели**
Каждое ребро соединяет ровно два узла	Правила 3, 4, 9: концы ребра – узлы from_node, to_node
Отсутствие висячих концов, не являющихся потребителями	Правила 3, 4; оставшиеся узлы END – действительные концы отпаек
Отсутствие пересечений (примыканий) без узла	Правило 4 (Т-ответвления)
Отсутствие дубликатов и вырожденных объектов	Правила 1, 2, 5
Уникальность идентификаторов	Правило 9: node_id, edge_id; прослеживаемость (src_layer, src_id)
Согласованность атрибутов связности	from_node, to_node формируются автоматически и ссылаются на существующие узлы
Выявление изолированных элементов	Правило 9: компоненты связности без источника (состояние 3); изолированные подстанции в протоколе

Пример применения правила 4 на реальных данных показан на рисунке 2.3: конец отпайки, не совпадавший с магистралью, после построения топологии стал узлом-ответвлением BRANCH.

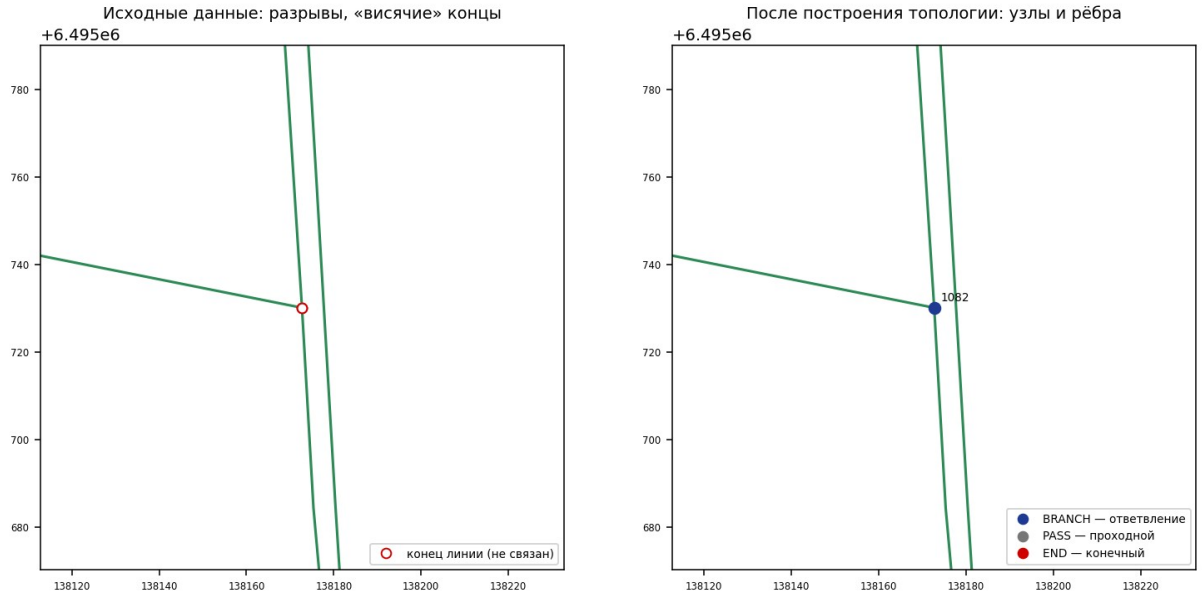


Рисунок 2.3 – Т-ответвление до (слева) и после (справа) построения топологии

Результаты применения правил к данным всей электросети Норвегии приведены в таблице 2.4.

Таблица 2.4 – Результаты построения топологической модели электросети Норвегии

№	Правило	Результат
1	Разбиение многочастных объектов, удаление повторяющихся вершин	475 объектов разбиты (155 839 частей)
2	Удаление дубликатов и вырожденных линий	554 дубликата, 86 вырожденных частей
3	Сведение концов, ε = 10 м	Перенесено 48 911 концов; 158 955 узлов
4	Т-ответвления, ε_T = 10 м, $c_{пар}$ = 0,94	Создано 492 узла-ответвления
5	Стягивание коротких рёбер	Стянуто 5 845 рёбер
6	Подстанции, r_g = 80 м, r_s = 100 м	1 595 станций (из 2 090 точек), 2 835 шинных рёбер; 280 станций без примыкающих линий
7	Внешнее питание, U_{ext} = 110 кВ	390 узлов EXTERNAL
8	Условные вводы, R_{feed} = 1500 м	2 042 ребра VIRT
9	Идентификация и типизация	154 722 ребра, 159 543 узла (STATION 1 595, EXTERNAL 390, BRANCH 28 501, PASS 82 671, END 46 386); 8 012 компонент, из них 521 с источником
10	Привязка опор, r_m = 25 м	718 210 из 734 293 опор (97,8 %)

Число висячих узлов сократилось с 72 245 до 46 386, из которых оставшиеся являются преимущественно действительными концами отпаек распределительной сети. Протяжённость линий, связанных с источниками, составляет 75 155 км из 97 525 км (77,1 %); оставшаяся часть – фидеры распределительной сети, связь которых с подстанциями проходит по подземным кабелям, отсутствующим в открытом наборе.

Выбор значения допуска ε является компромиссом: при малом ε сохраняются ложные разрывы, при большом возникают ложные соединения близко расположенных, но не связанных линий. Для обоснования выбора исследована зависимость характеристик модели от ε при отключённом правиле 8 (таблица 2.5, рисунок 2.4).

Таблица 2.5 – Влияние допуска сведения концов на характеристики модели сети Норвегии

ε, м	Число узлов	Висячих узлов (END)	Компонент связности	Связано с источником, км
0	179 945	67 609	21 642	48 867
1	169 442	54 195	14 159	54 412
5	162 570	48 302	10 982	58 292
10	159 543	46 386	10 054	59 811
20	155 845	44 278	9 024	61 610
30	152 828	42 805	8 373	62 751

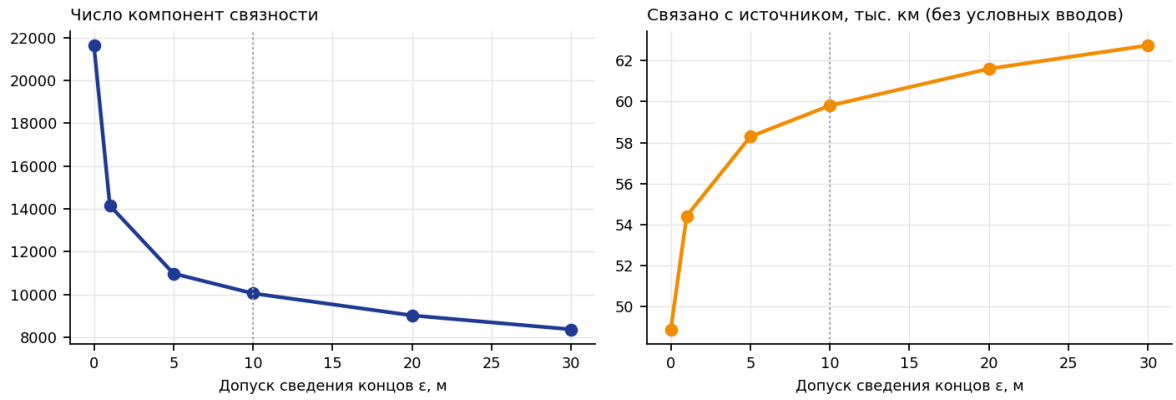


Рисунок 2.4 – Зависимость числа компонент и протяжённости запитанной сети Норвегии от допуска ε

Наибольший эффект даёт устранение погрешностей оцифровки до 1 м: число компонент сокращается с 21 642 до 14 159. В диапазоне 1–10 м устраняются разрывы, характерные для данных, собранных от разных сетевых организаций (ещё 4,1 тыс. компонент). При дальнейшем увеличении ε прирост замедляется, а число сливаемых узлов растёт, что повышает риск ложных соединений в плотной застройке. Значение ε = 10 м принято как обеспечивающее устранение большинства разрывов при сохранении структуры сети. Аналогично обосновано значение радиуса условных вводов R_{feed} = 1500 м (таблица 2.6), соответствующее длине типичного кабельного ввода от подстанции.

Таблица 2.6 – Влияние радиуса условных вводов на характеристики модели сети Норвегии

R_{feed}, м	**Условных вводов**	**Компонент связности**	**Связано с источником, км**
0 (правило отключено)	0	10 054	59 811
500	752	9 302	68 415
1000	1 425	8 629	72 399
1500	2 042	8 012	75 155
2000	2 658	7 396	77 381
3000	3 692	6 362	81 112

Правило условных вводов влияет только на полноту модели, но не на достоверность анализа: участки, не связанные с источником, классифицируются отдельно (состояние 3, подраздел 2.3) и не учитываются как обесточенные вследствие отключения.

2.3 Модель определения обесточенных участков сети

Пусть оператором задано множество отключаемых рёбер $E_{off} \subseteq E$ и узлов $V_{off} \subseteq V$. Отключение узла моделирует вывод из работы сборных шин подстанции или точки соединения: все рёбра, инцидентные узлу, теряют связь через него. Сеть после отключения описывается **активным графом**

$$G' = (V \setminus V_{off},\ \{\, e = (u, v) \in E \setminus E_{off} : u \notin V_{off},\ v \notin V_{off} \,\}). \qquad (2.10)$$

Граф G' может быть несвязным. Обозначим $C = \{C_1, \ldots, C_k\}$ множество его компонент связности (максимальных связных подграфов), $C_i \subseteq V$.

Определение. Обесточенным участком называется компонента связности активного графа, не содержащая ни одного включённого источника питания. Множество обесточенных узлов

$$V_{out} = \cup \{ C_i \in C : C_i \cap S = \emptyset \}. \quad (2.11)$$

Эквивалентно, множество узлов под напряжением L – это множество узлов, достижимых в G' хотя бы из одного включённого источника:

$$L(E_{off}, V_{off}) = \{ v \in V : \exists s \in S \setminus V_{off}, \text{существует путь из } s \text{ в } v \text{ в графе } G' \}, \quad (2.12)$$

причём $V_{out} = V \setminus V_{off} \setminus L$. Множество L находится одним поиском в ширину, стартующим одновременно из всех источников (многоисточниковый BFS) [26].

Определение (2.11) требует уточнения для реальных данных, неполнота которых приводит к существованию участков, не связанных с источниками уже в нормальном режиме. Обозначим $L_0 = L(\varnothing, \varnothing)$ множество узлов под напряжением в нормальном режиме. Тогда состояние ребра e = (u, v) определяется функцией

$$\sigma(e) = 1, \text{ если } e \in E_{off};\ 0, \text{ если } u \in L \text{ или } v \in L;\ 2, \text{ если } u \in L_0 \text{ или } v \in L_0;\ 3 \text{ – иначе}, \quad (2.13)$$

где 0 – ребро под напряжением; 1 – отключено оператором; 2 – обесточено вследствие отключения; 3 – не имеет связи с источником и в нормальном режиме (разрыв исходных данных). Аналогично определяется состояние узла, а опора наследует состояние ребра, к которому она привязана.

Введение состояния 3 принципиально для достоверности: оно отделяет последствия моделируемого отключения от дефектов исходных данных. Без него все участки, не связанные с источниками из-за неполноты данных, отображались бы как обесточенные при любом отключении. Ребро считается под напряжением, если под напряжением хотя бы один его конец: участок линии, примыкающий к

отключённому узлу, остаётся под напряжением со стороны источника до места разрыва.

Пример работы модели приведён на рисунке 2.1: при отключении ребра e_2 (A–B) источник И1 сохраняет связь только с узлом A, а все рёбра кольца B–C–D–G–F–E переходят в состояние 2. Если бы узел G был связан со вторым источником, кольцо осталось бы под напряжением – модель автоматически учитывает резервирование. Алгоритм реализации модели приведён на рисунке 2.5.

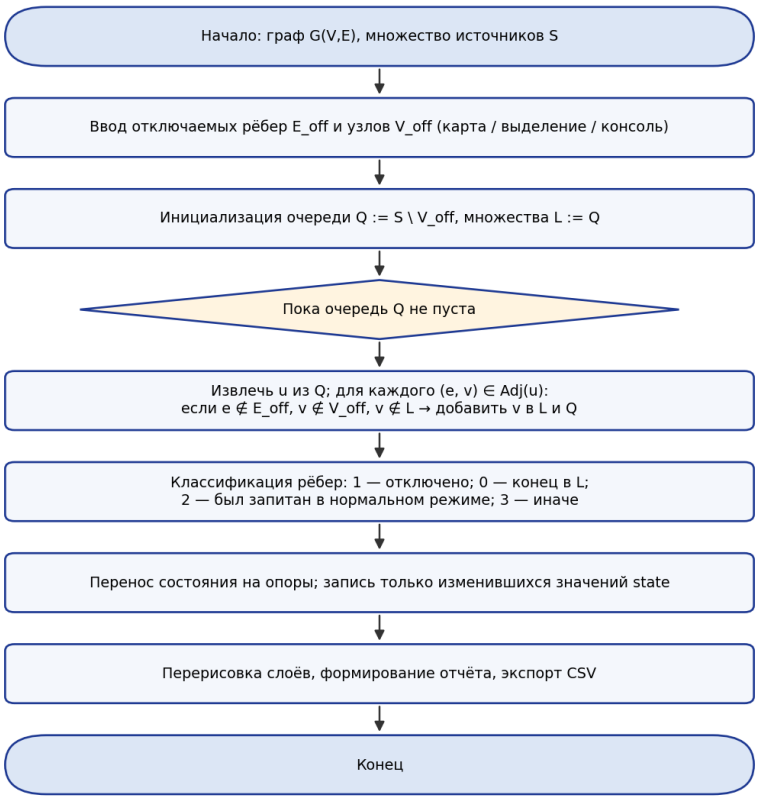


Рисунок 2.5 – Алгоритм определения обесточенных элементов

Утверждение (корректность). Алгоритм находит в точности множество L. Действительно, BFS добавляет узел v в L только при наличии ребра из уже достигнутого узла, не входящего в E_{off} и не ведущего в V_{off}, то есть по пути в G′. Обратно, если существует путь из источника s в v в G′, то по индукции по длине пути каждый узел пути будет извлечён из очереди, а значит, v будет достигнут [26, 40].

Вычислительная сложность. Каждый узел помещается в очередь не более одного раза, каждое ребро просматривается не более двух раз, поэтому

$$T(G) = O(|V| + |E|), \tag{2.14}$$

а объём дополнительной памяти – O(|V|). Для сети Норвегии полный обход занимает в среднем 117–121 мс. Алгоритмы кратчайших путей (например, алгоритм Дейкстры [42]) имеют сложность O(|E| + |V| log |V|) и избыточны для задачи связности. Анализ N-1 при полном обходе требует времени O(m·(|V| + |E|)); для m ≈ 10^5 линий это около 3 ч, что не отвечает требованию оперативности. Поэтому разработана инкрементальная реализация модели.

Для оценки масштаба последствий используются показатели: число $N_{уч}$ и протяжённость ΣL обесточенных рёбер вида LINE, в том числе по классам напряжения; число обесточенных узлов, в том числе ответвлений $N_{отв}$; число обесточенных опор; перечень затронутых линий и подстанций.

2.4 Инкрементальный алгоритм определения обесточенных участков для сетей большой размерности

Отключение одного или нескольких элементов изменяет состояние лишь небольшой части сети: в эксперименте на 1000 случайных авариях сети Норвегии обесточивалось в среднем 72 участка из 154 722 (подраздел 3.4). Полный обход сети при каждом отключении выполняет, таким образом, в основном избыточную работу. Предлагаемый инкрементальный алгоритм использует рассчитанное один раз множество L_0 и пересчитывает только фрагменты, примыкающие к отключённым элементам.

Определим множество стартовых узлов

$$Z = \left(\bigcup_{e=(u,v)\in E_off} \{u, v\} \cup \bigcup_{w\in V_off} \mathrm{Adj}(w) \right) \setminus V_{off}. \tag{2.15}$$

Если узел $z \in Z$ не входит в L_0, он не имел питания и до отключения и не рассматривается. Для остальных узлов выполняется поиск в ширину в графе G′ из z, который **останавливается при достижении любого включённого источника или узла, уже отнесённого к запитанным**. Если источник достигнут, весь просмотренный фрагмент F относится к запитанным; если фрагмент исчерпан без достижения источника, он относится к обесточенным: D := D ∪ F. Итоговые

состояния получаются из состояний нормального режима заменой: узлы $V_{off} \rightarrow 1$, узлы $D \rightarrow 2$; пересчитываются только рёбра, инцидентные $D \cup V_{off}$, и рёбра E_{off}. Схема алгоритма приведена на рисунке 2.6.

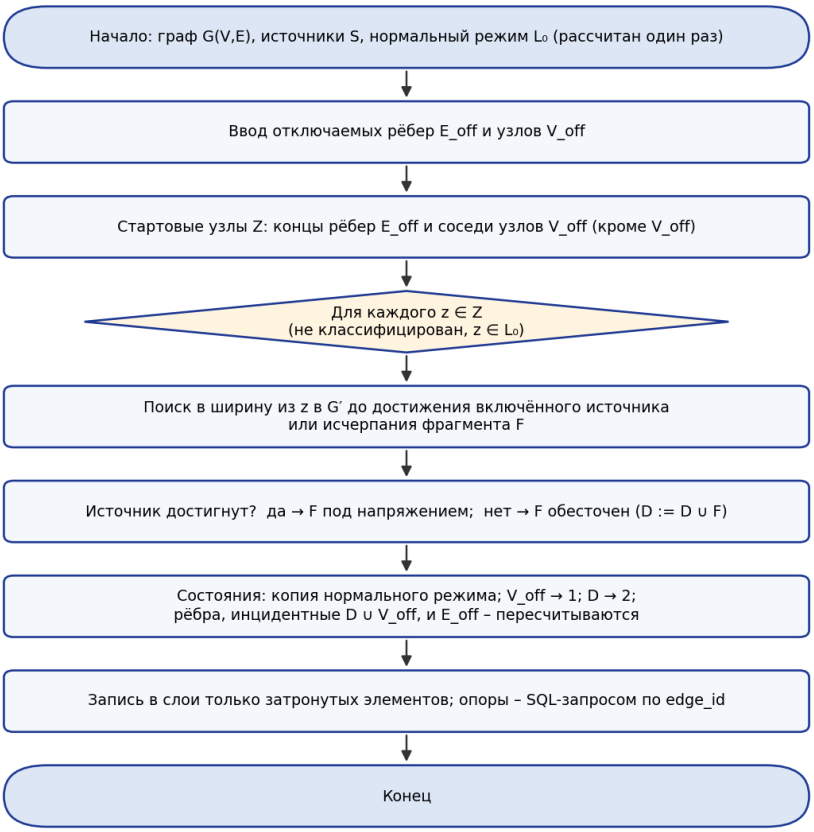


Рисунок 2.6 – Инкрементальный алгоритм определения обесточенных элементов

Утверждение (эквивалентность). Инкрементальный алгоритм даёт те же состояния, что и полный многоисточниковый поиск в ширину. Действительно, узел, не связанный в G' путём ни с одним стартовым узлом, связан с источником теми же путями, что и в G (все его пути к источникам не проходят через отключённые элементы, иначе он был бы связан со стартовым узлом), и сохраняет состояние нормального режима. Узел из фрагмента F связан с источником в G' тогда и только тогда, когда поиск из F достигает источника. Рёбра, не инцидентные изменившимся узлам и не входящие в E_{off}, сохраняют состояние, так как состояния их концов не изменились.

Эквивалентность подтверждена вычислительным экспериментом: для 300 случайных сценариев на фрагменте сети и 60 сценариев на сети Норвегии

(отключение до трёх линий и до двух узлов одновременно) состояния всех рёбер и узлов, полученные обоими алгоритмами, совпали полностью.

Сложность. Время работы составляет $O(|F_\Sigma| + |E(F_\Sigma)|)$, где F_Σ – суммарный размер просмотренных фрагментов: для обесточенной части – её размер, для запитанной – путь до ближайшего источника. В сети с высокой плотностью источников (1985 источников на 159 543 узла) поиск из запитанной части останавливается быстро. На сети Норвегии среднее время расчёта составило 3,3 мс против 116,6 мс для полного обхода – в 36 раз быстрее (рисунок 2.7); анализ N-1 для 97 126 линий выполняется за 4,5 мин вместо ≈ 3 ч.

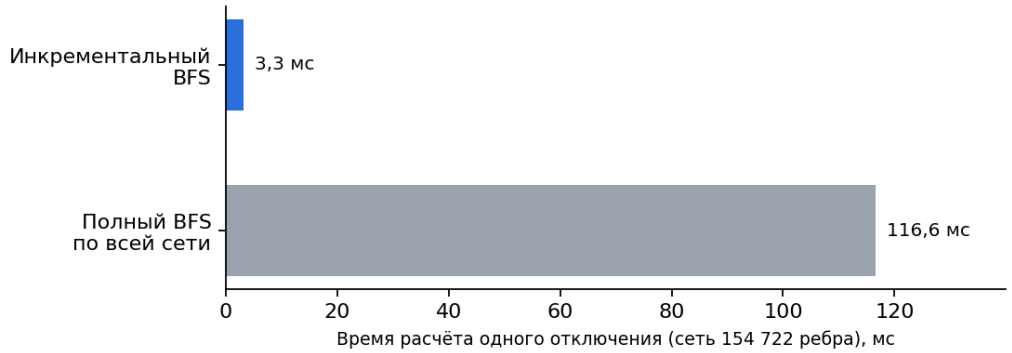


Рисунок 2.7 – Время расчёта одного отключения полным и инкрементальным алгоритмом

Дополнительно алгоритм возвращает множество затронутых элементов, что позволяет записывать в слои ГИС только их состояния и выбирать из базы данных только опоры затронутых линий. Это обеспечивает оперативность не только расчёта, но и отображения результата.

2.5 Модель показателей оперативности анализа

Время анализа одной аварии оператором вручную определяется числом участков и ответвлений, которые необходимо проследить по схеме. Трудоёмкость ручного анализа оценивается моделью

$$T_{руч} = t_0 + t_{уч}\cdot(N_{уч} + N_{откл}) + t_{отв}\cdot N_{отв}, \qquad (2.16)$$

где t_0 = 60 с – время поиска места отключения и подготовки; $t_{уч}$ = 8 с – время прослеживания, проверки и выделения одного участка; $t_{отв}$ = 15 с – время анализа

одного ответвления (проверка наличия резервного питания); $N_{откл}$ – число отключённых элементов. Значения параметров являются экспертной оценкой; для их проверки в программе предусмотрен режим хронометража ручного анализа (подраздел 3.3).

Время автоматизированного анализа складывается из времени расчёта и времени отображения результата:

$$T_{авт} = t_{расч}(|F_{\Sigma}|) + t_{отобр}(N_{изм}), \tag{2.17}$$

где $t_{расч}$ – время инкрементального расчёта, зависящее от размера просмотренного фрагмента; $t_{отобр}$ – время перекраски слоёв ГИС, зависящее от числа элементов $N_{изм}$, состояние которых изменилось. Однократные операции (построение топологии, загрузка модели) выполняются заблаговременно и во время анализа не учитываются.

Коэффициент повышения оперативности для множества из K сценариев:

$$K_{оп} = (\Sigma_k\ T_{руч,k})\ /\ (\Sigma_k\ T_{авт,k}). \tag{2.18}$$

Вероятность своевременного анализа оценивается по выборке сценариев как доля аварий, время анализа которых не превышает допустимого:

$$P_{св} = (1/K)\cdot\Sigma_k\ 1[T_{ан,k} \le T_{доп}], \tag{2.19}$$

где $1[\cdot]$ – индикаторная функция; $T_{доп}$ – допустимое время анализа, которое в работе условно принято равным 5 мин (время, в течение которого диспетчер должен определить зону обесточения и приступить к переключениям).

Трудоёмкость анализа N-1 для m элементов:

$$T_{N\text{-}1} = \Sigma_{j=1..m}\ T_{ан,j}. \tag{2.20}$$

Модель (2.16)–(2.20) позволяет сравнить ручной и автоматизированный анализ на одной и той же выборке сценариев, сформированной случайным образом на реальной сети (подраздел 3.6).

2.6 Методика оперативного анализа аварийных отключений

На основе разработанных моделей предложена методика оперативного анализа аварийных отключений, включающая шесть этапов (рисунок 2.8).



Рисунок 2.8 – Этапы методики анализа аварийных отключений

Этап 1. Подготовка исходных данных. Выбирается источник данных (файловая геобаза или папка Shapefile) и, при необходимости, прямоугольная область обработки. Проверяются наличие слоёв линий, подстанций и опор, их система координат (прямоугольная метрическая) и наличие атрибута напряжения.

Этап 2. Построение топологической модели. Выполняется автоматически по правилам 1–10 (подраздел 2.2) с заданными допусками. Для сети Норвегии этап выполняется однократно (около 8 мин), результат – GeoPackage и протокол построения.

Этап 3. Верификация модели. По протоколу проверяются число компонент связности и компонент с источником, доля протяжённости сети, связанной с источниками, число изолированных подстанций. Участки в состоянии 3 визуально сопоставляются с известной схемой сети. При выявлении ложных разрывов или соединений корректируются допуски или признак is_source отдельных узлов, после чего этап 2 повторяется.

Этап 4. Выбор отключаемых элементов. Оператор задаёт множества E_{off} и V_{off} одним из четырёх способов: поиском по наименованию линии или станции; щелчком по линии или узлу на карте (повторный щелчок возвращает элемент в работу); стандартным инструментом выделения QGIS; командой консоли.

Этап 5. Анализ связности. Выполняется инкрементальный поиск в ширину и классификация элементов по состояниям 0–3 (подразделы 2.3, 2.4). При включённом автоматическом пересчёте анализ выполняется после каждого изменения множества отключений.

Этап 6. Отображение и документирование. Элементы перекрашиваются на карте: отключённые оператором – красным, обесточенные вследствие отключения – оранжевым, находящиеся под напряжением – цветом класса напряжения. Формируется сводка (число и протяжённость обесточенных участков по классам напряжения, число узлов, ответвлений и опор, подстанции, потерявшие питание, перечень затронутых линий) и при необходимости выгружается в CSV.

Этапы 1–3 выполняются заблаговременно, а этапы 4–6 – в темпе поступления информации об отключениях, что и обеспечивает оперативность анализа. Методика применима в **оперативном** режиме (анализ фактического отключения), в **планировочном** режиме (согласование ремонтных заявок, анализ N-1), при **послеаварийном** анализе и **тренировке** персонала; в планировочном режиме этапы 4–6 выполняются многократно программно через консоль.

Выводы по 2 разделу

Разработано математическое представление электрической сети в виде неориентированного мультиграфа с тремя видами рёбер и пятью типами узлов; показано, что матрица смежности сети Норвегии заняла бы 3,2–25,5 ГБ, поэтому в реализации использованы списки смежности. Сравнение алгоритмов DFS и BFS показало, что полный обход требует 120–135 мс, а инкрементальный поиск в ширину – 3,1 мс, что обосновало его выбор (задача 4).

Разработана модель геометрического допуска и формализованы десять правил построения топологической модели, в том числе правило Т-ответвлений с условиями совместимости напряжения и непараллельности; обоснован выбор целочисленных идентификаторов с прослеживаемостью к исходным объектам. Для сети Норвегии построена модель из 154 722 рёбер и 159 543 узлов; исследование

чувствительности обосновало допуск ε = 10 м и радиус условных вводов 1500 м (задача 4).

Формализовано понятие обесточенного участка как компоненты связности активного графа без источника; разработана модель четырёх состояний элементов, отделяющая последствия отключения от разрывов исходных данных, и инкрементальный алгоритм, эквивалентный полному обходу, но в 36 раз более быстрый (задача 5).

Разработана модель показателей оперативности (время анализа, коэффициент повышения оперативности, вероятность своевременного анализа, трудоёмкость анализа N-1) и методика оперативного анализа из шести этапов, в которой трудоёмкие операции вынесены на заблаговременные этапы (задачи 6, 7).

3 РАЗРАБОТКА ПРОГРАММНОГО КОМПЛЕКСА И ОЦЕНИВАНИЕ ОПЕРАТИВНОСТИ АНАЛИЗА

3.1 Общее описание программы

Описание программы составлено с учётом требований ГОСТ 19.402–78 и ГОСТ 19.105–78 [43, 44].

Назначение. Программный комплекс «Анализ аварийных отключений» (файл `outage_qgis.py`) предназначен для автоматизированного построения топологической модели электрической сети по пространственным данным и оперативного определения обесточенных участков при отключении произвольного множества линий и узлов с отображением результата на карте ГИС QGIS.

Условия применения. Программа функционирует в ГИС QGIS 3.x (проверено на QGIS Desktop 3.40.3) под управлением Windows, Linux или macOS и не требует установки дополнительных модулей Python. Для сети общенационального масштаба рекомендуется оперативная память не менее 8 ГБ. Исходные данные должны быть представлены в прямоугольной метрической системе координат.

Язык и объём. Программа написана на языке Python 3 [35] и содержит около 1500 строк; исходный текст приведён в приложении А. Программа состоит из вычислительного ядра, не зависящего от QGIS, и интерфейсной части, использующей PyQGIS и Qt [31].

Состав и функции. Состав программы приведён в таблице 3.1.

Таблица 3.1 – Состав и функции компонентов программного комплекса

Компонент	Функция
PROFILE, BBOX, LOAD_MASTS	Выбор набора данных (вся Норвегия / район), области обработки и режима загрузки опор
DEFAULT_PARAMS	Параметры топологии (допуски, пороги)
_PointGrid, _SegGrid	Пространственные индексы точек и отрезков на регулярной сетке
_UF	Система непересекающихся множеств для кластеризации концов линий и точек подстанций
build_topology()	Реализация правил 1–10, формирование рёбер, узлов, привязки опор и статистики
OutageGraph	Граф в виде списков смежности; analyze() – инкрементальный алгоритм, analyze_full() – эталонный полный обход
build_network()	Чтение слоёв FileGDB или Shapefile, вызов build_topology(), запись GeoPackage и протокола
style_layers()	Условные знаки по полю state, классу напряжения и масштабу, подписи станций
OutageTool	Чтение графа из GeoPackage, ведение множеств отключений, расчёт, запись изменений, отчёт, CSV, поиск, тест скорости, ручной замер
PickTool	Инструмент карты: выбор ближайшего узла или ребра через пространственный индекс GeoPackage
OutageDock	Панель управления (QDockWidget): поиск, список отключений, кнопки, окно результата

Входные данные. Программа читает файловую геобазу NVE непосредственно средствами QGIS (драйвер OpenFileGDB [30]), без предварительной конвертации; из каждого слоя запрашиваются только необходимые атрибуты (таблица 3.2). Предусмотрена обработка прямоугольного фрагмента (параметр BBOX) и набора Shapefile.

Таблица 3.2 – Используемые слои и атрибуты исходных данных

Слой	Тип	Используемые атрибуты
el_luftlinje, el_hengekabel, el_sjokabel	линия	navn (наименование), spenning (напряжение, кВ), nettniva (уровень сети), eier (владелец); идентификатор объекта
el_transformatorstasjon	точка	navn, spenning; идентификатор объекта
el_mast	точка	только геометрия и идентификатор объекта

Выходные данные. Топологическая модель сохраняется в файле GeoPackage [34] `elektro_topo_norge.gpkg` (154 МБ), содержащем слои рёбер net_edges (таблица 3.3), узлов net_nodes (таблица 3.4) и опор net_masts, а также протокол построения `elektro_topo_norge_topology_report.txt` (приложение В).

Таблица 3.3 – Структура слоя рёбер net_edges

Поле	Тип	Назначение
edge_id	целое	Уникальный идентификатор ребра
from_node, to_node	целое	Идентификаторы начального и конечного узла
kind	строка	LINE – линия, BUS – шина станции, VIRT – условный ввод
src_layer, src_id	строка, целое	Исходный слой и идентификатор исходного объекта (прослеживаемость)
name, volt, level, owner	строка, вещественное	Наименование, напряжение (кВ), уровень сети, владелец
length	вещественное	Длина, м
comp, fed	целое	Номер компоненты связности; признак связи с источником
state	целое	Состояние: 0 – под напряжением, 1 – отключено, 2 – обесточено, 3 – нет связи с источником

Таблица 3.4 – Структура слоя узлов net_nodes

Поле	Тип	Назначение
node_id	целое	Уникальный идентификатор узла
ntype	строка	STATION, EXTERNAL, BRANCH, PASS, END
degree	целое	Степень узла
is_source	целое	1 – источник питания (допускает ручную правку)
name, volt	строка, вещественное	Наименование станции, напряжение
comp, fed, state	целое	Компонента связности, признак связи с источником, состояние

Слой опор net_masts (734 293 объекта) содержит поля mast_id, src_id, edge_id (ребро, к которому привязана опора), dist и state. По полю edge_id создаётся индекс SQLite [45], что позволяет выбирать опоры отключённых и обесточенных линий запросом за доли секунды, не загружая 734 тыс. объектов в память.

3.2 Схемы алгоритмов программы и обеспечение оперативности

Архитектура программного комплекса приведена на рисунке 3.1. Схемы алгоритмов построения топологии и определения обесточенных элементов, выполненные с учётом ГОСТ 19.701–90 [46], приведены на рисунках 2.2, 2.5 и 2.6.

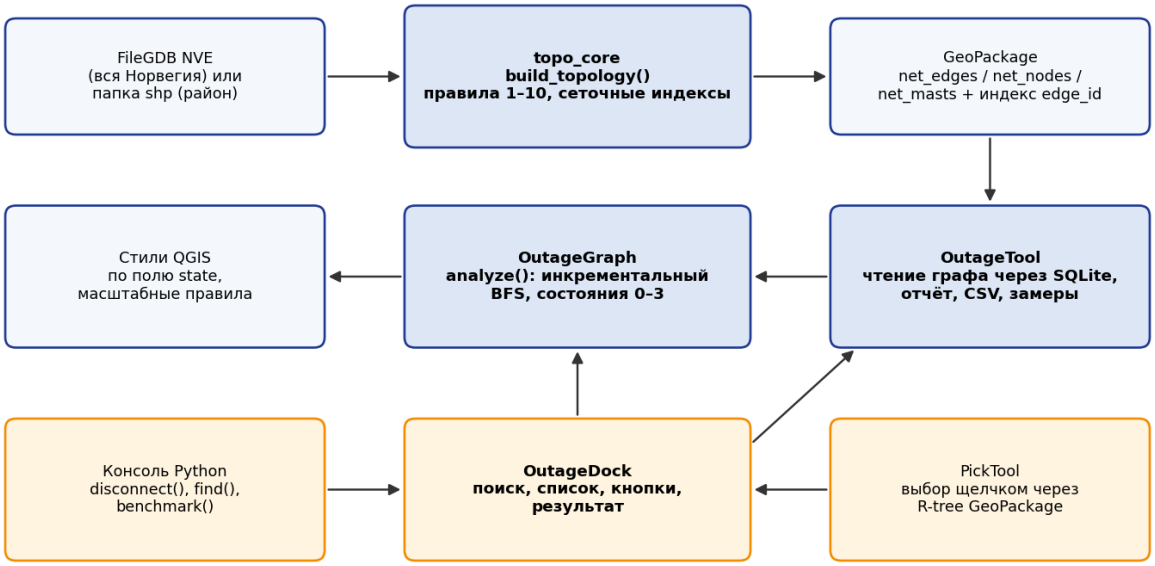


Рисунок 3.1 – Архитектура программного комплекса

Ядро расчёта – инкрементальный метод analyze() класса OutageGraph – реализует алгоритм подраздела 2.4; фрагмент листинга приведён ниже (полный текст – в приложении А):

```
for s0 in start:                          # концы отключённых элементов
    if s0 in live or s0 in dead or s0 not in base:
        continue
    seen = {s0}; q = deque([s0]); found = s0 in src
    while q and not found:
        u = q.popleft()
        for eid, v in adj[u]:
            if eid in off_e or v in off_n or v in seen:
                continue
            if v in live or v in src:     # достигнут источник – стоп
                found = True; break
            seen.add(v); q.append(v)
    (live if found else dead).update(seen)
```

Первая версия программы, разработанная для фрагмента сети, при переходе к общенациональным данным потребовала бы 152 с на загрузку графа и около 120 мс на каждый расчёт. Для обеспечения оперативности приняты следующие решения:

1) атрибуты рёбер и узлов читаются напрямую из таблиц GeoPackage средствами SQLite, без создания объектов QGIS и без геометрии (время загрузки графа сократилось со 152 до 3–10 с);

2) опоры не загружаются в память: их идентификаторы выбираются SQL-запросом по индексу edge_id только для линий, состояние которых изменилось;

3) выбор объекта щелчком выполняется через пространственный индекс R-tree GeoPackage, а не перебором всех объектов;

4) расчёт выполняется инкрементальным алгоритмом (в 36 раз быстрее полного обхода), а в слои записываются только затронутые элементы;

5) при построении топологии отрезки регистрируются в сеточном индексе только вдоль своей трассы, а поиск станций для условных вводов выполняется через индекс, что сократило время построения с 59 до 21 с на вычислительном ядре;

6) условные знаки зависят от масштаба: при мелком масштабе станции отображаются малыми значками, узлы и опоры скрываются, подписи станций выводятся только в масштабе крупнее 1:400 000.

Временные характеристики программы, измеренные в QGIS 3.40.3 на компьютере пользователя, приведены в таблице 3.5. Статистика построенной в QGIS топологии полностью совпала с результатом автономного запуска вычислительного ядра.

Таблица 3.5 – Временные характеристики программы на сети Норвегии (QGIS 3.40.3)

Операция	**Время**
Чтение файловой геобазы (линии, станции, 734 тыс. опор)	91 с
Построение топологии (правила 1–10)	194 с
Запись GeoPackage (154 МБ) и протокола	≈ 3,5 мин
Итого первое построение (однократно, заблаговременно)	8,2 мин
Загрузка готовой модели при последующих запусках	2,6–9,7 с
Расчёт одного отключения (30 случайных сценариев)	13,6 мс
Расчёт с перекраской слоёв	≈ 14 мс; крупная авария с тысячами опор – до 1,4 с

3.3 Руководство оператора

Руководство оператора составлено с учётом ГОСТ 19.505–79 [47]; полный текст приведён в приложении Б.

Запуск. Программа запускается в консоли Python QGIS (меню «Модули» → «Консоль Python») одной командой exec(open(путь, encoding='utf-8').read()). При первом запуске строится топологическая модель, при последующих – загружается готовая. В проект добавляется группа слоёв «Анализ аварийных отключений – Норвегия», а справа от карты – панель управления (рисунок 3.2).

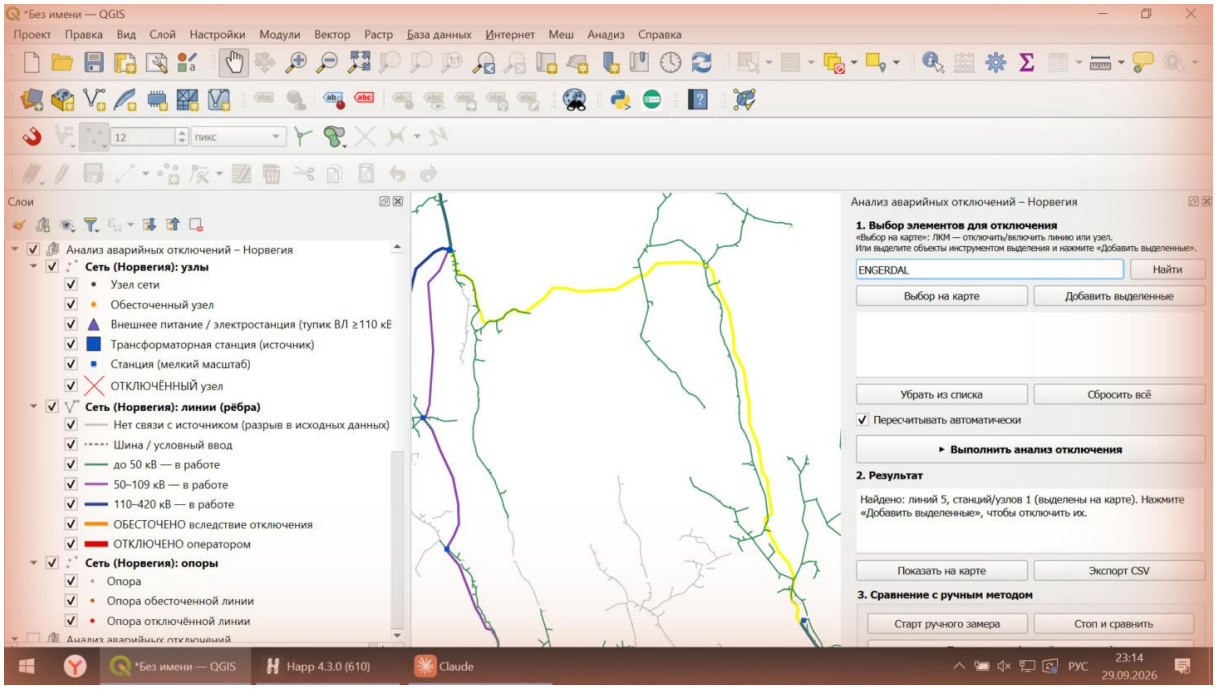


Рисунок 3.2 – Панель управления и поиск станции ENGERDAL по наименованию

Выбор отключаемых узлов и линий. Панель содержит поле поиска по наименованию, группу выбора элементов (режим «Выбор на карте», при котором щелчок левой кнопкой по линии или узлу отключает элемент, а повторный щелчок возвращает его в работу; добавление объектов, выделенных стандартным инструментом QGIS; список отключений; кнопки удаления и сброса), выполнение анализа и вывод результата, а также группу сравнения с ручным методом.

Отображение результата. После каждого изменения списка отключений (при установленном флажке «Пересчитывать автоматически») элементы

перекрашиваются в соответствии с условными обозначениями (таблица 3.6), а в окне «Результат» выводится сводка.

Таблица 3.6 – Условные обозначения состояний элементов

Обозначение	Значение
Красная штриховая линия, красный крест	Элемент отключён оператором (состояние 1)
Оранжевая линия, оранжевые точки	Обесточено вследствие отключения: линии, узлы, опоры (состояние 2)
Синяя / фиолетовая / зелёная линия	Под напряжением: 110–420 кВ / 50–109 кВ / до 50 кВ
Серая тонкая линия	Нет связи с источником в исходных данных (состояние 3)
Пунктир	Шина станции или условный кабельный ввод
Синий квадрат / фиолетовый треугольник	Трансформаторная станция / внешнее питание (тупик ВЛ ≥ 110 кВ)

Режим сравнения с ручным методом. По кнопке «Старт ручного замера» программа отображает только отключённые элементы, активирует инструмент выделения и запускает секундомер; испытуемый вручную выделяет линии, которые, по его мнению, окажутся обесточенными. По кнопке «Стоп и сравнить» программа сравнивает выделение с расчётом и выводит время, число верных, лишних и пропущенных участков и коэффициент ускорения. Кнопка «Тест скорости» выполняет серию случайных сценариев и сравнивает время программы с оценкой по модели (2.16).

Команды консоли (таблица 3.7) позволяют автоматизировать массовые расчёты, в том числе анализ N-1.

Таблица 3.7 – Команды консоли Python

Команда	Назначение
OUTAGE_TOOL.find('NORSENGA')	Найти рёбра и узлы по наименованию
OUTAGE_TOOL.disconnect(lines=[…], nodes=[…])	Отключить линии и узлы и выполнить расчёт
OUTAGE_TOOL.reset()	Вернуть сеть в нормальный режим
OUTAGE_TOOL.export_csv()	Сохранить отчёт в CSV
OUTAGE_TOOL.benchmark(30)	Тест скорости на случайных сценариях
build_network(); start()	Перестроить топологию и перезапустить панель

Сообщения оператору: «Слои сети не загружены» – выполнить build_network(); «Увеличьте масштаб карты для выбора объекта» – приблизить карту или воспользоваться поиском; «Рядом с точкой нет линии или узла сети» – щёлкнуть ближе к объекту.

3.4 Результаты решения контрольной задачи

Контрольная задача решалась на топологической модели всей электросети Норвегии (рисунок 3.3): 97,5 тыс. км линий 0,4–420 кВ, 1595 трансформаторных станций, 390 точек внешнего питания.

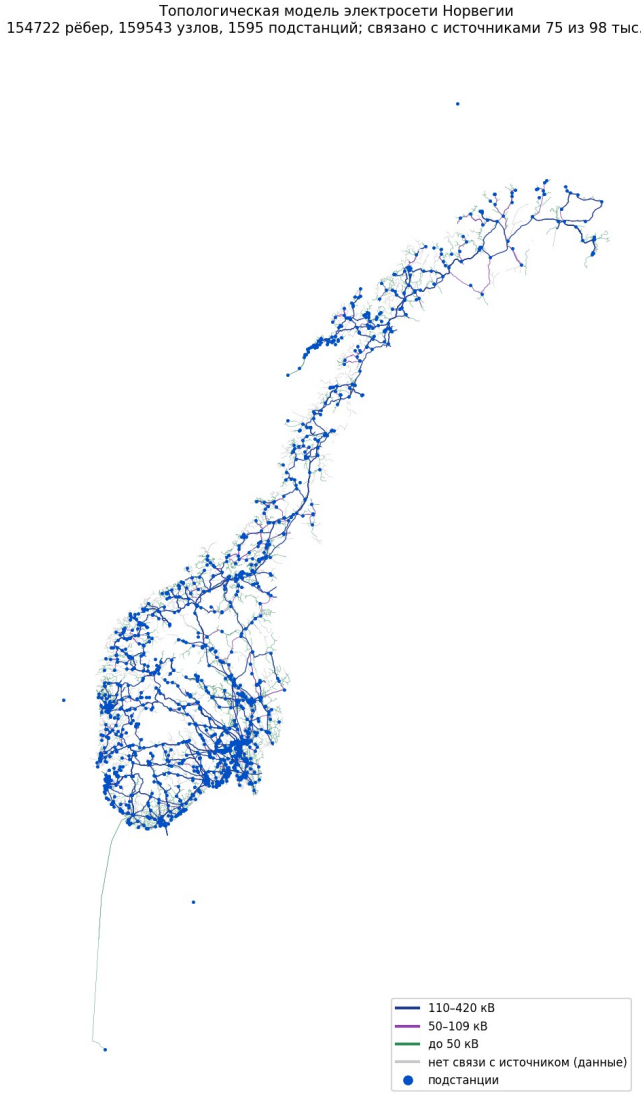


Рисунок 3.3 – Топологическая модель электросети Норвегии

Рассмотрены сценарии отключений разной природы – отключение узлов (подстанций) и линий (таблица 3.8).

Таблица 3.8 – Результаты решения контрольной задачи на сети Норвегии

Сценарий	Обесточено участков	Длина, км	Узлов (ответвлений)	Опор	Расчёт, мс
1. Отключение подстанции NORSENGA (Конгсвингер)	717	377,57	705 (184)	3 834	3,0
2. Отключение подстанции ENGERDAL	395	323,28	398 (88)	3 498	3,4
3. Отключение ВЛ 420 кВ L0705 Rød–Hasle (все 16 участков)	2	55,58	8 (4)	140	4,4
4. Авария на ВЛ 22 кВ (ребро 24265, 1049 м)	9	6,58	9 (2)	41	4,3
5. Авария на ВЛ 22 кВ «BAKKEN BEK1» (проверка по фрагменту)	74	39,36	74 (13)	540	1,9

Время в таблице 3.8 приведено для вычислительного ядра; в среде QGIS расчёт тех же сценариев занимал 16–29 мс, а с перекраской слоёв – до 1,4 с для сценариев с тысячами обесточенных опор.

Сценарий 1. Отключение подстанции NORSENGA приводит к обесточению разветвлённой распределительной сети 22 кВ протяжённостью 377,57 км, включающей 184 ответвления и 3834 опоры (рисунки 3.4, 3.5). Линии 132 кВ, проходящие рядом, остаются под напряжением.

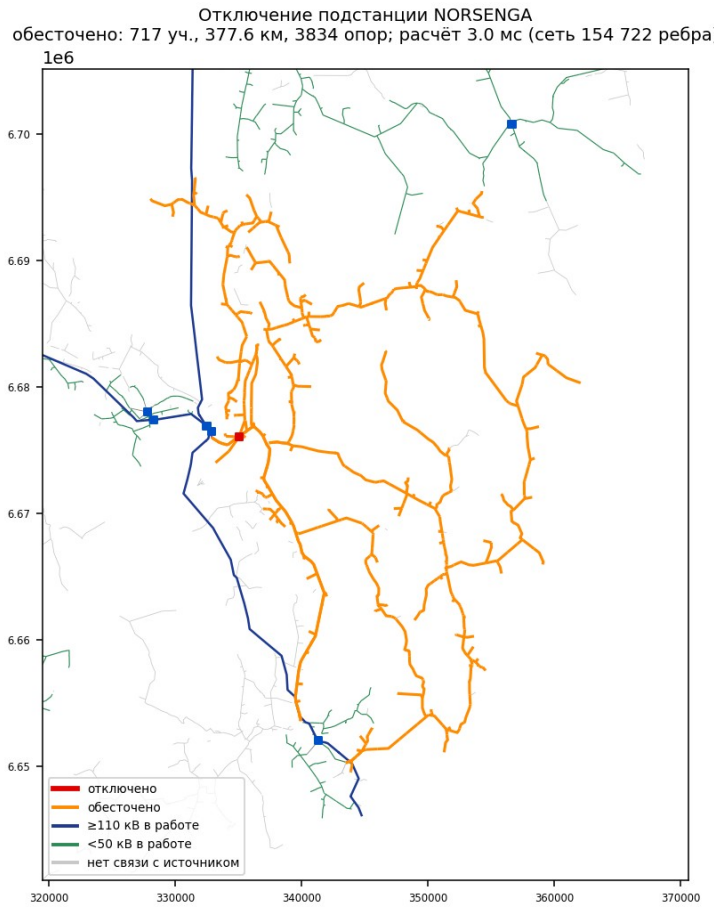


Рисунок 3.4 – Сценарий 1: отключение подстанции NORSENGA (расчёт)



Рисунок 3.5 – Сценарий 1 в QGIS: отключённая станция отмечена красным крестом, обесточенная сеть – оранжевым

Сценарий 2. Отключение подстанции ENGERDAL обесточивает 395 участков распределительной сети 22 кВ общей протяжённостью 323,28 км и 3498 опор (рисунок 3.6).

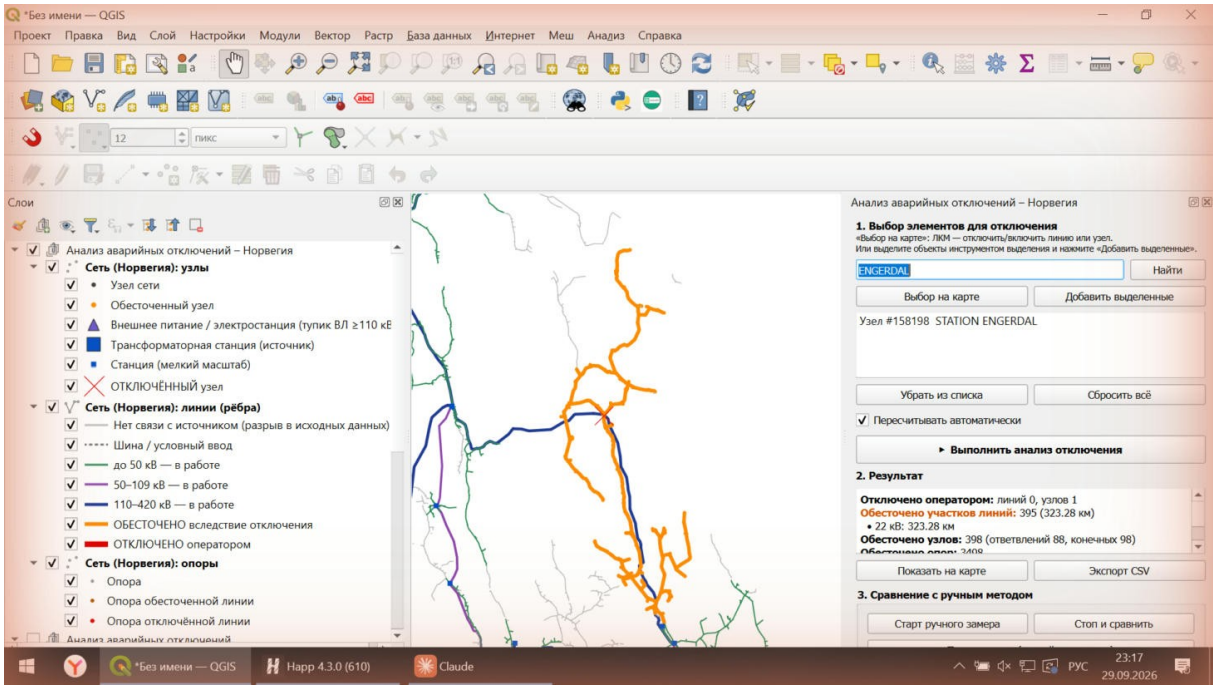


Рисунок 3.6 – Сценарий 2 в QGIS: отключение подстанции ENGERDAL

Сценарий 3. Вывод из работы магистрали 420 кВ Rød–Hasle целиком (16 участков) обесточивает лишь 2 участка ответвления общей длиной 55,58 км (рисунки 3.7, 3.8): системообразующая сеть 420 кВ кольцевая, и питание подстанций сохраняется по другим связям. Результат иллюстрирует преимущество расчёта связности перед «визуальным» анализом, при котором отключение магистрали высшего напряжения может быть ошибочно воспринято как обесточение большой территории.

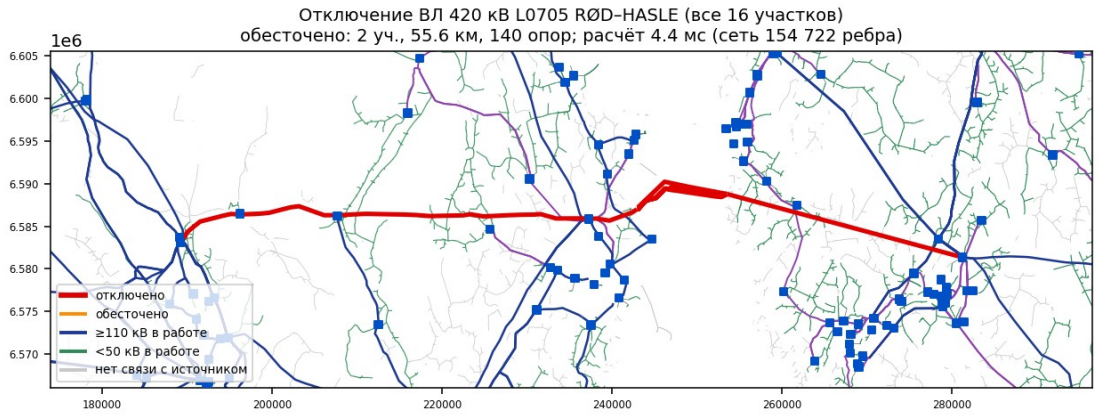


Рисунок 3.7 – Сценарий 3: отключение ВЛ 420 кВ Rød–Hasle (расчёт)

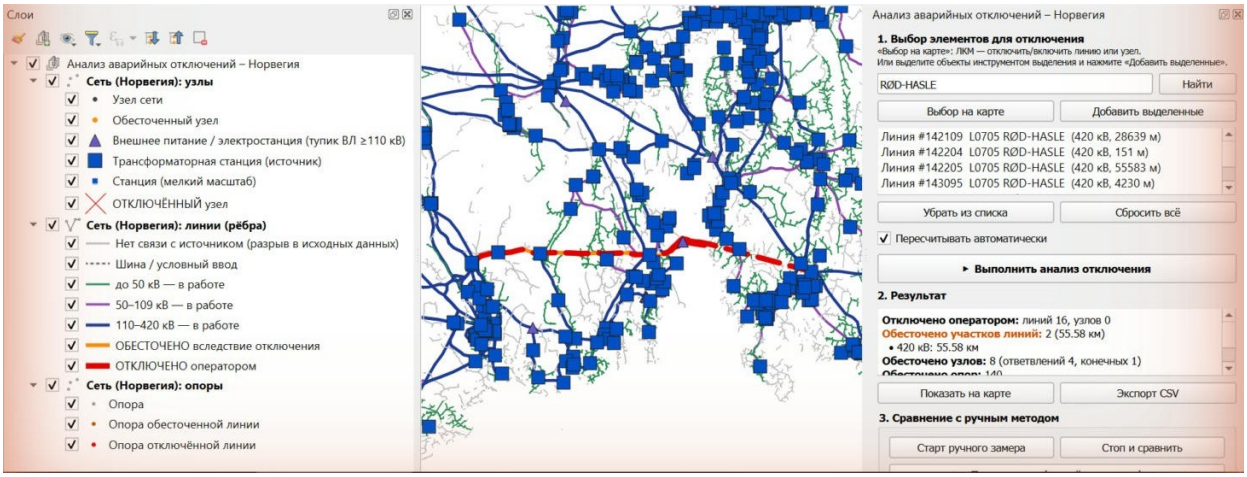


Рисунок 3.8 – Сценарий 3 в QGIS: линия найдена по наименованию и отключена целиком

Сценарий 5 выполнен для проверки согласованности моделей: отключение фидера «BAKKEN BEK1» в общенациональной модели обесточивает 74 участка протяжённостью 39,36 км против 73 участков и 39,28 км в модели фрагмента сети.

Расхождение в один участок объясняется тем, что во фрагменте граница района отсекала часть сети; результаты по существу совпадают.

Анализ N-1 подстанций. Последовательно отключены все 1595 станций (9,3 с). Отключение 1131 станции (70,9 %) приводит к обесточению других участков. Наиболее критичные станции распределительной сети приведены на рисунке 3.9. Наибольший формальный результат (888 км) даёт преобразовательная станция межгосударственной связи постоянного тока «EDC»: поскольку подводные кабели NorNed и Skagerrak не имеют атрибута напряжения, их противоположные концы не распознаются как внешнее питание. Этот случай исключён из рисунка как ограничение исходных данных.

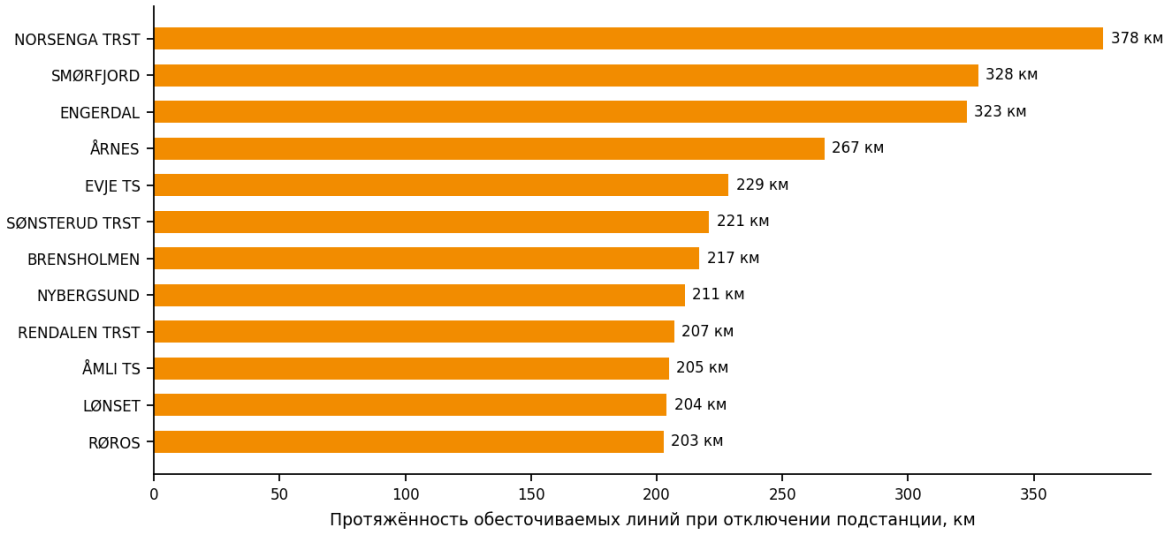


Рисунок 3.9 – Наиболее критичные подстанции распределительной сети по протяжённости обесточиваемых линий

Анализ N-1 линий. Отключены по очереди все 97 126 линий, связанных с источниками; расчёт занял 4,5 мин. Отключение 60 024 линий (61,8 %) приводит к обесточению других участков, остальные являются тупиковыми отпайками либо резервируются.

Статистический эксперимент. Смоделировано 1000 случайных аварий с отключением от одной до трёх линий. Распределение масштаба аварий приведено на рисунке 3.10: в 17,6 % случаев отключение не приводит к обесточению других

участков, в среднем обесточивается 71,8 участка и 8,6 ответвления, максимально – 3383 участка.

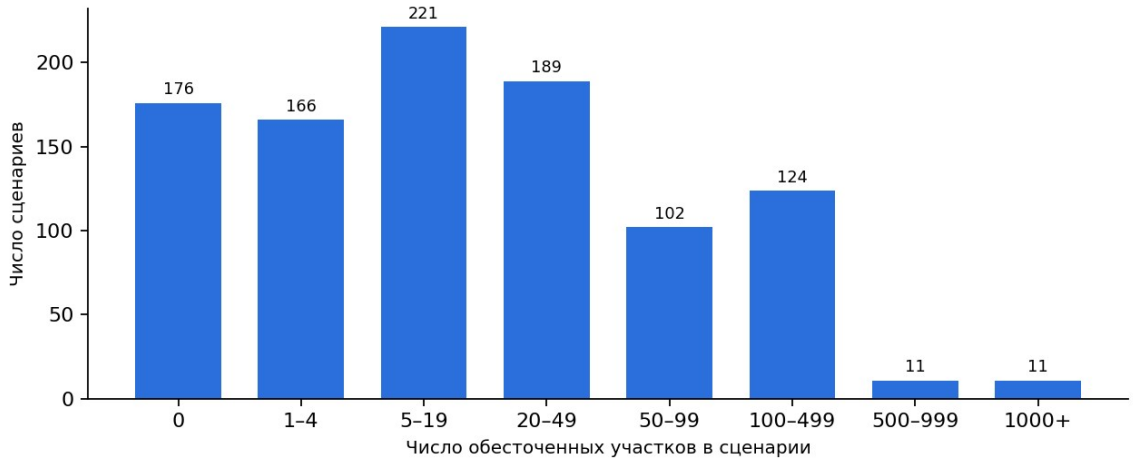


Рисунок 3.10 – Распределение сценариев по числу обесточенных участков (сеть Норвегии, 1000 сценариев)

Достоверность результатов подтверждена совпадением результатов инкрементального и эталонного алгоритмов во всех проверочных сценариях, совпадением результатов в QGIS и автономного ядра, а также согласованностью моделей фрагмента и всей сети.

3.5 Применение программного комплекса для решения прикладных задач

Разработанный комплекс позволяет решать все классы задач, выделенные в подразделе 1.1 (таблица 1.1).

Задача 1. Оперативный анализ отключения диспетчером. Исходные данные: поступило сообщение об отключении подстанции NORSENGA. Решение: диспетчер находит станцию поиском по наименованию и отключает её; через ≈ 14 мс на карте выделены 717 обесточенных участков (377,6 км, 184 ответвления), в окне результата – перечень затронутых линий. Практический результат: зона обесточения и перечень линий для переключений получены до начала переговоров с персоналом.

Задача 2. Оценка надёжности схемы и выявление критичных элементов. Исходные данные: требуется определить элементы, отказ которых приводит к наибольшим последствиям. Решение: анализ N-1 всех 1595 станций (9,3 с) и 97 126

линий (4,5 мин) через консоль. Практический результат: ранжированный перечень критичных подстанций (рисунок 3.9) и линий для обоснования резервирования и установки коммутационных аппаратов.

Задача 3. Планирование ремонтной схемы. Исходные данные: заявка на вывод в ремонт ВЛ 420 кВ Rød–Hasle. Решение: моделирование отключения всех 16 участков линии показывает, что обесточиваются лишь 2 участка ответвления (55,6 км), остальная сеть резервируется. Практический результат: обоснованное согласование заявки с мероприятиями только для одного ответвления.

Задача 4. Послеаварийный анализ и оценка недоотпуска. Недоотпуск электроэнергии при перерыве длительностью $T_{откл}$ вычисляется по найденному множеству обесточенных узлов:

$$\Delta W = \Sigma_{v \in V_out} P_v \cdot T_{откл}, \quad (3.1)$$

где P_v – средняя мощность нагрузки, присоединённой к узлу v. В открытом наборе NVE нагрузки отсутствуют; при добавлении атрибута нагрузки в слой узлов расчёт ΔW выполняется для каждой аварии журнала автоматически. Практический результат: количественная оценка ущерба для разработки противоаварийных мероприятий.

Задача 5. Тренировка диспетчерского персонала. Преподаватель задаёт сценарий аварии, обучаемый в режиме ручного замера выделяет обесточенные участки на карте, после чего программа сравнивает ответ с расчётом и выводит время, число верных, лишних и пропущенных участков. Практический результат: объективная оценка навыков оперативного анализа.

3.6 Оценивание оперативного и технико-экономического эффекта

Оперативность анализа оценена по модели подраздела 2.5 на выборке из 1000 случайных аварий сети Норвегии (подраздел 3.4). Для каждой аварии время ручного анализа рассчитано по модели (2.16), время автоматизированного – измерено. Результаты приведены в таблице 3.9 и на рисунках 3.11, 3.12.

Таблица 3.9 – Показатели оперативности анализа аварийных отключений на сети Норвегии

Показатель	Ручной анализ	Программный комплекс
Среднее время анализа одной аварии $T_{ан}$	779 с (13,0 мин)	2,2 мс (ядро); ≈ 14 мс в QGIS с перекраской
Медиана / 90-й процентиль времени анализа	225 с / 1514 с	–
Максимальное время анализа	30 283 с (8,4 ч)	8,3 мс (ядро); до 1,4 с в QGIS
Вероятность своевременного анализа $P_{св}$ при $T_{доп}$ = 5 мин	57,4 %	100 %
То же при $T_{доп}$ = 10 мин	75,2 %	100 %
Трудоёмкость анализа N-1 для 97 126 линий T_{N-1}	≈ 11 600 ч (6,6 человеко-лет)	4,5 мин
Коэффициент повышения оперативности $K_{оп}$	–	≈ $5{,}6 \cdot 10^4$ (по времени в QGIS); не менее $2 \cdot 10^4$ по встроенному тесту
Полнота выявления обесточенных участков	зависит от исполнителя	100 %

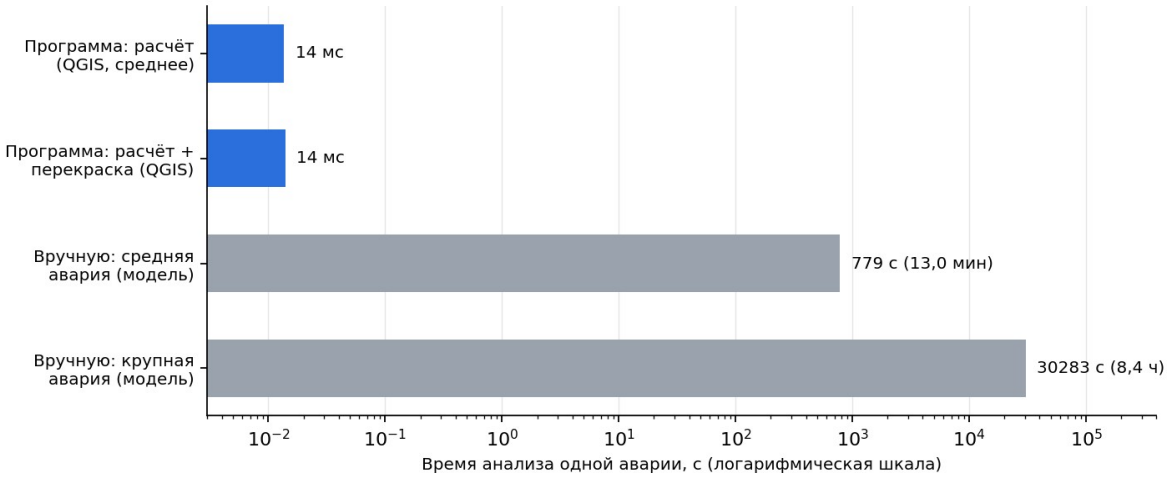


Рисунок 3.11 – Время анализа одной аварии программой и вручную (логарифмическая шкала)

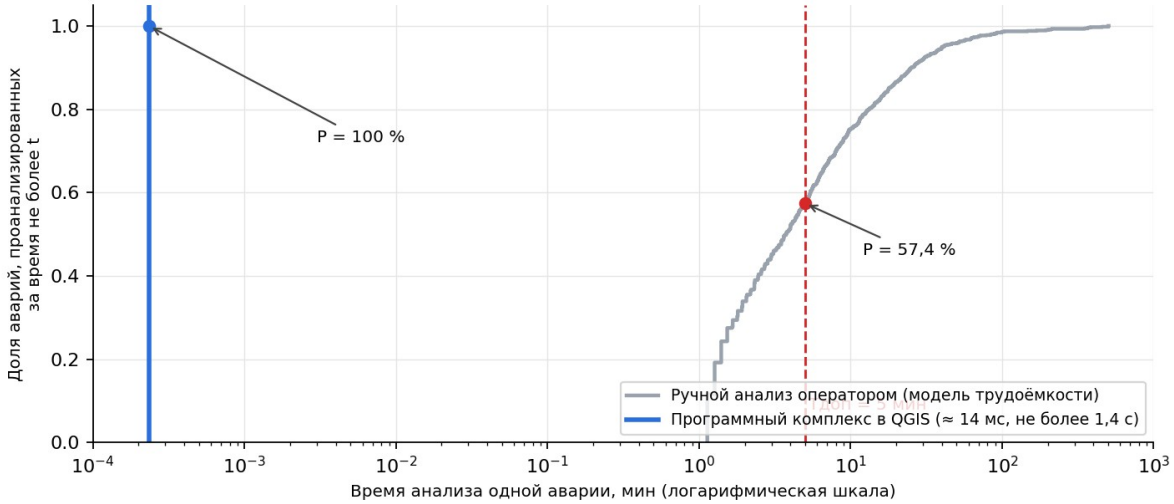


Рисунок 3.12 – Функция распределения времени анализа аварий и вероятность своевременного анализа

При ручном анализе только 57,4 % аварий анализируются за 5 мин, а крупные аварии требуют часов; программный комплекс анализирует любую аварию из выборки менее чем за 1,4 с, то есть обеспечивает $P_{св} = 1$ при любом разумном $T_{доп}$. Для проверки параметров модели (2.16) на фрагменте сети был выполнен пробный ручной замер встроенным хронометражем: за 18,9 с испытуемый выделил 1 верный участок из 73 (полнота 1 %), тогда как программа нашла все 73 участка за 0,22 с с учётом перерисовки карты. Это подтверждает, что оценки по модели не завышены, а ручной анализ при ограничении времени не обеспечивает достоверности.

Технический эффект выражается в сокращении времени анализа одного отключения с 13 мин (в среднем) до 14 мс – более чем на четыре порядка, в полноте выявления обесточенных участков, в корректном учёте кольцевых связей и в возможности анализа N-1 сети общенационального масштаба за минуты.

Экономический эффект оценивается по экономии трудозатрат персонала и снижению ущерба от недоотпуска электроэнергии за счёт сокращения продолжительности перерывов. Расчёт выполнен при условных исходных данных для одной крупной сетевой организации (таблица 3.10), которые при внедрении подлежат уточнению.

Таблица 3.10 – Условные исходные данные для оценки эффективности

Параметр	Обозначение	Значение
Число аварийных отключений в год	$N_{ав}$	1000
Сокращение времени анализа одного отключения	Δt	779 с (0,2164 ч)
Стоимость часа работы инженера с учётом страховых взносов (30 %)	$C_{ч}$	910 руб./ч
Средняя мощность обесточенной нагрузки	$P_{ср}$	200 кВт
Удельный ущерб потребителей от недоотпуска	y_0	50 руб./(кВт·ч)
Трудоёмкость разработки и внедрения	$T_р$	240 ч

Годовая экономия трудозатрат оперативного персонала:

$$Э_{тр} = N_{ав} \cdot \Delta t \cdot C_{ч} = 1000 \cdot 0{,}2164 \cdot 910 \approx 197 \text{ тыс. руб.} \quad (3.2)$$

Годовое снижение ущерба от недоотпуска электроэнергии:

$$Э_{нед} = N_{ав} \cdot \Delta t \cdot P_{ср} \cdot y_0 = 1000 \cdot 0{,}2164 \cdot 200 \cdot 50 \approx 2\ 164 \text{ тыс. руб.} \quad (3.3)$$

Единовременные затраты на разработку и внедрение (QGIS распространяется свободно, лицензии не требуются):

$$К = T_р \cdot C_{ч} = 240 \cdot 910 \approx 218 \text{ тыс. руб.} \quad (3.4)$$

Годовой экономический эффект $Э = Э_{тр} + Э_{нед} \approx 2\ 361$ тыс. руб., срок окупаемости

$$T_{ок} = К \ / \ Э = 218 \ / \ 2\ 361 \approx 0{,}09 \text{ года} \approx 1{,}1 \text{ мес.} \quad (3.5)$$

Экономия на анализе N-1 в расчёт не включена, так как при ручном методе такой анализ для сети общенационального масштаба не выполняется вовсе. Сводные показатели приведены в таблице 3.11.

Таблица 3.11 – Показатели оперативного и технико-экономического эффекта

Показатель	**Ручной метод**	**Программный комплекс**
Время анализа одного отключения	13 мин (до 8,4 ч)	≈ 14 мс
Вероятность своевременного анализа ($T_{доп}$ = 5 мин)	57,4 %	100 %
Полнота выявления обесточенных участков	зависит от исполнителя	100 %
Учёт кольцевых связей и резервирования	не гарантирован	автоматически
Анализ N-1 (97 126 линий)	≈ 11 600 ч	4,5 мин
Затраты на лицензии ПО	–	0 руб.
Годовой экономический эффект (условно)	–	≈ 2,4 млн руб.
Срок окупаемости	–	≈ 1,1 мес.

Даже при уменьшении условных значений $N_{ав}$, $P_{ср}$ и y_0 на порядок срок окупаемости не превышает одного года.

Выводы по 3 разделу

Разработан программный комплекс на языке Python для консоли QGIS 3.x (около 1500 строк), выполняющий непосредственное чтение общенациональных данных, построение топологии, выбор отключаемых узлов и линий на карте и поиском, выделение отключённых и обесточенных элементов цветом, формирование отчёта, анализ N-1 и сравнение с ручным методом. Составлены описание программы, схемы алгоритмов и руководство оператора (задача 6).

Благодаря инкрементальному алгоритму, чтению данных через SQLite и выборочной перекраске слоёв время отклика на сети из 155 тыс. рёбер составляет около 14 мс, что удовлетворяет требованию не более 1 с.

Решение контрольной задачи для пяти сценариев подтвердило корректность модели, в том числе учёт кольцевых связей сети 420 кВ; анализ N-1 выявил 1131 критичную подстанцию из 1595 и 60 024 критичные линии из 97 126. Показано применение комплекса для пяти классов прикладных задач (задача 7).

По сравнению с ручным анализом оператором среднее время анализа сокращено с 13 мин до 14 мс, вероятность своевременного анализа при $T_{доп}$ = 5 мин повышена с 57,4 до 100 %, трудоёмкость анализа N-1 – с ≈ 11 600 ч до 4,5 мин; при

условных исходных данных годовой экономический эффект составляет ≈ 2,4 млн руб. при сроке окупаемости ≈ 1,1 мес. (задача 7).

Заключение

Выпускная квалификационная работа посвящена вопросам повышения оперативности анализа данных об аварийных отключениях электрической сети за счёт автоматизированного построения топологической модели сети в открытой ГИС QGIS и программного определения обесточенных участков. Работоспособность предложенных решений подтверждена на данных всей электросети Норвегии (154 506 линий, 2090 подстанций, 734 293 опоры) в среде QGIS 3.40.3.

В первом разделе выполнен анализ задачи оперативного анализа аварийных отключений.

1) Анализ роли и места анализа отключений в системе оперативно-диспетчерского управления показал, что время анализа аддитивно входит в продолжительность перерыва электроснабжения и влияет на регулируемые показатели надёжности. Инженерные задачи классифицированы по цели анализа, типу события и категории потребителей; обоснована система показателей оперативности: время анализа, коэффициент повышения оперативности, вероятность своевременного анализа и трудоёмкость анализа N-1 (задача 1 решена).

2) Рассмотрены теоретические основы геометрической топологии в ГИС, топологические правила и ошибки (недолёт, перелёт, висячий узел, пересечение без узла, дублирование). Установлено, что исходный набор NVE топологически некорректен: 72 245 висячих концов, 554 дубликата, 475 многочастных объектов, отсутствие полей from_node и to_node (задача 2 решена).

3) Сравнительный анализ ArcGIS, MapInfo, GRASS GIS и QGIS, а также анализ проблемы вендорной зависимости обосновали выбор открытой цепочки QGIS – Python – GeoPackage с непосредственным чтением FileGDB без программного обеспечения Esri и без сторонних библиотек Python (задача 3 решена).

Во втором разделе разработаны модели и методика оперативного анализа.

4) Разработано математическое представление сети в виде неориентированного мультиграфа; показано, что матрица смежности для сети Норвегии заняла бы до 25,5 ГБ, поэтому применены списки смежности; сравнение алгоритмов поиска в глубину и в ширину обосновало выбор поиска в ширину. Разработаны модель геометрического допуска и десять формализованных правил построения топологической модели, включая правило Т-ответвлений с условиями совместимости напряжения и непараллельности линий, а также процедура присвоения уникальных идентификаторов и атрибутов from_node, to_node. Построена топологическая модель сети Норвегии из 154 722 рёбер и 159 543 узлов; обоснованы допуск ε = 10 м и радиус условных вводов 1500 м (задача 4 решена).

5) Формализовано понятие обесточенного участка как компоненты связности активного графа без источника питания; разработана модель классификации элементов по четырём состояниям, отделяющая последствия отключения от разрывов исходных данных, и инкрементальный алгоритм её реализации, эквивалентность которого полному обходу доказана и подтверждена экспериментом; алгоритм в 36 раз быстрее полного обхода (3,3 мс против 116,6 мс) (задача 5 решена).

Разработаны модель показателей оперативности и методика оперативного анализа из шести этапов, в которой трудоёмкие операции построения и верификации модели выполняются заблаговременно.

В третьем разделе разработан программный комплекс и выполнено оценивание оперативности.

6) На основе предложенной методики разработан программный комплекс на языке Python для консоли QGIS (около 1500 строк), обеспечивающий построение топологии, выбор отключаемых узлов и линий щелчком на карте, поиском по наименованию и инструментом выделения, выделение отключённых элементов красным, а обесточенных – оранжевым цветом, формирование отчёта, анализ N-1 и сравнение с ручным методом. Составлены описание программы, схемы алгоритмов и руководство оператора (задача 6 решена).

7) Решена контрольная задача для пяти сценариев отключения подстанций и линий, выполнены анализ N-1 (1131 критичная подстанция из 1595, 60 024 критичные линии из 97 126) и статистический эксперимент на 1000 случайных авариях; показано применение комплекса для пяти классов прикладных задач. Среднее время анализа одного отключения сокращено с 13 мин при ручном анализе оператором до 14 мс, вероятность своевременного анализа при допустимом времени 5 мин повышена с 57,4 до 100 %, трудоёмкость анализа N-1 – с ≈ 11 600 ч до 4,5 мин. При условных исходных данных годовой экономический эффект составляет ≈ 2,4 млн руб. при сроке окупаемости ≈ 1,1 мес. (задача 7 решена).

Разработанные модели, методика и программный комплекс позволяют повысить оперативность анализа данных об аварийных отключениях более чем на четыре порядка при полном выявлении обесточенных участков и исключении вендорной зависимости. Таким образом, все поставленные задачи решены, цель выпускной квалификационной работы достигнута.

Ограничениями разработки являются топологический (а не режимный) характер анализа, отсутствие в исходных данных коммутационных аппаратов, электростанций и подземных кабелей (компенсируемое правилами внешнего питания и условных вводов), а также отсутствие атрибута напряжения у межгосударственных кабелей постоянного тока. Направлениями развития являются учёт коммутационных аппаратов и нагрузок, расчёт недоотпуска электроэнергии и интеграция с данными телемеханики.

Перечень сокращений и условных обозначений

ВЛ – воздушная линия электропередачи;

ГИС – геоинформационная система;

КЛ – кабельная линия;

ОДУ – оперативно-диспетчерское управление;

ПС, ТС – подстанция, трансформаторная станция;

СК – система координат;

BFS (Breadth-First Search) – поиск в ширину;

DFS (Depth-First Search) – поиск в глубину;

FileGDB – файловая геобаза данных формата ESRI File Geodatabase;

GPKG – формат GeoPackage;

NVE (Norges vassdrags- og energidirektorat) – Норвежское управление водных ресурсов и энергетики;

SAIDI, SAIFI – индексы средней продолжительности и средней частоты прекращения электроснабжения;

SHP – формат ESRI Shapefile;

UUID – универсальный уникальный идентификатор;

$G = (V, E)$ – граф сети с множеством узлов V и множеством рёбер E;

$S \subseteq V$ – множество источников питания;

E_{off}, V_{off} – множества отключённых рёбер и узлов;

L_0 – множество узлов под напряжением в нормальном режиме;

ε – допуск сведения концов линий, м;

$T_{ан}$, $T_{руч}$, $T_{авт}$ – время анализа, время ручного и автоматизированного анализа;

$K_{оп}$ – коэффициент повышения оперативности;

$P_{св}$ – вероятность своевременного анализа;

N-1 – критерий надёжности при отказе одного элемента сети.

Приложение А

(обязательное)

Исходный текст программы outage_qgis.py

Ниже приведён полный исходный текст программного комплекса (версия 2.0). Файл сохраняется в кодировке UTF-8 и запускается в консоли Python QGIS командой exec(open(путь, encoding='utf-8').read()).

```
# -*- coding: utf-8 -*-
"""
ПРОГРАММА АНАЛИЗА АВАРИЙНЫХ ОТКЛЮЧЕНИЙ ЭЛЕКТРИЧЕСКОЙ СЕТИ  (QGIS 3.x, консоль Python)   версия 2.0
--------------------------------------------------------------------------------
Запуск: Модули -> Консоль Python -> в консоли выполнить:
    exec(open(r'C:\\Users\\ASUS\\Desktop\\elktro\\outage_tool\\outage_qgis.py', encoding='utf-
8').read())

PROFILE выбирает набор данных:
  'norge'  – вся электросеть Норвегии из FileGDB NVE (≈155 тыс. линий, 2090 подстанций, 734 тыс. опор);
  'region' – фрагмент сети (папка !!!ТОчняк, shp).
При первом запуске исходные слои приводятся к топологической сети по правилам (см. руководство)
и сохраняются в GeoPackage (слои net_edges, net_nodes, net_masts). Затем открывается панель.
Для всей Норвегии первое построение занимает несколько минут, последующие запуски – загрузка готовой
сети.
"""
import math, time

# ============================ НАСТРОЙКИ ============================
PROFILE = 'norge'          # 'norge' – вся сеть (FileGDB), 'region' – фрагмент (shp)
BASE = r'C:\Users\ASUS\Desktop\elktro'
PROFILES = {
    'norge': {'src': BASE + r'\Energi_0000_Norge_25833_Nettanlegg_FGDB\
Energi_0000_Norge_25833_Nettanlegg_FGDB.gdb',
              'gpkg': BASE + r'\outage_tool\elektro_topo_norge.gpkg', 'tag': 'Норвегия'},
    'region': {'src': BASE + r'\!!!ТОчняк',
               'gpkg': BASE + r'\outage_tool\elektro_topo.gpkg', 'tag': 'район'},
}
BBOX = None        # (xmin, ymin, xmax, ymax) в EPSG:25833 – обработать только часть сети; None – всё
LOAD_MASTS = True  # False – не загружать опоры (быстрее и меньше памяти)
REBUILD = False    # True – перестроить топологию заново, даже если GeoPackage уже есть
# Параметры топологии (допуски в метрах) – словарь DEFAULT_PARAMS ниже
# ===================================================================
SRC_DIR = PROFILES[PROFILE]['src']
OUT_GPKG = PROFILES[PROFILE]['gpkg']
TAG = PROFILES[PROFILE]['tag']

# ==== TOPO_CORE_BEGIN ====
# Чистый Python (без внешних библиотек): построение топологической сети
# и расчёт последствий аварийных отключений. Используется и в QGIS, и автономно.
from collections import defaultdict, deque

DEFAULT_PARAMS = {
    'SNAP_TOL': 10.0,        # м: концы линий ближе этого расстояния сливаются в один узел
    'TJUNC_TOL': 10.0,       # м: конец линии ближе этого к середине другой линии -> Т-ответвление
    'STATION_GROUP': 80.0,   # м: точки подстанции ближе этого объединяются в одну станцию
    'STATION_RADIUS': 100.0, # м: узлы линий в этом радиусе присоединяются к шинам станции
    'MAST_TOL': 25.0,        # м: опора привязывается к ближайшей линии в этом радиусе
    'MIN_LEN': 0.01,         # м: минимальная длина ребра
    'PARALLEL_COS': 0.94,    # |cos| угла > 0.94 (угол < 20°) - линии параллельны, Т-узел не создаётся
    'EXT_VOLT': 110.0,       # кВ: тупиковые концы линий >= этого напряжения = внешнее питание (граница
района)
    'AUTO_FEED_RADIUS': 1500.0,  # м: изолированный участок РС подключается к ближайшей станции
                                 # условным кабельным вводом (0 - отключить правило)
}


def _d(a, b):
    return math.hypot(a[0] - b[0], a[1] - b[1])
```

```
def _plen(pts):
    return sum(_d(pts[i], pts[i + 1]) for i in range(len(pts) - 1))


class _UF:
    def __init__(self, n):
        self.p = list(range(n))

    def f(self, x):
        p = self.p
        while p[x] != x:
            p[x] = p[p[x]]
            x = p[x]
        return x

    def u(self, a, b):
        a, b = self.f(a), self.f(b)
        if a != b:
            self.p[max(a, b)] = min(a, b)


class _PointGrid:
    def __init__(self, pts, cell):
        self.cell = cell
        self.pts = pts
        self.g = defaultdict(list)
        for i, p in enumerate(pts):
            self.g[(int(p[0] // cell), int(p[1] // cell))].append(i)

    def near(self, p, r):
        c = self.cell
        cx0, cx1 = int((p[0] - r) // c), int((p[0] + r) // c)
        cy0, cy1 = int((p[1] - r) // c), int((p[1] + r) // c)
        out = []
        for cx in range(cx0, cx1 + 1):
            for cy in range(cy0, cy1 + 1):
                for i in self.g.get((cx, cy), ()):
                    if _d(self.pts[i], p) <= r:
                        out.append(i)
        return out


class _SegGrid:
    """Индекс отрезков полилиний: (edge_idx, seg_idx, a, b)."""

    def __init__(self, edges_pts, cell=50.0, skip=None):
        self.cell = cell
        self.g = defaultdict(list)
        self.segs = []
        for ei, pts in enumerate(edges_pts):
            if skip and skip(ei):
                continue
            for si in range(len(pts) - 1):
                a, b = pts[si], pts[si + 1]
                k = len(self.segs)
                self.segs.append((ei, si, a, b))
                # ячейки вдоль отрезка (шаг cell/2), а не весь охватывающий прямоугольник
                L = math.hypot(b[0] - a[0], b[1] - a[1])
                ns = int(L / (cell * 0.5)) + 1
                cells = set()
                for j in range(ns + 1):
                    t = j / ns
                    cells.add((int((a[0] + t * (b[0] - a[0])) // cell), int((a[1] + t * (b[1] -
a[1])) // cell)))
                for c in cells:
                    self.g[c].append(k)

    def near(self, p, r):
        """Возвращает список (dist, edge_idx, seg_idx, t, proj) для отрезков в радиусе r."""
        c = self.cell
        seen = set()
        out = []
        rr = r + c * 0.3
        for cx in range(int((p[0] - rr) // c), int((p[0] + rr) // c) + 1):
            for cy in range(int((p[1] - rr) // c), int((p[1] + rr) // c) + 1):
                for k in self.g.get((cx, cy), ()):
                    if k in seen:
                        continue
                    seen.add(k)
                    ei, si, a, b = self.segs[k]
```

```
                    dx, dy = b[0] - a[0], b[1] - a[1]
                    L2 = dx * dx + dy * dy
                    t = 0.0 if L2 == 0 else max(0.0, min(1.0, ((p[0] - a[0]) * dx + (p[1] - a[1]) * dy)
/ L2))
                    q = (a[0] + t * dx, a[1] + t * dy)
                    dd = _d(p, q)
                    if dd <= r:
                        out.append((dd, ei, si, t, q))
        out.sort(key=lambda x: x[0])
        return out


def _volt_ok(v1, v2):
    return v1 is None or v2 is None or v1 == 0 or v2 == 0 or abs(v1 - v2) < 1e-6 or \
        {v1, v2} <= {22.0, 24.0} or {v1, v2} <= {11.0, 12.0}


def build_topology(lines, stations, masts, params=None, log=print):
    """
    lines:    [{'parts': [[(x,y),..],..], 'layer': str, 'src_id': int, 'name': str, 'volt': float|None,
                'level': str, 'owner': str}]
    stations: [{'pt': (x,y), 'name': str, 'volt': float|None, 'src_id': int}]
    masts:    [{'pt': (x,y), 'src_id': int}]
    Возвращает dict(edges=[...], nodes=[...], masts=[...], stats={...})
    """
    P = dict(DEFAULT_PARAMS)
    if params:
        P.update(params)
    st = {}
    t0 = time.perf_counter()

    # ---- Правило 1-2: одночастные геометрии, очистка, удаление дубликатов ----
    edges = []
    n_multi = n_short = n_dup = n_vtx = 0
    seen = set()
    for ln in lines:
        parts = ln['parts']
        if len(parts) > 1:
            n_multi += 1
        for part in parts:
            pts = []
            for p in part:
                p = (float(p[0]), float(p[1]))
                if pts and _d(pts[-1], p) < 1e-6:
                    n_vtx += 1
                    continue
                pts.append(p)
            if len(pts) < 2 or _plen(pts) < P['MIN_LEN']:
                n_short += 1
                continue
            key = tuple((round(x, 2), round(y, 2)) for x, y in pts)
            if key in seen or tuple(reversed(key)) in seen:
                n_dup += 1
                continue
            seen.add(key)
            e = dict(ln)
            e.pop('parts', None)
            e['pts'] = pts
            edges.append(e)
    st['source_lines'] = len(lines)
    st['multipart_exploded'] = n_multi
    st['short_removed'] = n_short
    st['duplicates_removed'] = n_dup
    st['dup_vertices_removed'] = n_vtx

    # ---- Правило 3: сведение концов линий в общие узлы (snapping) ----
    ends = []
    for e in edges:
        ends.append(e['pts'][0])
        ends.append(e['pts'][-1])
    grid = _PointGrid(ends, P['SNAP_TOL'])
    uf = _UF(len(ends))
    n_before = len(set(ends))
    for i, p in enumerate(ends):
        for j in grid.near(p, P['SNAP_TOL']):
            if j > i:
                uf.u(i, j)
    groups = defaultdict(list)
    for i in range(len(ends)):
        groups[uf.f(i)].append(i)
```

```
    node_xy = {}
    end_node = [None] * len(ends)
    nid = 0
    moved = 0
    for r, idx in groups.items():
        xs = sum(ends[i][0] for i in idx) / len(idx)
        ys = sum(ends[i][1] for i in idx) / len(idx)
        node_xy[nid] = (xs, ys)
        for i in idx:
            end_node[i] = nid
            if _d(ends[i], (xs, ys)) > 1e-6:
                moved += 1
        nid += 1
    for k, e in enumerate(edges):
        e['from'] = end_node[2 * k]
        e['to'] = end_node[2 * k + 1]
        e['pts'][0] = node_xy[e['from']]
        e['pts'][-1] = node_xy[e['to']]
    st['endpoints_snapped'] = moved
    st['nodes_after_snap'] = len(node_xy)

    # ---- Правило 4: Т-ответвления (конец линии на середине другой линии) ----
    inc = defaultdict(list)
    for k, e in enumerate(edges):
        inc[e['from']].append(k)
        inc[e['to']].append(k)
    sg = _SegGrid([e['pts'] for e in edges], cell=max(50.0, P['TJUNC_TOL'] * 3))
    splits = defaultdict(list)  # edge -> [(seg, t, node, proj)]
    for n, xy in node_xy.items():
        my = set(inc[n])
        if len(inc[n]) != 1:
            continue          # только «висячие» концы; узел-развилка не цепляется к проходящей линии
        k0 = inc[n][0]
        p0 = edges[k0]['pts']
        dv = (p0[1][0] - p0[0][0], p0[1][1] - p0[0][1]) if edges[k0]['from'] == n else \
             (p0[-2][0] - p0[-1][0], p0[-2][1] - p0[-1][1])
        myv = [edges[k].get('volt') for k in my]
        for dd, ei, si, t, q in sg.near(xy, P['TJUNC_TOL']):
            if ei in my:
                continue
            tgt = edges[ei]
            sa, sb = tgt['pts'][si], tgt['pts'][si + 1]
            sv = (sb[0] - sa[0], sb[1] - sa[1])
            nn = math.hypot(*dv) * math.hypot(*sv)
            if nn > 0 and abs(dv[0] * sv[0] + dv[1] * sv[1]) / nn > P['PARALLEL_COS']:
                continue      # почти параллельные линии (двухцепные ВЛ) не соединяем
            if _d(q, tgt['pts'][0]) < P['SNAP_TOL'] or _d(q, tgt['pts'][-1]) < P['SNAP_TOL']:
                continue
            if not any(_volt_ok(v, tgt.get('volt')) for v in myv):
                continue
            splits[ei].append((si, t, n, q))
            break
    # переносим узлы на проекцию и режем линии
    alias = {}
    for ei, lst in splits.items():
        for si, t, n, q in lst:
            node_xy[n] = q
    for k, e in enumerate(edges):
        e['pts'][0] = node_xy[e['from']]
        e['pts'][-1] = node_xy[e['to']]
    new_edges = []
    n_tj = 0
    for k, e in enumerate(edges):
        if k not in splits:
            new_edges.append(e)
            continue
        lst = sorted(splits[k], key=lambda s: (s[0], s[1]))
        pts = e['pts']
        cur = [pts[0]]
        cur_from = e['from']
        si_done = 0
        for si, t, n, q in lst:
            while si_done < si:
                si_done += 1
                cur.append(pts[si_done])
            if _d(cur[-1], q) > 1e-6:
                cur.append(q)
            if _plen(cur) < P['MIN_LEN']:
                alias[n] = cur_from
                cur = [q]
```

```
                continue
            ne = dict(e)
            ne['pts'] = cur
            ne['from'] = cur_from
            ne['to'] = n
            new_edges.append(ne)
            n_tj += 1
            cur = [q]
            cur_from = n
        while si_done < len(pts) - 1:
            si_done += 1
            cur.append(pts[si_done])
        ne = dict(e)
        ne['pts'] = cur
        ne['from'] = cur_from
        ne['to'] = e['to']
        if _plen(cur) >= P['MIN_LEN']:
            new_edges.append(ne)
        else:
            alias[e['to']] = cur_from
    for e in new_edges:
        e['from'] = alias.get(e['from'], e['from'])
        e['to'] = alias.get(e['to'], e['to'])
    edges = new_edges
    st['t_junctions'] = sum(len(v) for v in splits.values())

    # удаляем вырожденные петли
    loops = [e for e in edges if e['from'] == e['to'] and _plen(e['pts']) < 2 * P['SNAP_TOL']]
    edges = [e for e in edges if not (e['from'] == e['to'] and _plen(e['pts']) < 2 * P['SNAP_TOL'])]
    st['short_edges_collapsed'] = len(loops)
    for e in edges:
        e['kind'] = 'LINE'

    # ---- Правило 5: подстанции как узлы-источники с шинами ----
    spts = [s['pt'] for s in stations]
    suf = _UF(len(spts))
    sgrid = _PointGrid(spts, P['STATION_GROUP'])
    for i, p in enumerate(spts):
        for j in sgrid.near(p, P['STATION_GROUP']):
            if j > i:
                suf.u(i, j)
    sgroups = defaultdict(list)
    for i in range(len(spts)):
        sgroups[suf.f(i)].append(i)
    used_nodes = set()
    for e in edges:
        used_nodes.add(e['from'])
        used_nodes.add(e['to'])
    used_list = sorted(used_nodes)
    ngrid = _PointGrid([node_xy[n] for n in used_list], P['STATION_RADIUS'])
    station_nodes = {}
    best = {}  # line node -> (dist, station_node)
    for r, idx in sgroups.items():
        idx.sort(key=lambda i: (0 if 'TS' in (stations[i].get('name') or '').upper() else 1, i))
        main = stations[idx[0]]
        names = []
        for i in idx:
            nm = (stations[i].get('name') or '').strip()
            if nm and nm not in names:
                names.append(nm)
        volts = [stations[i].get('volt') for i in idx if stations[i].get('volt')]
        sn = nid
        nid += 1
        node_xy[sn] = main['pt']
        station_nodes[sn] = {'name': ' / '.join(names), 'volt': max(volts) if volts else None}
        for i in idx:
            for j in ngrid.near(spts[i], P['STATION_RADIUS']):
                ln = used_list[j]
                dd = _d(spts[i], node_xy[ln])
                if ln not in best or dd < best[ln][0]:
                    best[ln] = (dd, sn)
    n_bus = n_merge = 0
    merge = {}
    for ln, (dd, sn) in best.items():
        if dd < 0.5:
            merge[ln] = sn
            n_merge += 1
        else:
            edges.append({'kind': 'BUS', 'pts': [node_xy[sn], node_xy[ln]], 'from': sn, 'to': ln,
                          'layer': 'bus', 'name': 'Шина ' + station_nodes[sn]['name'],
```

```
                        'volt': None, 'level': '', 'owner': '', 'src_id': None})
            n_bus += 1
    for e in edges:
        e['from'] = merge.get(e['from'], e['from'])
        e['to'] = merge.get(e['to'], e['to'])
    st['stations'] = len(station_nodes)
    st['bus_edges'] = n_bus
    st['stations_isolated'] = sum(1 for sn in station_nodes if sn not in {b for _, b in best.values()})

    # ---- Правило 6: перенумерация, степени, типы узлов, внешнее питание ----
    deg = defaultdict(int)
    maxv = defaultdict(float)
    for e in edges:
        deg[e['from']] += 1
        deg[e['to']] += 1
        v = e.get('volt') or 0.0
        maxv[e['from']] = max(maxv[e['from']], v)
        maxv[e['to']] = max(maxv[e['to']], v)
    old2new = {}
    nodes = []
    for old in sorted(set(list(deg.keys()) + list(station_nodes.keys()))):
        new = len(nodes) + 1
        old2new[old] = new
        d = deg.get(old, 0)
        if old in station_nodes:
            ntype, name, volt, src = 'STATION', station_nodes[old]['name'], station_nodes[old]['volt'],
1
        else:
            ntype = 'END' if d == 1 else ('PASS' if d == 2 else 'BRANCH')
            name, volt, src = '', None, 0
            if d == 1 and maxv[old] >= P['EXT_VOLT'] > 0:
                ntype, name, volt, src = 'EXTERNAL', 'Внешнее питание (граница района)', maxv[old], 1
        nodes.append({'nid': new, 'x': node_xy[old][0], 'y': node_xy[old][1], 'ntype': ntype,
                      'degree': d, 'is_source': src, 'name': name, 'volt': volt})
    for k, e in enumerate(edges):
        e['eid'] = k + 1
        e['from'] = old2new[e['from']]
        e['to'] = old2new[e['to']]
        e['pts'][0] = (nodes[e['from'] - 1]['x'], nodes[e['from'] - 1]['y'])
        e['pts'][-1] = (nodes[e['to'] - 1]['x'], nodes[e['to'] - 1]['y'])
        e['length'] = _plen(e['pts'])

    # ---- Правило 7: связность ----
    def components():
        adj = defaultdict(list)
        for e in edges:
            adj[e['from']].append(e['to'])
            adj[e['to']].append(e['from'])
        comp = {}
        c = 0
        for n in nodes:
            if n['nid'] in comp:
                continue
            c += 1
            q = deque([n['nid']])
            comp[n['nid']] = c
            while q:
                u = q.popleft()
                for v in adj[u]:
                    if v not in comp:
                        comp[v] = c
                        q.append(v)
        return comp, c

    comp, c = components()
    fed = set(comp[n['nid']] for n in nodes if n['is_source'])
    st['components_initial'] = c
    st['components_with_source_initial'] = len(fed)
    # Правило 7б: условный кабельный ввод к изолированным участкам распределительной сети
    n_virt = 0
    if P['AUTO_FEED_RADIUS'] > 0:
        stn = [n for n in nodes if n['ntype'] == 'STATION']
        bycomp = defaultdict(list)
        for n in nodes:
            if comp[n['nid']] not in fed and n['ntype'] != 'STATION':
                bycomp[comp[n['nid']]].append(n)
        sgr = _PointGrid([(s_['x'], s_['y']) for s_ in stn], max(P['AUTO_FEED_RADIUS'], 1.0))
        for cc, ns in bycomp.items():
            bestv = None
            for a in ns:
```

```
                for j in sgr.near((a['x'], a['y']), P['AUTO_FEED_RADIUS']):
                    s_ = stn[j]
                    dd = math.hypot(a['x'] - s_['x'], a['y'] - s_['y'])
                    if bestv is None or dd < bestv[0]:
                        bestv = (dd, a, s_)
            if bestv:
                dd, a, s_ = bestv
                edges.append({'kind': 'VIRT', 'pts': [(s_['x'], s_['y']), (a['x'], a['y'])],
                              'from': s_['nid'], 'to': a['nid'], 'layer': 'virtual',
                              'name': 'Условный ввод от ' + s_['name'], 'volt': None, 'level': '',
                              'owner': '', 'src_id': None, 'eid': len(edges) + 1, 'length': dd})
                a['degree'] += 1
                s_['degree'] += 1
                n_virt += 1
        comp, c = components()
        fed = set(comp[n['nid']] for n in nodes if n['is_source'])
    st['virtual_feeds'] = n_virt
    for n in nodes:
        n['comp'] = comp[n['nid']]
        n['fed'] = 1 if n['comp'] in fed else 0
    for e in edges:
        e['comp'] = comp[e['from']]
        e['fed'] = 1 if e['comp'] in fed else 0
    st['components'] = c
    st['components_with_source'] = len(fed)
    st['edges'] = len(edges)
    st['line_edges'] = sum(1 for e in edges if e['kind'] == 'LINE')
    st['nodes'] = len(nodes)
    st['node_types'] = dict((t, sum(1 for n in nodes if n['ntype'] == t)) for t in ('STATION',
'EXTERNAL', 'BRANCH', 'PASS', 'END'))
    st['len_km'] = sum(e['length'] for e in edges if e['kind'] == 'LINE') / 1000.0
    st['len_fed_km'] = sum(e['length'] for e in edges if e['kind'] == 'LINE' and e['fed']) / 1000.0

    # ---- Правило 8: привязка опор к линиям ----
    lsg = _SegGrid([e['pts'] for e in edges], cell=50.0, skip=lambda i: edges[i]['kind'] != 'LINE')
    out_m = []
    n_bound = 0
    for k, m in enumerate(masts):
        near = lsg.near(m['pt'], P['MAST_TOL'])
        if near:
            dd, ei = near[0][0], near[0][1]
            eid = edges[ei]['eid']
            n_bound += 1
        else:
            dd, eid = None, None
        out_m.append({'mid': k + 1, 'pt': m['pt'], 'edge_id': eid, 'dist': dd, 'src_id':
m.get('src_id')})
    st['masts'] = len(masts)
    st['masts_bound'] = n_bound
    st['build_sec'] = time.perf_counter() - t0
    return {'edges': edges, 'nodes': nodes, 'masts': out_m, 'stats': st, 'params': P}


class OutageGraph:
    """Граф сети для расчёта отключений. edges: [(eid, from, to)], sources: [nid]."""
    ST_ON, ST_OFF, ST_DEAD, ST_NOFEED = 0, 1, 2, 3

    def __init__(self, edges, node_ids, sources):
        self.edges = {eid: (a, b) for eid, a, b in edges}
        self.node_ids = set(node_ids)
        self.sources = set(sources)
        self.adj = defaultdict(list)
        for eid, a, b in edges:
            self.adj[a].append((eid, b))
            self.adj[b].append((eid, a))
        self.base_nodes = self._reach(set(), set())
        self._base_es = self._base_ns = None

    def _reach(self, off_e, off_n):
        seen = set()
        q = deque()
        for s in self.sources:
            if s not in off_n:
                seen.add(s)
                q.append(s)
        adj = self.adj
        while q:
            u = q.popleft()
            for eid, v in adj[u]:
                if eid in off_e or v in off_n or v in seen:
```

```
                    continue
                seen.add(v)
                q.append(v)
        return seen

    def analyze(self, off_edges=(), off_nodes=()):
        """Возвращает (edge_state, node_state, secs). Состояния:
        0 - под напряжением, 1 - отключено оператором, 2 - обесточено в результате отключения,
        3 - не имеет связи с источником даже в нормальном режиме.
        Инкрементальный алгоритм: пересчитываются только фрагменты сети, примыкающие к отключённым
        элементам; поиск в ширину из фрагмента останавливается при достижении включённого источника."""
        t0 = time.perf_counter()
        off_e, off_n = set(off_edges), set(off_nodes)
        base = self.base_nodes
        if self._base_es is None:
            self._base_es = {eid: (0 if (a in base or b in base) else 3) for eid, (a, b) in
self.edges.items()}
            self._base_ns = {n: (0 if n in base else 3) for n in self.node_ids}
        es = dict(self._base_es)
        ns = dict(self._base_ns)
        self.touched_e, self.touched_n = set(), set()   # элементы, состояние которых может отличаться
от нормального
        if not off_e and not off_n:
            return es, ns, time.perf_counter() - t0
        adj, src = self.adj, self.sources
        # стартовые узлы: концы отключённых рёбер и соседи отключённых узлов
        start = set()
        for eid in off_e:
            a, b = self.edges[eid]
            start.add(a); start.add(b)
        for n in off_n:
            for eid, v in adj[n]:
                start.add(v)
        start -= off_n
        live, dead = set(), set()
        for s0 in start:
            if s0 in live or s0 in dead or s0 not in base:
                continue
            seen = {s0}
            q = deque([s0])
            found = s0 in src
            while q and not found:
                u = q.popleft()
                for eid, v in adj[u]:
                    if eid in off_e or v in off_n or v in seen:
                        continue
                    if v in live or v in src:
                        found = True
                        break
                    seen.add(v)
                    q.append(v)
            if found:
                live |= seen
            else:
                dead |= seen
        for n in off_n:
            ns[n] = 1
        for n in dead:
            ns[n] = 2
        touched = set(off_e)
        for n in dead | off_n:
            for eid, v in adj[n]:
                touched.add(eid)
        self.touched_e, self.touched_n = touched, dead | off_n
        for eid in touched:
            if eid in off_e:
                es[eid] = 1
                continue
            a, b = self.edges[eid]
            la = a in base and a not in dead and a not in off_n
            lb = b in base and b not in dead and b not in off_n
            if la or lb:
                es[eid] = 0
            elif a in base or b in base:
                es[eid] = 2
            else:
                es[eid] = 3
        return es, ns, time.perf_counter() - t0

    def analyze_full(self, off_edges=(), off_nodes=()):
```

```
        """Эталонный (полный) расчёт многоисточниковым поиском в ширину – для проверки."""
        t0 = time.perf_counter()
        off_e, off_n = set(off_edges), set(off_nodes)
        live = self._reach(off_e, off_n)
        base = self.base_nodes
        es = {}
        for eid, (a, b) in self.edges.items():
            if eid in off_e:
                es[eid] = 1
            elif a in live or b in live:
                es[eid] = 0
            elif a in base or b in base:
                es[eid] = 2
            else:
                es[eid] = 3
        ns = {}
        for n in self.node_ids:
            if n in off_n:
                ns[n] = 1
            elif n in live:
                ns[n] = 0
            elif n in base:
                ns[n] = 2
            else:
                ns[n] = 3
        return es, ns, time.perf_counter() - t0
# ==== TOPO_CORE_END ====

# ======================================================================
#                ЧАСТЬ QGIS: построение слоёв, стили, панель
# ======================================================================
import os, random, csv, datetime
from collections import defaultdict
from qgis.core import (QgsProject, QgsVectorLayer, QgsFeature, QgsGeometry, QgsPointXY,
QgsVectorFileWriter,
                       QgsCoordinateTransform, QgsRuleBasedRenderer, QgsLineSymbol, QgsMarkerSymbol,
                       QgsPalLayerSettings, QgsTextFormat, QgsVectorLayerSimpleLabeling, QgsRectangle,
                       QgsFeatureRequest, NULL)
from qgis.PyQt.QtCore import Qt
from qgis.PyQt.QtGui import QColor, QFont
from qgis.PyQt.QtWidgets import (QDockWidget, QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
QListWidget,
                                 QLabel, QTextEdit, QCheckBox, QMessageBox, QGroupBox, QLineEdit,
QApplication)
from qgis.gui import QgsMapTool
from qgis.utils import iface

SRC_LAYERS = ['el_luftlinje', 'el_hengekabel', 'el_sjokabel']
STATION_LAYER = 'el_transformatorstasjon'
MAST_LAYER = 'el_mast'
L_EDGES, L_NODES, L_MASTS = 'net_edges', 'net_nodes', 'net_masts'
TITLE = {L_EDGES: 'Сеть (%s): линии (рёбра)' % TAG, L_NODES: 'Сеть (%s): узлы' % TAG, L_MASTS: 'Сеть
(%s): опоры' % TAG}
GROUP_NAME = 'Анализ аварийных отключений – %s' % TAG
STATE_TXT = {0: 'под напряжением', 1: 'отключено оператором', 2: 'обесточено', 3: 'нет связи с
источником'}
PICK_MAX_TOL = 1500.0   # м: при более мелком масштабе выбор на карте отключается (слишком много
объектов)


def _msg(text):
    print(text)
    try:
        iface.mainWindow().statusBar().showMessage(text)
        QApplication.processEvents()
    except Exception:
        pass


def _val(f, name, conv=None):
    try:
        v = f.attribute(name)
    except KeyError:
        return None
    if v is None or v == NULL:
        return None
    if conv:
        try:
            return conv(v)
        except (TypeError, ValueError):
```

```
            return None
    return v


def _src_layer(name, src):
    """Слой исходных данных: из FileGDB (.gdb), из папки shp или уже открытый в проекте из того же
источника."""
    s_norm = src.replace('\\', '/').lower()
    for l in QgsProject.instance().mapLayersByName(name):
        if isinstance(l, QgsVectorLayer) and l.isValid() and l.source().replace('\\',
'/').lower().startswith(s_norm):
            return l
    if src.lower().endswith('.gdb'):
        l = QgsVectorLayer('%s|layername=%s' % (src, name), name, 'ogr')
        return l if l.isValid() else None
    p = os.path.join(src, name + '.shp')
    if os.path.exists(p):
        l = QgsVectorLayer(p, name, 'ogr')
        if l.isValid():
            return l
    return None


def _request(lyr, fields, bbox=None, geom=True):
    rq = QgsFeatureRequest()
    names = [n for n in fields if lyr.fields().indexOf(n) >= 0]
    rq.setSubsetOfAttributes(names, lyr.fields())
    if not geom:
        rq.setFlags(QgsFeatureRequest.NoGeometry)
    if bbox:
        rq.setFilterRect(QgsRectangle(*bbox))
    return rq


def _xy_parts(g):
    if g is None or g.isEmpty():
        return []
    if g.isMultipart():
        return [[(p.x(), p.y()) for p in part] for part in g.asMultiPolyline()]
    return [[(p.x(), p.y()) for p in g.asPolyline()]]


def _xy_point(g):
    if g is None or g.isEmpty():
        return None
    p = g.asMultiPoint()[0] if g.isMultipart() else g.asPoint()
    return (p.x(), p.y())


def _write_layer(mem, path, lname, first):
    o = QgsVectorFileWriter.SaveVectorOptions()
    o.driverName = 'GPKG'
    o.layerName = lname
    o.fileEncoding = 'UTF-8'
    o.actionOnExistingFile = (QgsVectorFileWriter.CreateOrOverwriteFile if first
                              else QgsVectorFileWriter.CreateOrOverwriteLayer)
    res = QgsVectorFileWriter.writeAsVectorFormatV3(mem, path,
QgsProject.instance().transformContext(), o)
    if res[0] != QgsVectorFileWriter.NoError:
        raise RuntimeError('Ошибка записи %s: %s' % (lname, res[1]))


def _add_chunked(layer, feats, chunk=50000):
    for i in range(0, len(feats), chunk):
        layer.dataProvider().addFeatures(feats[i:i + chunk])


def build_network(src=None, gpkg=None, params=None, load=True, bbox=None, masts=None):
    """Правила топологии -> GeoPackage со слоями net_edges / net_nodes / net_masts."""
    src = src or SRC_DIR
    gpkg = gpkg or OUT_GPKG
    bbox = bbox if bbox is not None else BBOX
    masts_on = LOAD_MASTS if masts is None else masts
    os.makedirs(os.path.dirname(gpkg), exist_ok=True)
    t0 = time.perf_counter()
    lines, crs = [], None
    for name in SRC_LAYERS:
        lyr = _src_layer(name, src)
        if lyr is None:
            print('  ! слой не найден:', name)
```

```
            continue
        crs = crs or lyr.crs()
        _msg('Чтение %s…' % name)
        for f in lyr.getFeatures(_request(lyr, ['OBJECTID', 'navn', 'spenning', 'nettniva', 'eier'],
bbox)):
            parts = _xy_parts(f.geometry())
            if parts:
                lines.append({'parts': parts, 'layer': name, 'src_id': _val(f, 'OBJECTID', int) or
f.id(),
                              'name': _val(f, 'navn', str) or '', 'volt': _val(f, 'spenning', float),
                              'level': _val(f, 'nettniva', str) or '', 'owner': _val(f, 'eier', str) or
''})
    if not lines:
        raise RuntimeError('Не найдены исходные линии. Проверьте источник: %s' % src)
    stations, masts = [], []
    lyr = _src_layer(STATION_LAYER, src)
    if lyr:
        for f in lyr.getFeatures(_request(lyr, ['OBJECTID', 'navn', 'spenning'], bbox)):
            p = _xy_point(f.geometry())
            if p:
                v = _val(f, 'spenning', float)
                stations.append({'pt': p, 'name': _val(f, 'navn', str) or '', 'volt': v or None,
                                 'src_id': _val(f, 'OBJECTID', int) or f.id()})
    lyr = _src_layer(MAST_LAYER, src) if masts_on else None
    if lyr:
        _msg('Чтение опор…')
        for f in lyr.getFeatures(_request(lyr, ['OBJECTID'], bbox)):
            p = _xy_point(f.geometry())
            if p:
                masts.append({'pt': p, 'src_id': _val(f, 'OBJECTID', int) or f.id()})
    _msg('Прочитано за %.0f с: линий %d, станций %d, опор %d. Построение топологии…'
         % (time.perf_counter() - t0, len(lines), len(stations), len(masts)))
    res = build_topology(lines, stations, masts, params)
    del lines
    authid = crs.authid() if crs else 'EPSG:25833'
    _msg('Топология построена (%.0f с). Запись GeoPackage…' % (time.perf_counter() - t0))

    me = QgsVectorLayer('LineString?crs=
%s&field=edge_id:integer&field=from_node:integer&field=to_node:integer'

'&field=kind:string(8)&field=src_layer:string(24)&field=src_id:integer&field=name:string(120)'

'&field=volt:double&field=level:string(40)&field=owner:string(80)&field=length:double'
                       '&field=comp:integer&field=fed:integer&field=state:integer' % authid, L_EDGES,
'memory')
    feats = []
    for e in res['edges']:
        f = QgsFeature(me.fields())
        f.setGeometry(QgsGeometry.fromPolylineXY([QgsPointXY(x, y) for x, y in e['pts']]))
        f.setAttributes([e['eid'], e['from'], e['to'], e['kind'], e.get('layer'), e.get('src_id'),
                         (e.get('name') or '')[:120], e.get('volt'), e.get('level'), (e.get('owner') or
'')[:80],
                         round(e['length'], 2), e['comp'], e['fed'], 0 if e['fed'] else 3])
        feats.append(f)
    _add_chunked(me, feats)

    mn = QgsVectorLayer('Point?crs=
%s&field=node_id:integer&field=ntype:string(10)&field=degree:integer'

'&field=is_source:integer&field=name:string(120)&field=volt:double&field=comp:integer'
                       '&field=fed:integer&field=state:integer' % authid, L_NODES, 'memory')
    feats = []
    for n in res['nodes']:
        f = QgsFeature(mn.fields())
        f.setGeometry(QgsGeometry.fromPointXY(QgsPointXY(n['x'], n['y'])))
        f.setAttributes([n['nid'], n['ntype'], n['degree'], n['is_source'], (n['name'] or '')[:120],
n['volt'],
                         n['comp'], n['fed'], 0 if n['fed'] else 3])
        feats.append(f)
    _add_chunked(mn, feats)

    fed_e = {e['eid']: e['fed'] for e in res['edges']}
    mm = QgsVectorLayer('Point?crs=%s&field=mast_id:integer&field=src_id:integer&field=edge_id:integer'
                       '&field=dist:double&field=state:integer' % authid, L_MASTS, 'memory')
    feats = []
    for m in res['masts']:
        f = QgsFeature(mm.fields())
        f.setGeometry(QgsGeometry.fromPointXY(QgsPointXY(*m['pt'])))
        st = 0 if (m['edge_id'] and fed_e.get(m['edge_id'])) else 3
        f.setAttributes([m['mid'], m['src_id'], m['edge_id'],
```

```
                        round(m['dist'], 2) if m['dist'] is not None else None, st])
            feats.append(f)
        _add_chunked(mm, feats)
        feats = None

        # снимаем слои с карты, если они уже открыты (иначе файл заблокирован)
        for lid, l in list(QgsProject.instance().mapLayers().items()):
            if l.source().replace('\\', '/').lower().startswith(gpkg.replace('\\', '/').lower()):
                QgsProject.instance().removeMapLayer(lid)
        _write_layer(me, gpkg, L_EDGES, True)
        _write_layer(mn, gpkg, L_NODES, False)
        _write_layer(mm, gpkg, L_MASTS, False)
        me = mn = mm = None

        st = res['stats']
        rep = os.path.splitext(gpkg)[0] + '_topology_report.txt'
        with open(rep, 'w', encoding='utf-8') as fh:
            fh.write('Отчёт о построении топологии (%s), источник: %s\n' %
(datetime.datetime.now().strftime('%Y-%m-%d %H:%M'), src))
            fh.write('  BBOX = %s\n' % (bbox,))
            for k, v in res['params'].items():
                fh.write('  параметр %s = %s\n' % (k, v))
            for k, v in st.items():
                fh.write('  %s: %s\n' % (k, v))
            fh.write('  total_sec (с чтением и записью): %.1f\n' % (time.perf_counter() - t0))
        _msg('Топология построена за %.0f с: рёбер %d, узлов %d, компонент %d (с источником %d). Файл: %s'
             % (time.perf_counter() - t0, st['edges'], st['nodes'], st['components'],
st['components_with_source'], gpkg))
        if load:
            load_network_layers(gpkg)
        return st


    # ---------------------------------------------------------------- стили
    def _line(color, width, style='solid'):
        return QgsLineSymbol.createSimple({'color': color, 'width': str(width), 'line_style': style,
                                           'capstyle': 'round', 'joinstyle': 'round'})


    def _mark(shape, color, size, outline='#222222'):
        return QgsMarkerSymbol.createSimple({'name': shape, 'color': color, 'size': str(size),
                                             'outline_color': outline, 'outline_width': '0.2'})


    def _rules(layer, rules):
        root = QgsRuleBasedRenderer.Rule(None)
        for sym, expr, label, min_scale in rules:
            if min_scale == -1:     # правило только для мелких масштабов (мельче 1:250 000)
                r = QgsRuleBasedRenderer.Rule(sym, 250000, 0, expr, label)
            else:
                r = QgsRuleBasedRenderer.Rule(sym, 0, min_scale, expr, label)
            root.appendChild(r)
        layer.setRenderer(QgsRuleBasedRenderer(root))


    def style_layers(le, ln, lm):
        if le:
            _rules(le, [
                (_line('#bdbdbd', 0.26), '"state" = 3', 'Нет связи с источником (разрыв в исходных
данных)', 0),
                (_line('#666666', 0.3, 'dot'), '"state" = 0 AND "kind" <> \'LINE\'', 'Шина / условный
ввод', 0),
                (_line('#2e8b57', 0.35), '"state" = 0 AND "kind" = \'LINE\' AND coalesce("volt",0) < 50',
'до 50 кВ — в работе', 0),
                (_line('#8e44ad', 0.6), '"state" = 0 AND "kind" = \'LINE\' AND "volt" >= 50 AND "volt" <
110', '50–109 кВ — в работе', 0),
                (_line('#1f3a93', 0.8), '"state" = 0 AND "kind" = \'LINE\' AND "volt" >= 110', '110–420 кВ
— в работе', 0),
                (_line('#ff8c00', 1.0), '"state" = 2', 'ОБЕСТОЧЕНО вследствие отключения', 0),
                (_line('#e00000', 1.5, 'dash'), '"state" = 1', 'ОТКЛЮЧЕНО оператором', 0),
            ])
        if lm:
            _rules(lm, [
                (_mark('circle', '#9e9e9e', 0.7, '#9e9e9e'), '"state" IN (0, 3)', 'Опора', 60000),
                (_mark('circle', '#b35900', 0.8, '#b35900'), '"state" = 2', 'Опора обесточенной линии',
60000),
                (_mark('circle', '#e00000', 0.9, '#e00000'), '"state" = 1', 'Опора отключённой линии',
60000),
            ])
        if ln:
```

```
        _rules(ln, [
            (_mark('circle', '#424242', 1.0), '"state" IN (0, 3) AND "is_source" = 0', 'Узел сети',
60000),
            (_mark('circle', '#ff8c00', 1.2, '#ff8c00'), '"state" = 2', 'Обесточенный узел', 250000),
            (_mark('triangle', '#6a5acd', 3.0), '"state" = 0 AND "is_source" = 1 AND "ntype"
<> \'STATION\'',
             'Внешнее питание / электростанция (тупик ВЛ ≥110 кВ)', 250000),
            (_mark('square', '#0050c8', 3.6), '"state" = 0 AND "ntype" = \'STATION\'',
'Трансформаторная станция (источник)', 250000),
            (_mark('square', '#0050c8', 0.7, '#0050c8'), '"state" = 0 AND "ntype" = \'STATION\'',
'Станция (мелкий масштаб)', -1),
            (_mark('cross2', '#e00000', 5.0, '#e00000'), '"state" = 1', 'ОТКЛЮЧЁННЫЙ узел', 0),
        ])
        s = QgsPalLayerSettings()
        s.fieldName = 'CASE WHEN "ntype" = \'STATION\' THEN "name" END'
        s.isExpression = True
        s.scaleVisibility = True
        s.minimumScale = 400000   # подписи станций – только при масштабе крупнее 1:400 000
        s.maximumScale = 0
        tf = QgsTextFormat()
        tf.setSize(8)
        tf.setColor(QColor('#0a2a6b'))
        s.setFormat(tf)
        ln.setLabeling(QgsVectorLayerSimpleLabeling(s))
        ln.setLabelsEnabled(True)
    for l in (le, ln, lm):
        if l:
            l.triggerRepaint()


def load_network_layers(gpkg=None):
    gpkg = gpkg or OUT_GPKG
    prj = QgsProject.instance()
    root = prj.layerTreeRoot()
    grp = root.findGroup(GROUP_NAME) or root.insertGroup(0, GROUP_NAME)
    out = {}
    for lname in (L_NODES, L_EDGES, L_MASTS):
        for l in prj.mapLayersByName(TITLE[lname]):
            prj.removeMapLayer(l.id())
        l = QgsVectorLayer('%s|layername=%s' % (gpkg, lname), TITLE[lname], 'ogr')
        if not l.isValid():
            raise RuntimeError('Не удалось открыть %s из %s' % (lname, gpkg))
        prj.addMapLayer(l, False)
        grp.addLayer(l)
        out[lname] = l
    style_layers(out[L_EDGES], out[L_NODES], out[L_MASTS])
    return out


# ---------------------------------------------------------------- инструмент карты
class PickTool(QgsMapTool):
    def __init__(self, canvas, callback):
        super().__init__(canvas)
        self.callback = callback
        self.setCursor(Qt.CrossCursor)

    def canvasReleaseEvent(self, e):
        pt = self.toMapCoordinates(e.pos())
        tol = self.canvas().mapUnitsPerPixel() * 7
        self.callback(pt, tol, e.button() == Qt.RightButton)


# ---------------------------------------------------------------- анализатор
class OutageTool:
    def __init__(self):
        prj = QgsProject.instance()

        def get(lname):
            ls = prj.mapLayersByName(TITLE[lname])
            return ls[0] if ls else None
        self.le, self.ln, self.lm = get(L_EDGES), get(L_NODES), get(L_MASTS)
        if not self.le or not self.ln:
            raise RuntimeError('Слои сети не загружены. Выполните build_network() или
load_network_layers().')
        self.off_e, self.off_n = set(), set()
        self.last = None
        self.manual_t0 = None
        self._node_idx = self._edge_idx = None
        self.reload()
```

```
    # ---- загрузка графа
    def reload(self):
        t0 = time.perf_counter()
        _msg('Загрузка графа сети…')
        self.E, self.N = {}, {}
        self.cur_e, self.cur_n = {}, {}
        if self._reload_sqlite():
            pass
        else:
            self._reload_qgis()
        self.fid2eid = {v['fid']: k for k, v in self.E.items()}
        self.fid2nid = {v['fid']: k for k, v in self.N.items()}
        self._reload_masts()
        self.G = OutageGraph([(k, v['a'], v['b']) for k, v in self.E.items()], list(self.N.keys()),
                             [k for k, v in self.N.items() if v['src']])
        _msg('Граф загружен: %d рёбер, %d узлов, %d источников, %d опор, %.1f с'
             % (len(self.E), len(self.N), len(self.G.sources), self.n_masts, time.perf_counter() - t0))

    def _reload_sqlite(self):
        """Быстрое чтение атрибутов напрямую из GeoPackage (SQLite)."""
        import sqlite3
        src_e, src_n = self.le.source().split('|')[0], self.ln.source().split('|')[0]
        if not (src_e.lower().endswith('.gpkg') and src_n.lower().endswith('.gpkg')):
            return False
        try:
            con = sqlite3.connect(src_e)
            for fid, eid, a, b, kind, name, volt, lay, ln, st in con.execute(
                    'SELECT fid, edge_id, from_node, to_node, kind, name, volt, src_layer, length,
state FROM %s' % L_EDGES):
                self.E[eid] = {'fid': fid, 'a': a, 'b': b, 'kind': kind, 'name': name or '', 'volt':
volt,
                               'layer': lay or '', 'len': ln or 0.0}
                self.cur_e[eid] = st
            con.close()
            con = sqlite3.connect(src_n)
            for fid, nid, src, nt, name, st in con.execute(
                    'SELECT fid, node_id, is_source, ntype, name, state FROM %s' % L_NODES):
                self.N[nid] = {'fid': fid, 'src': src or 0, 'type': nt, 'name': name or ''}
                self.cur_n[nid] = st
            con.close()
            return True
        except Exception as ex:
            print('Чтение через SQLite невозможно (%s), используется QGIS' % ex)
            self.E, self.N, self.cur_e, self.cur_n = {}, {}, {}, {}
            return False

    def _reload_qgis(self):
        for f in self.le.getFeatures(_request(self.le, ['edge_id', 'from_node', 'to_node', 'kind',
'name', 'volt',
                                                        'src_layer', 'length', 'state'], geom=False)):
            eid = f['edge_id']
            self.E[eid] = {'fid': f.id(), 'a': f['from_node'], 'b': f['to_node'], 'kind': f['kind'],
                           'name': _val(f, 'name', str) or '', 'volt': _val(f, 'volt', float),
                           'layer': _val(f, 'src_layer', str) or '', 'len': _val(f, 'length', float) or
0.0}
            self.cur_e[eid] = _val(f, 'state', int)
        for f in self.ln.getFeatures(_request(self.ln, ['node_id', 'is_source', 'ntype', 'name',
'state'], geom=False)):
            nid = f['node_id']
            self.N[nid] = {'fid': f.id(), 'src': _val(f, 'is_source', int) or 0, 'type': f['ntype'],
                           'name': _val(f, 'name', str) or ''}
            self.cur_n[nid] = _val(f, 'state', int)

    def _reload_masts(self):
        # опоры не загружаются в память: их идентификаторы выбираются из GeoPackage по индексу edge_id
        self.mdb = None
        self.n_masts = 0
        if self.lm:
            try:
                import sqlite3
                path = self.lm.source().split('|')[0]
                self.mdb = sqlite3.connect(path, check_same_thread=False)
                self.mdb.execute('CREATE INDEX IF NOT EXISTS idx_%s_edge ON %s(edge_id)' % (L_MASTS,
L_MASTS))
                self.mdb.commit()
                self.n_masts = self.mdb.execute('SELECT count(*) FROM %s' % L_MASTS).fetchone()[0]
            except Exception as ex:
                print('Опоры: доступ через SQLite невозможен (%s)' % ex)
                self.mdb = None
```

```
    def _masts_of(self, eids, count=False):
        """Идентификаторы (или число) опор, привязанных к рёбрам eids."""
        if not self.mdb or not eids:
            return 0 if count else []
        eids = list(eids)
        res = 0 if count else []
        for i in range(0, len(eids), 5000):
            lst = ','.join(str(int(e)) for e in eids[i:i + 5000])
            if count:
                res += self.mdb.execute('SELECT count(*) FROM %s WHERE edge_id IN (%s)' % (L_MASTS,
lst)).fetchone()[0]
            else:
                res += [(r[0], r[1]) for r in self.mdb.execute('SELECT fid, edge_id FROM %s WHERE
edge_id IN (%s)' % (L_MASTS, lst))]
        return res

    # ---- управление списком отключений
    def disconnect(self, lines=(), nodes=(), run=True):
        """Добавить линии (edge_id) и узлы (node_id) к отключению и пересчитать. Возвращает краткий
итог."""
        bad = [e for e in lines if e not in self.E] + [n for n in nodes if n not in self.N]
        self.off_e.update(e for e in lines if e in self.E)
        self.off_n.update(n for n in nodes if n in self.N)
        if bad:
            print('Не найдены id:', bad)
        if not run:
            return None
        r = self.run()
        dead = [k for k, s in r['es'].items() if s == 2 and self.E[k]['kind'] == 'LINE']
        km = sum(self.E[k]['len'] for k in dead) / 1000
        return 'Обесточено участков: %d (%.2f км), расчёт %.1f мс' % (len(dead), km, r['t_calc'] *
1000)

    def toggle_line(self, eid):
        self.off_e.symmetric_difference_update({eid})

    def toggle_node(self, nid):
        self.off_n.symmetric_difference_update({nid})

    def clear(self):
        self.off_e.clear()
        self.off_n.clear()

    def add_selected(self):
        n = 0
        for f in self.le.selectedFeatures():
            self.off_e.add(f['edge_id'])
            n += 1
        for f in self.ln.selectedFeatures():
            self.off_n.add(f['node_id'])
            n += 1
        self.le.removeSelection()
        self.ln.removeSelection()
        return n

    # ---- расчёт
    def run(self, paint=True):
        es, ns, dt = self.G.analyze(self.off_e, self.off_n)
        t1 = time.perf_counter()
        if paint:
            self._paint(es, ns)
        self.last = {'es': es, 'ns': ns, 't_calc': dt, 't_total': dt + (time.perf_counter() - t1)}
        return self.last

    def _paint(self, es, ns):
        # после первого расчёта сравниваются только элементы, затронутые текущим и предыдущим
отключением
        te, tn = getattr(self.G, 'touched_e', None), getattr(self.G, 'touched_n', None)
        if te is not None and getattr(self, '_prev_te', None) is not None:
            keys_e, keys_n = te | self._prev_te, tn | self._prev_tn
        else:
            keys_e, keys_n = es.keys(), ns.keys()
        self._prev_te, self._prev_tn = set(te or ()), set(tn or ())
        ch_edges = [k for k in keys_e if self.cur_e.get(k) != es[k]]
        ms = {fid: es[eid] for fid, eid in self._masts_of(ch_edges)}
        for lyr, cur, new, keys, key in ((self.le, self.cur_e, es, keys_e, lambda k: self.E[k]['fid']),
                                         (self.ln, self.cur_n, ns, keys_n, lambda k: self.N[k]
['fid'])):
            idx = lyr.fields().indexOf('state')
            ch = {}
```

```
            for k in keys:
                s = new[k]
                if cur.get(k) != s:
                    ch[key(k)] = {idx: s}
                    cur[k] = s
            if ch:
                lyr.dataProvider().changeAttributeValues(ch)
                lyr.triggerRepaint()
        if self.lm and ms:
            idx = self.lm.fields().indexOf('state')
            self.lm.dataProvider().changeAttributeValues({fid: {idx: s} for fid, s in ms.items()})
            self.lm.triggerRepaint()

    def reset(self):
        self.clear()
        return self.run()

    # ---- отчёт
    def report(self):
        if not self.last:
            self.run()
        es, ns = self.last['es'], self.last['ns']
        dead = [k for k, s in es.items() if s == 2 and self.E[k]['kind'] == 'LINE']
        by_v = {}
        for k in dead:
            v = self.E[k]['volt']
            key = '%g кВ' % v if v else 'н/д'
            by_v[key] = by_v.get(key, 0.0) + self.E[k]['len']
        names = sorted({self.E[k]['name'] for k in dead if self.E[k]['name']})
        dnodes = [k for k, s in ns.items() if s == 2]
        dst = [self.N[k]['name'] for k in dnodes if self.N[k]['type'] == 'STATION']
        n_masts = self._masts_of([k for k, s in es.items() if s == 2], count=True)
        lines = ['<b>Отключено оператором:</b> линий %d, узлов %d' % (len(self.off_e),
len(self.off_n)),
                 '<b style="color:#d35400">Обесточено участков линий:</b> %d (%.2f км)'
                 % (len(dead), sum(self.E[k]['len'] for k in dead) / 1000.0)]
        for k in sorted(by_v, key=lambda x: -by_v[x]):
            lines.append('  • %s: %.2f км' % (k, by_v[k] / 1000.0))
        lines.append('<b>Обесточено узлов:</b> %d (ответвлений %d, конечных %d)'
                     % (len(dnodes), sum(1 for k in dnodes if self.N[k]['type'] == 'BRANCH'),
                        sum(1 for k in dnodes if self.N[k]['type'] == 'END')))
        lines.append('<b>Обесточено опор:</b> %d' % n_masts)
        if dst:
            lines.append('<b>Потеряли питание подстанции (%d):</b> %s' % (len(dst), ';
'.join(dst[:20])))
        off_src = [self.N[k]['name'] or self.N[k]['type'] for k in self.off_n if self.N[k]['src']]
        if off_src:
            lines.append('<b>Отключены источники:</b> ' + '; '.join(off_src))
        if names:
            lines.append('<b>Затронутые линии (%d):</b> %s%s' % (len(names), '; '.join(names[:25]),
                                                                 ' …' if len(names) > 25 else ''))
        lines.append('<i>Время расчёта: %.1f мс (с перерисовкой %.1f мс); сеть: %d рёбер</i>'
                     % (self.last['t_calc'] * 1000, self.last['t_total'] * 1000, len(self.E)))
        return '<br>'.join(lines)

    def export_csv(self, path=None):
        if not self.last:
            self.run()
        path = path or os.path.join(os.path.dirname(OUT_GPKG), 'reports',
                                    'outage_%s_%s.csv' % (PROFILE, datetime.datetime.now().strftime('%Y
%m%d_%H%M%S')))
        os.makedirs(os.path.dirname(path), exist_ok=True)
        with open(path, 'w', newline='', encoding='utf-8-sig') as fh:
            w = csv.writer(fh, delimiter=';')
            w.writerow(['тип', 'id', 'состояние', 'вид', 'название', 'напряжение_кВ', 'исходный_слой',
'длина_м'])
            for k, s in sorted(self.last['es'].items()):
                if s in (1, 2):
                    e = self.E[k]
                    w.writerow(['линия', k, STATE_TXT[s], e['kind'], e['name'], e['volt'] or '',
e['layer'],
                                round(e['len'], 1)])
            for k, s in sorted(self.last['ns'].items()):
                if s in (1, 2):
                    n = self.N[k]
                    w.writerow(['узел', k, STATE_TXT[s], n['type'], n['name'], '', '', ''])
        return path

    def dead_extent(self):
        fe = [self.E[k]['fid'] for k, s in (self.last or {}).get('es', {}).items() if s in (1, 2)]
```

```
        fn = [self.N[k]['fid'] for k in self.off_n]
        r = None
        for lyr, fids in ((self.le, fe), (self.ln, fn)):
            if not fids:
                continue
            rq = QgsFeatureRequest().setFilterFids(fids)
            rq.setSubsetOfAttributes([])
            for f in lyr.getFeatures(rq):
                b = f.geometry().boundingBox()
                if r is None:
                    r = QgsRectangle(b)
                else:
                    r.combineExtentWith(b)
        if r is None:
            return None
        pad = max(r.width(), r.height(), 200) * 0.15
        return QgsRectangle(r.xMinimum() - pad, r.yMinimum() - pad, r.xMaximum() + pad, r.yMaximum() +
pad)

    # ---- поиск объекта по клику (через пространственный индекс GeoPackage)
    def pick(self, x, y, tol):
        pt = QgsGeometry.fromPointXY(QgsPointXY(x, y))
        best = None
        rq = QgsFeatureRequest().setFilterRect(QgsRectangle(x - 3 * tol, y - 3 * tol, x + 3 * tol, y +
3 * tol))
        rq.setSubsetOfAttributes([])
        for f in self.ln.getFeatures(rq):
            k = self.fid2nid.get(f.id())
            if k is None:
                continue
            d = f.geometry().distance(pt)
            if self.N[k]['src']:
                d = d / 3.0   # приоритет станций и точек питания (они рисуются крупным значком)
            if d <= tol and (best is None or d < best[0]):
                best = (d, 'node', k)
        if best:
            return best[1], best[2]
        rq = QgsFeatureRequest().setFilterRect(QgsRectangle(x - tol, y - tol, x + tol, y + tol))
        rq.setSubsetOfAttributes([])
        for f in self.le.getFeatures(rq):
            k = self.fid2eid.get(f.id())
            if k is None:
                continue
            d = f.geometry().distance(pt)
            if d <= tol and (best is None or d < best[0]):
                best = (d, 'line', k)
        return (best[1], best[2]) if best else (None, None)

    def find(self, text, limit=500):
        """Поиск линий и станций по наименованию (без учёта регистра). Возвращает (рёбра, узлы)."""
        t = text.strip().lower()
        if not t:
            return [], []
        es = [k for k, e in self.E.items() if t in e['name'].lower()][:limit]
        ns = [k for k, n in self.N.items() if n['name'] and t in n['name'].lower()][:limit]
        return es, ns

    def label(self, kind, k):
        if kind == 'line':
            e = self.E[k]
            v = ('%g кВ' % e['volt']) if e['volt'] else ('шина' if e['kind'] == 'BUS' else e['kind'])
            return 'Линия #%d  %s  (%s, %.0f м)' % (k, e['name'] or '—', v, e['len'])
        n = self.N[k]
        return 'Узел #%d  %s %s' % (k, n['type'], n['name'])

    # ---- производительность
    def benchmark(self, n=None, max_lines=3, seed=1, t0_manual=60.0, t_edge=8.0, t_branch=15.0):
        """Случайные сценарии отключения: время программы vs модель ручного анализа оператором."""
        n = n or (100 if len(self.E) < 20000 else 30)
        rnd = random.Random(seed)
        cand = [k for k, e in self.E.items() if e['kind'] == 'LINE' and self.G.edges[k][0] in
self.G.base_nodes]
        rows = []
        for i in range(n):
            off = rnd.sample(cand, rnd.randint(1, max_lines))
            es, ns, dt = self.G.analyze(off, ())
            nd = sum(1 for k, s in es.items() if s == 2 and self.E[k]['kind'] == 'LINE')
            nb = sum(1 for k, s in ns.items() if s == 2 and self.N[k]['type'] == 'BRANCH')
            manual = t0_manual + t_edge * (nd + len(off)) + t_branch * nb
            rows.append((len(off), nd, nb, dt, manual))
```

```
        tp = sum(r[3] for r in rows) / n
        tm = sum(r[4] for r in rows) / n
        full = self.run()
        txt = ('Сеть: %d рёбер. Сценариев: %d. Среднее: обесточено %.1f участков, %.1f
ответвлений.<br>'
               'Программа: %.1f мс на расчёт (с перерисовкой ~%.0f мс).<br>'
               'Модель ручного анализа: %.0f с (%.1f мин) = %g + %g·N<sub>уч</sub> +
%g·N<sub>отв</sub>.<br>'
               '<b>Ускорение ≈ %.0f раз</b> (по полному времени с перерисовкой).'
               % (len(self.E), n, sum(r[1] for r in rows) / n, sum(r[2] for r in rows) / n, tp * 1000,
                  (tp + full['t_total'] - full['t_calc']) * 1000, tm, tm / 60, t0_manual, t_edge,
t_branch,
                  tm / max(tp + full['t_total'] - full['t_calc'], 1e-6)))
        return txt, rows

    # ---- ручной замер
    def manual_start(self):
        """Показывает только отключённые оператором элементы (без последствий) и запускает
секундомер."""
        es, ns, dt = self.G.analyze(self.off_e, self.off_n)
        self._paint({k: (1 if s == 1 else (3 if s == 3 else 0)) for k, s in es.items()},
                    {k: (1 if s == 1 else (3 if s == 3 else 0)) for k, s in ns.items()})
        self.le.removeSelection()
        iface.setActiveLayer(self.le)
        iface.actionSelect().trigger()
        self.manual_t0 = time.perf_counter()

    def manual_stop(self):
        if self.manual_t0 is None:
            return 'Ручной замер не запущен.'
        tm = time.perf_counter() - self.manual_t0
        self.manual_t0 = None
        sel = {f['edge_id'] for f in self.le.selectedFeatures()}
        r = self.run()
        truth = {k for k, s in r['es'].items() if s == 2 and self.E[k]['kind'] == 'LINE'}
        sel_l = {k for k in sel if self.E[k]['kind'] == 'LINE'}
        tp = len(sel_l & truth)
        fp = len(sel_l - truth)
        fn = len(truth - sel_l)
        self.le.removeSelection()
        rec = tp / max(len(truth), 1)
        txt = ('Ручной метод: <b>%.1f с</b>, выбрано %d участков: верно %d, лишних %d, пропущено %d '
               '(полнота %.0f%%).<br>Программа: <b>%.1f мс</b>, найдено 100%% участков.<br>'
               '<b>Программа быстрее в %.0f раз</b>' % (tm, len(sel_l), tp, fp, fn, 100.0 * rec,
                                                        r['t_total'] * 1000, tm / max(r['t_total'], 1e-
6)))
        if 0 < rec < 1:
            txt += (' (при полной ручной обработке ≈ %.0f с, т.е. в %.0f раз)'
                    % (tm / rec, tm / rec / max(r['t_total'], 1e-6)))
        return txt + '.'


# ---------------------------------------------------------------------- панель
class OutageDock(QDockWidget):
    def __init__(self, tool):
        super().__init__('Анализ аварийных отключений – %s' % TAG)
        self.setObjectName('OutageDock')
        self.tool = tool
        self.canvas = iface.mapCanvas()
        self.map_tool = PickTool(self.canvas, self.on_pick)
        w = QWidget()
        v = QVBoxLayout(w)
        v.addWidget(QLabel('<b>1. Выбор элементов для отключения</b><br>'
                           '<small>«Выбор на карте»: ЛКМ — отключить/включить линию или узел.<br>'
                           'Или выделите объекты инструментом выделения и нажмите «Добавить
выделенные».</small>'))
        h = QHBoxLayout()
        self.ed_find = QLineEdit()
        self.ed_find.setPlaceholderText('Поиск линии / станции по названию…')
        self.ed_find.returnPressed.connect(self.on_find)
        b = QPushButton('Найти')
        b.clicked.connect(self.on_find)
        h.addWidget(self.ed_find)
        h.addWidget(b)
        v.addLayout(h)
        h = QHBoxLayout()
        self.b_pick = QPushButton('Выбор на карте')
        self.b_pick.setCheckable(True)
        self.b_pick.toggled.connect(self.on_pick_mode)
        b_sel = QPushButton('Добавить выделенные')
```

```
        b_sel.clicked.connect(self.on_add_selected)
        h.addWidget(self.b_pick)
        h.addWidget(b_sel)
        v.addLayout(h)
        self.lst = QListWidget()
        self.lst.setMaximumHeight(130)
        v.addWidget(self.lst)
        h = QHBoxLayout()
        b = QPushButton('Убрать из списка')
        b.clicked.connect(self.on_remove)
        h.addWidget(b)
        b = QPushButton('Сбросить всё')
        b.clicked.connect(self.on_reset)
        h.addWidget(b)
        v.addLayout(h)
        self.cb_auto = QCheckBox('Пересчитывать автоматически')
        self.cb_auto.setChecked(True)
        v.addWidget(self.cb_auto)
        b = QPushButton('▶️  Выполнить анализ отключения')
        f = QFont()
        f.setBold(True)
        b.setFont(f)
        b.setMinimumHeight(30)
        b.clicked.connect(self.on_run)
        v.addWidget(b)
        v.addWidget(QLabel('<b>2. Результат</b>'))
        self.txt = QTextEdit()
        self.txt.setReadOnly(True)
        v.addWidget(self.txt)
        h = QHBoxLayout()
        b = QPushButton('Показать на карте')
        b.clicked.connect(self.on_zoom)
        h.addWidget(b)
        b = QPushButton('Экспорт CSV')
        b.clicked.connect(self.on_export)
        h.addWidget(b)
        v.addLayout(h)
        g = QGroupBox('3. Сравнение с ручным методом')
        gv = QVBoxLayout(g)
        h = QHBoxLayout()
        b = QPushButton('Старт ручного замера')
        b.clicked.connect(self.on_manual_start)
        h.addWidget(b)
        b = QPushButton('Стоп и сравнить')
        b.clicked.connect(self.on_manual_stop)
        h.addWidget(b)
        gv.addLayout(h)
        b = QPushButton('Тест скорости (случайные аварии)')
        b.clicked.connect(self.on_bench)
        gv.addWidget(b)
        v.addWidget(g)
        self.setWidget(w)
        self.refresh_list()

    def refresh_list(self):
        self.lst.clear()
        for k in sorted(self.tool.off_e):
            self.lst.addItem(self.tool.label('line', k))
            self.lst.item(self.lst.count() - 1).setData(Qt.UserRole, ('line', k))
        for k in sorted(self.tool.off_n):
            self.lst.addItem(self.tool.label('node', k))
            self.lst.item(self.lst.count() - 1).setData(Qt.UserRole, ('node', k))

    def after_change(self):
        self.refresh_list()
        if self.cb_auto.isChecked():
            self.on_run()

    def on_pick_mode(self, on):
        if on:
            self.prev_tool = self.canvas.mapTool()
            self.canvas.setMapTool(self.map_tool)
        else:
            self.canvas.unsetMapTool(self.map_tool)
            if getattr(self, 'prev_tool', None):
                self.canvas.setMapTool(self.prev_tool)

    def on_pick(self, pt, tol, right):
        dst = self.tool.le.crs()
        src = self.canvas.mapSettings().destinationCrs()
```

```
        if src != dst:
            tr = QgsCoordinateTransform(src, dst, QgsProject.instance())
            p2 = tr.transform(QgsPointXY(pt.x() + tol, pt.y()))
            pt = tr.transform(pt)
            tol = abs(p2.x() - pt.x()) or tol
        if tol > PICK_MAX_TOL:
            iface.messageBar().pushWarning('Анализ отключений', 'Увеличьте масштаб карты для выбора
объекта')
            return
        kind, k = self.tool.pick(pt.x(), pt.y(), tol)
        if kind is None:
            iface.messageBar().pushInfo('Анализ отключений', 'Рядом с точкой нет линии или узла сети')
            return
        if kind == 'line':
            self.tool.toggle_line(k)
        else:
            self.tool.toggle_node(k)
        self.after_change()

    def on_find(self):
        es, ns = self.tool.find(self.ed_find.text())
        self.tool.le.selectByIds([self.tool.E[k]['fid'] for k in es])
        self.tool.ln.selectByIds([self.tool.N[k]['fid'] for k in ns])
        if not es and not ns:
            iface.messageBar().pushInfo('Анализ отключений', 'Ничего не найдено')
            return
        box = None
        for lyr in (self.tool.le, self.tool.ln):
            if lyr.selectedFeatureCount():
                b = lyr.boundingBoxOfSelected()
                box = b if box is None else (box.combineExtentWith(b) or box)
        if box is not None:
            box.grow(max(box.width(), box.height(), 500) * 0.1)
            dstc = self.canvas.mapSettings().destinationCrs()
            if self.tool.le.crs() != dstc:
                box = QgsCoordinateTransform(self.tool.le.crs(), dstc,
QgsProject.instance()).transformBoundingBox(box)
            self.canvas.setExtent(box)
            self.canvas.refresh()
        self.txt.setHtml('Найдено: линий %d, станций/узлов %d (выделены на карте). '
                         'Нажмите «Добавить выделенные», чтобы отключить их.' % (len(es), len(ns)))

    def on_add_selected(self):
        n = self.tool.add_selected()
        if not n:
            iface.messageBar().pushWarning('Анализ отключений',
                                           'Нет выделенных объектов в слоях «%s» / «%s»' %
(TITLE[L_EDGES], TITLE[L_NODES]))
        self.after_change()

    def on_remove(self):
        for it in self.lst.selectedItems():
            kind, k = it.data(Qt.UserRole)
            (self.tool.off_e if kind == 'line' else self.tool.off_n).discard(k)
        self.after_change()

    def on_reset(self):
        self.tool.reset()
        self.refresh_list()
        self.txt.setHtml('Сеть восстановлена в нормальный режим.')

    def on_run(self):
        self.tool.run()
        self.txt.setHtml(self.tool.report())

    def on_zoom(self):
        r = self.tool.dead_extent()
        if r:
            src = self.tool.le.crs()
            dst = self.canvas.mapSettings().destinationCrs()
            if src != dst:
                r = QgsCoordinateTransform(src, dst, QgsProject.instance()).transformBoundingBox(r)
            self.canvas.setExtent(r)
            self.canvas.refresh()

    def on_export(self):
        p = self.tool.export_csv()
        iface.messageBar().pushSuccess('Анализ отключений', 'Отчёт сохранён: ' + p)

    def on_manual_start(self):
```

```
        if not self.tool.off_e and not self.tool.off_n:
            QMessageBox.information(self, 'Ручной замер', 'Сначала выберите отключаемые элементы.')
            return
        self.cb_auto.setChecked(False)
        if self.b_pick.isChecked():
            self.b_pick.setChecked(False)
        self.tool.manual_start()
        self.txt.setHtml('<b>Ручной замер запущен.</b><br>Показаны только отключённые элементы
(красным). '
                         'Выделите инструментом выделения ВСЕ линии, которые, по-вашему, окажутся
обесточены '
                         '(Ctrl — добавить к выделению), затем нажмите «Стоп и сравнить».')

    def on_manual_stop(self):
        self.txt.setHtml(self.tool.manual_stop())
        self.cb_auto.setChecked(True)

    def on_bench(self):
        txt, rows = self.tool.benchmark()
        self.txt.setHtml(txt)

    def closeEvent(self, e):
        if self.b_pick.isChecked():
            self.b_pick.setChecked(False)
        super().closeEvent(e)


def start():
    """Точка входа: строит/загружает сеть и открывает панель."""
    global OUTAGE_TOOL, OUTAGE_DOCK
    prj = QgsProject.instance()
    if REBUILD:
        build_network()
    elif not prj.mapLayersByName(TITLE[L_EDGES]):
        if os.path.exists(OUT_GPKG):
            load_network_layers(OUT_GPKG)
        else:
            build_network()
    else:
        style_layers(*[prj.mapLayersByName(TITLE[n])[0] if prj.mapLayersByName(TITLE[n]) else None
                       for n in (L_EDGES, L_NODES, L_MASTS)])
    OUTAGE_TOOL = OutageTool()
    OUTAGE_TOOL.run()
    try:
        OUTAGE_DOCK.close()
        iface.removeDockWidget(OUTAGE_DOCK)
    except Exception:
        pass
    OUTAGE_DOCK = OutageDock(OUTAGE_TOOL)
    iface.addDockWidget(Qt.RightDockWidgetArea, OUTAGE_DOCK)
    OUTAGE_DOCK.txt.setHtml('Сеть «%s» загружена: %d рёбер, %d узлов, %d источников питания, %d
опор.<br>'
                            'Найдите объект по названию или включите «Выбор на карте».'
                            % (TAG, len(OUTAGE_TOOL.E), len(OUTAGE_TOOL.N), len(OUTAGE_TOOL.G.sources),
OUTAGE_TOOL.n_masts))
    le = OUTAGE_TOOL.le
    ext = le.extent()
    dst = iface.mapCanvas().mapSettings().destinationCrs()
    if le.crs() != dst:
        ext = QgsCoordinateTransform(le.crs(), dst, prj).transformBoundingBox(ext)
    iface.mapCanvas().setExtent(ext)
    iface.mapCanvas().refresh()
    return OUTAGE_TOOL


OUTAGE_TOOL = globals().get('OUTAGE_TOOL')
OUTAGE_DOCK = globals().get('OUTAGE_DOCK')
start()
print('Готово. Объект OUTAGE_TOOL доступен в консоли: OUTAGE_TOOL.disconnect(lines=[...], nodes=[...]),
'
      'OUTAGE_TOOL.find("название"), OUTAGE_TOOL.reset(), OUTAGE_TOOL.benchmark()')
```

Приложение Б

(справочное)

Руководство оператора

Б.1 Назначение программы. Программа предназначена для оперативного анализа последствий аварийных и плановых отключений линий и узлов электрической сети: определения обесточенных участков, их выделения на карте и формирования отчёта.

Б.2 Условия выполнения. ПЭВМ под управлением Windows, Linux или macOS; QGIS версии 3.x (проверено на 3.40.3); оперативная память не менее 8 ГБ для сети общенационального масштаба; исходные данные в прямоугольной метрической системе координат.

Б.3 Установка. Скопировать папку outage_tool с файлом outage_qgis.py в папку проекта. В блоке НАСТРОЙКИ выбрать набор данных PROFILE ('norge' – вся сеть Норвегии из файловой геобазы, 'region' – фрагмент сети) и при необходимости исправить базовый путь BASE; параметр BBOX ограничивает обработку прямоугольной областью, LOAD_MASTS = False отключает загрузку опор.

Б.4 Запуск. Открыть консоль Python QGIS (меню «Модули» → «Консоль Python», Ctrl+Alt+P) и выполнить команду:

```
exec(open(r'C:\Users\ASUS\Desktop\elktro\outage_tool\outage_qgis.py', encoding='utf-8').read())
```

При первом запуске строится топологическая сеть (для всей Норвегии – около 8 мин; ход работы отображается в строке состояния) и создаётся файл elektro_topo_norge.gpkg; при последующих запусках готовая сеть загружается за несколько секунд. В проект добавляется группа «Анализ аварийных отключений – Норвегия» с тремя слоями, справа от карты открывается панель управления.

Б.5 Выбор отключаемых элементов. Ввести часть наименования линии или станции в поле поиска и нажать «Найти» – найденные объекты будут выделены и приближены; нажать «Добавить выделенные». Альтернативно – нажать «Выбор на карте» и щёлкнуть левой кнопкой мыши по линии или узлу (при масштабе крупнее примерно 1:800 000); повторный щелчок возвращает элемент в работу. Можно

также выделить объекты стандартным инструментом QGIS и нажать «Добавить выделенные».

Б.6 Анализ и результат. При установленном флажке «Пересчитывать автоматически» анализ выполняется после каждого изменения списка; иначе – по кнопке «Выполнить анализ отключения». Результат отображается цветом на карте (таблица 3.6) и сводкой в окне «Результат». Кнопка «Показать на карте» масштабирует карту к зоне обесточения, «Экспорт CSV» сохраняет отчёт, «Сбросить» возвращает сеть в нормальный режим.

Б.7 Сравнение с ручным методом. Выбрать отключаемые элементы, нажать «Старт ручного замера», вручную выделить линии, которые окажутся обесточены, нажать «Стоп и сравнить». Кнопка «Тест скорости» выполняет серию случайных сценариев.

Б.8 Изменение параметров. Параметры топологии задаются в словаре DEFAULT_PARAMS. После изменения установить REBUILD = True или выполнить в консоли build_network(); start(). Источники питания можно изменить, отредактировав поле is_source слоя узлов и выполнив OUTAGE_TOOL.reload().

Б.9 Сообщения. «Слои сети не загружены» – выполнить build_network(); «Не найдены исходные линии» – проверить путь к данным; «Увеличьте масштаб карты для выбора объекта» – приблизить карту или воспользоваться поиском; «Рядом с точкой нет линии или узла сети» – щёлкнуть ближе к объекту.

Приложение В

(справочное)

Протокол построения топологической модели электросети Норвегии

```
Отчёт о построении топологии (2026-09-29 22:33), источник: C:\Users\ASUS\Desktop\elktro\
Energi_0000_Norge_25833_Nettanlegg_FGDB\Energi_0000_Norge_25833_Nettanlegg_FGDB.gdb
  BBOX = None
  параметр SNAP_TOL = 10.0
  параметр TJUNC_TOL = 10.0
  параметр STATION_GROUP = 80.0
  параметр STATION_RADIUS = 100.0
  параметр MAST_TOL = 25.0
  параметр MIN_LEN = 0.01
  параметр PARALLEL_COS = 0.94
  параметр EXT_VOLT = 110.0
  параметр AUTO_FEED_RADIUS = 1500.0
  source_lines: 154506
  multipart_exploded: 475
  short_removed: 86
  duplicates_removed: 554
  dup_vertices_removed: 0
  endpoints_snapped: 48911
  nodes_after_snap: 158955
  t_junctions: 492
  short_edges_collapsed: 5845
  stations: 1595
  bus_edges: 2835
  stations_isolated: 280
  components_initial: 10054
  components_with_source_initial: 521
  virtual_feeds: 2042
  components: 8012
  components_with_source: 521
  edges: 154722
  line_edges: 149845
  nodes: 159543
  node_types: {'STATION': 1595, 'EXTERNAL': 390, 'BRANCH': 28501, 'PASS': 82671, 'END': 46386}
  len_km: 97524.61615048589
  len_fed_km: 75154.91216616711
  masts: 734293
  masts_bound: 718210
  build_sec: 194.3683559000492
  total_sec (с чтением и записью): 492.2
```

Список использованной литературы

1. Российская Федерация. Законы. Об электроэнергетике : Федеральный закон № 35-ФЗ : [принят Государственной думой 21 февраля 2003 года : одобрен Советом Федерации 12 марта 2003 года]. – Текст : электронный // КонсультантПлюс : справочная правовая система. – URL: http://www.consultant.ru/document/cons_doc_LAW_41502/ (дата обращения: 29.09.2026).

2. Об утверждении Правил оперативно-диспетчерского управления в электроэнергетике : постановление Правительства Российской Федерации от 27.12.2004 № 854. – Текст : электронный // КонсультантПлюс : справочная правовая система. – URL: http://www.consultant.ru (дата обращения: 29.09.2026).

3. Об утверждении методических указаний по расчету уровня надежности и качества поставляемых товаров и оказываемых услуг для организации по управлению единой национальной (общероссийской) электрической сетью и территориальных сетевых организаций : приказ Министерства энергетики Российской Федерации от 29.11.2016 № 1256. – Текст : электронный. – URL: https://rulaws.ru/acts/Prikaz-Minenergo-Rossii-ot-29.11.2016-N-1256/ (дата обращения: 29.09.2026).

4. Produktark: Nettanlegg / Norges vassdrags- og energidirektorat (NVE). – Text : electronic // Geonorge. – URL: https://register.geonorge.no/data/documents/Produktark_nettanlegg_v2_nettanlegg_.pdf (accessed: 29.09.2026).

5. Kraftlinjer : datasett / Norges vassdrags- og energidirektorat (NVE). – Text : electronic // data.norge.no. – URL: https://data.norge.no/en/datasets/5a99af63-9ae1-3635-b686-1dff6aceddba/kraftlinjer (accessed: 29.09.2026).

6. Лобовко, В. В. Геоинформационные системы : учебное пособие / В. В. Лобовко, В. И. Билан. – Санкт-Петербург : ВКА имени А. Ф. Можайского, 2016. – 242 с. – Текст : непосредственный.

7. Журкин, И. Г. Итерационная модель представления линейных пространственных данных / И. Г. Журкин, С. В. Наумов. – Текст : непосредственный // Информационные технологии. – 2003. – № 6. – С. 11–16.

8. ArcGIS Utility Network. – Esri. – Text : electronic. – URL: https://www.esri.com/en-us/arcgis/products/arcgis-utility-network/overview (accessed: 29.09.2026).

9. ГИС Zulu : геоинформационная система / ООО «Политерм». – Текст : электронный. – URL: https://www.politerm.com/products/geo/zulu/ (дата обращения: 29.09.2026).

10. Идельчик, В. И. Электрические системы и сети : учебник для вузов / В. И. Идельчик. – Москва : Энергоатомиздат, 1989. – 592 с. – Текст : непосредственный.

11. Правила устройства электроустановок. – 7-е изд. – Москва : НЦ ЭНАС, 2004. – 552 с. – Текст : непосредственный.

12. Руденко, Ю. Н. Надежность систем энергетики / Ю. Н. Руденко, И. А. Ушаков. – 2-е изд., перераб. и доп. – Новосибирск : Наука, 1989. – 328 с. – Текст : непосредственный.

13. Billinton, R. Reliability Evaluation of Power Systems / R. Billinton, R. N. Allan. – 2nd ed. – New York : Plenum Press, 1996. – 514 p. – Text : unmediated.

14. ГОСТ 32144–2013. Электрическая энергия. Совместимость технических средств электромагнитная. Нормы качества электрической энергии в системах электроснабжения общего назначения : межгосударственный стандарт : издание официальное : дата введения 2014-07-01. – Москва : Стандартинформ, 2014. – 16 с. – Текст : непосредственный.

15. Об утверждении Правил недискриминационного доступа к услугам по передаче электрической энергии и оказания этих услуг … и Правил технологического присоединения энергопринимающих устройств потребителей электрической энергии … : постановление Правительства Российской Федерации от 27.12.2004 № 861. – Текст : электронный // КонсультантПлюс : справочная правовая система. – URL: http://www.consultant.ru (дата обращения: 29.09.2026).

16. IEEE Std 1366–2012. IEEE Guide for Electric Power Distribution Reliability Indices. – New York : IEEE, 2012. – 43 p. – Text : unmediated.

17. ГОСТ Р 52438–2005. Географические информационные системы. Термины и определения : национальный стандарт Российской Федерации : издание официальное : дата введения 2006-07-01. – Москва : Стандартинформ, 2006. – 16 с. – Текст : непосредственный.

18. ДеМерс, М. Н. Географические информационные системы. Основы / М. Н. ДеМерс ; перевод с английского. – Москва : Дата+, 1999. – 490 с. – Текст : непосредственный.

19. Горбачев, В. Г. Что такое «топологические» отношения в цифровой картографии и для чего топологические отношения нужны в геоинформатике? / В. Г. Горбачев. – Текст : электронный // Интегро : [сайт]. – URL: http://www.integro.ru (дата обращения: 29.09.2026).

20. Власов, М. Ю. Концептуальные топологические отношения / М. Ю. Власов, В. Г. Горбачев. – Текст : непосредственный // Информационный бюллетень ГИС-Ассоциации. – 1996. – № 2 (4). – С. 17.

21. Садыков, С. С. Формальное определение топологических отношений между картографическими объектами в ГИС / С. С. Садыков, Д. Е. Андрианов, С. В. Еремеев. – Текст : непосредственный // Обработка информации: методы и системы : сборник научных статей. – Москва : Горячая линия – Телеком, 2003.

22. Андрианов, Д. Е. Алгоритм представления сложных топологических отношений в геоинформационных системах / Д. Е. Андрианов, А. В. Булаев. – Текст : непосредственный // Труды Российского научно-технического общества радиотехники, электроники и связи имени А. С. Попова. Серия: Цифровая обработка сигналов и её применение. – Москва, 2006. – Вып. VIII-2. – С. 604–606.

23. QGIS 3.40 User Guide. – Text : electronic // QGIS Documentation. – URL: https://docs.qgis.org/3.40/en/docs/user_manual/ (accessed: 29.09.2026).

24. Лурье, И. К. Геоинформационное картографирование. Методы геоинформатики и цифровой обработки космических снимков : учебник / И. К. Лурье. – Москва : КДУ, 2010. – 424 с. – Текст : непосредственный.

25. Харари, Ф. Теория графов / Ф. Харари ; перевод с английского В. П. Козырева ; под редакцией Г. П. Гаврилова. – Москва : Мир, 1973. – 300 с. – Текст : непосредственный.

26. Алгоритмы: построение и анализ / Т. Х. Кормен, Ч. И. Лейзерсон, Р. Л. Ривест, К. Штайн. – 3-е изд. – Москва : Вильямс, 2013. – 1328 с. – Текст : непосредственный.

27. ArcGIS Network Analyst. – Esri. – Text : electronic. – URL: https://www.esri.com/en-us/arcgis/products/arcgis-network-analyst (accessed: 29.09.2026).

28. MapInfo Pro : desktop GIS. – Precisely. – Text : electronic. – URL: https://www.precisely.com/product/precisely-mapinfo/mapinfo-pro (accessed: 29.09.2026).

29. GRASS GIS : v.net – Performs network maintenance. – Text : electronic // GRASS GIS manual. – URL: https://grass.osgeo.org/grass-stable/manuals/v.net.html (accessed: 29.09.2026).

30. OpenFileGDB – ESRI File Geodatabase vector driver. – Text : electronic // GDAL documentation. – URL: https://gdal.org/drivers/vector/openfilegdb.html (accessed: 29.09.2026).

31. PyQGIS 3.40 developer cookbook. – Text : electronic // QGIS Documentation. – URL: https://docs.qgis.org/3.40/en/docs/pyqgis_developer_cookbook/ (accessed: 29.09.2026).

32. Программный комплекс RastrWin3. – Текст : электронный. – URL: https://www.rastrwin.ru/ (дата обращения: 29.09.2026).

33. ESRI Shapefile Technical Description : an ESRI White Paper. – Redlands : Environmental Systems Research Institute, 1998. – 34 p. – Text : electronic. – URL: https://www.esri.com/content/dam/esrisites/sitecore-archive/Files/Pdfs/library/whitepapers/pdfs/shapefile.pdf (accessed: 29.09.2026).

34. OGC GeoPackage Encoding Standard. – Open Geospatial Consortium. – Text : electronic. – URL: https://www.geopackage.org/spec/ (accessed: 29.09.2026).

35. Python 3 documentation / Python Software Foundation. – Text : electronic. – URL: https://docs.python.org/3/ (accessed: 29.09.2026).

36. NetworkX documentation. – Text : electronic. – URL: https://networkx.org/documentation/stable/ (accessed: 29.09.2026).

37. Оре, О. Теория графов / О. Оре ; перевод с английского. – 2-е изд. – Москва : Наука, 1980. – 336 с. – Текст : непосредственный.

38. Кристофидес, Н. Теория графов. Алгоритмический подход / Н. Кристофидес ; перевод с английского. – Москва : Мир, 1978. – 432 с. – Текст : непосредственный.

39. Асанов, М. О. Дискретная математика: графы, матроиды, алгоритмы : учебное пособие / М. О. Асанов, В. А. Баранский, В. В. Расин. – 5-е изд. – Санкт-Петербург : Лань, 2020. – 364 с. – Текст : непосредственный.

40. Tarjan, R. Depth-first search and linear graph algorithms / R. Tarjan. – Text : unmediated // SIAM Journal on Computing. – 1972. – Vol. 1, No. 2. – P. 146–160.

41. RFC 9562. Universally Unique IDentifiers (UUIDs) / K. Davis, B. Peabody, P. Leach. – Internet Engineering Task Force, 2024. – Text : electronic. – URL: https://www.rfc-editor.org/rfc/rfc9562 (accessed: 29.09.2026).

42. Dijkstra, E. W. A note on two problems in connexion with graphs / E. W. Dijkstra. – Text : unmediated // Numerische Mathematik. – 1959. – Vol. 1. – P. 269–271.

43. ГОСТ 19.402–78. Единая система программной документации. Описание программы : межгосударственный стандарт : издание официальное. – Москва : Стандартинформ, 2010. – 3 с. – Текст : непосредственный.

44. ГОСТ 19.105–78. Единая система программной документации. Общие требования к программным документам : межгосударственный стандарт : издание официальное. – Москва : Стандартинформ, 2010. – 4 с. – Текст : непосредственный.

45. SQLite Documentation. – Text : electronic. – URL: https://www.sqlite.org/docs.html (accessed: 29.09.2026).

46. ГОСТ 19.701–90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила

выполнения : межгосударственный стандарт : издание официальное. – Москва : Стандартинформ, 2010. – 22 с. – Текст : непосредственный.

47. ГОСТ 19.505–79. Единая система программной документации. Руководство оператора. Требования к содержанию и оформлению : межгосударственный стандарт : издание официальное. – Москва : Стандартинформ, 2010. – 3 с. – Текст : непосредственный.